

分类号: TP393.0
论文编号: 2021022313

密级: 公开

贵 州 大 学
2024 届硕士研究生学位论文

多集群异构算力调度优化技术研究实现

学科专业: 软件工程
研究方向: 不区分研究方向
导 师: 蒋朝惠
研 究 生: 杨守瞻

中国 ▪ 贵州 ▪ 贵阳
2024 年 06 月

硕士学位论文答辩委员会名单

多集群异构算力调度优化技术研究实现

答辩人：杨守瞻

答辩委员会委员：

贵州大学 教 授：王以松（主席）

贵州大学 教 授：王丽会

贵州大学 副教授：段 迅

贵州大学 副教授：崔允贺

贵州师范大学 教 授：张仁津

答辩秘书：陈婷婷 助理实验师

答辩时间：2024 年 06 月 04 日

答辩地点：贵州大学博学楼 13 楼

摘要

在以 Karmada 和 Kubernetes 技术实现的算力调度系统中，默认调度策略处理多样化任务时面临执行效率低、全局多维资源（包括 CPU、GPU、内存、磁盘 IO、网络带宽）负载不平衡等问题。针对上述挑战，本文通过多维细粒度算力度量模型量度节点性能，设计高效的资源监控策略，并构建多目标优化算力调度模型，以改进默认的调度策略，有效提升任务处理效率和负载平衡。本文主要工作如下：

针对任务特性无法与算力资源有效匹配，导致任务执行效率低的问题，提出多维细粒度算力度量模型。从节点的细粒度硬件参数出发，构建包括通用算力、智能算力、存储能力及网络能力 4 个维度的算力度量指标体系。采用层次分析法（Analytic Hierarchy Process, AHP）确定各维度下的指标权重，并基于逼近理想解排序法（Technique for Order Preference by Similarity to Ideal Solution, TOPSIS）决策模型对各维度下的算力指标进行综合评分，以此作为节点各维度的算力值，作为算力调度模型的决策依据。

针对 Prometheus 联邦多集群监控下带来极大跨集群网络流量，导致系统性能低的问题，设计多集群监控框架（Multi-Cluster Monitor, MCM）。在成员集群中，使用 PromQL 从 Prometheus 中汇总监控数据，提高监控数据粒度；向控制平面集群上报监控数据时，过滤掉数据中无用的标签值、避免未变化的数据上报、压缩数据，以减少多集群监控产生的跨集群网络流量。

针对 Kubernetes 默认调度策略只考虑 CPU 和内存两个指标，且无法满足多类型任务调度的特定需求的问题，设计多目标优化调度策略。引入算力值估算任务执行时间；纳入 GPU、磁盘 IO、网络带宽指标，构建多维资源负载平衡模型；构建多目标优化调度模型，并使用 C-TAEA（Two-Archive Evolutionary Algorithm for Constrained Multiobjective Optimization）算法求解出最佳调度方案。

在上述关键技术攻关的基础上，实现多集群异构算力调度系统，并对系统进行测试。实验结果表明，本文提出的算力度量模型在不同算力维度下能有效识别节点间的性能差异；在能够获得相同监控数据条件下，与 Prometheus 联邦监控相比，本文设计的多集群监控策略减少了 89.2%至 93.7%的跨集群网络流量；

与 Karmada+Kubernetes 默认的调度策略相比，在本文设计的算力调度策略下，批次任务的平均完成时间减少了 15.36%，最大完工时间减少了 21.88%，且能够保证较高的多维资源负载平衡。

关键词：算力度量；算力调度；多集群监控；Kubernetes；Karmada

ABSTRACT

In computational scheduling systems implemented with Karmada and Kubernetes technologies, the default scheduling strategy often faces challenges such as low execution efficiency and imbalanced global multi-dimensional resource loads (including CPU, GPU, memory, disk IO, and network bandwidth) when handling diverse tasks. To address these challenges, this paper proposes a multi-dimensional fine-grained computing power measurement model to evaluate node performance, designs an efficient resource monitoring strategy, and constructs a multi-objective optimization scheduling model to improve the default scheduling strategy, thereby effectively enhancing task processing efficiency and load balancing. The main contributions of this thesis are as follows:

To address the issue of task characteristics not effectively matching computing resources, leading to low task execution efficiency, a multi-dimensional fine-grained computing power measurement model is proposed. Starting from the fine-grained hardware parameters of nodes, a computing power metric system is constructed, including four dimensions: general computing power, intelligent computing power, storage capability, and network capability. The Analytic Hierarchy Process (AHP) is used to determine the weight of each dimension, and the Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) decision model is used to comprehensively score the computing power metrics of each dimension. These scores are used as the computing power values of the nodes, serving as the basis for the scheduling decision model.

To address the issue of significant cross-cluster network traffic under Prometheus federated multi-cluster monitoring, which leads to low system performance, a Multi-Cluster Monitor (MCM) framework is designed. In member clusters, PromQL is used to aggregate monitoring data from Prometheus, improving the granularity of monitoring data. When reporting monitoring data to the control plane cluster, useless label values are filtered out, unchanged data reports are avoided, and the reported data is compressed to reduce cross-cluster network traffic generated by multi-cluster monitoring.

To address the issue that Kubernetes' default scheduling strategy only considers CPU and memory metrics and cannot meet the specific needs of multi-type task scheduling, a multi-objective optimization scheduling strategy is designed. The strategy introduces computing power values to estimate task execution time, incorporates GPU, disk IO, and network bandwidth metrics, constructs a multi-dimensional resource load balancing model, and constructs a multi-objective optimization scheduling model. The C-TAEA (Two-Archive Evolutionary Algorithm for Constrained Multiobjective Optimization) algorithm is used to solve for the optimal scheduling solution.

Based on the above key technologies, a multi-cluster heterogeneous computing power scheduling system is implemented and tested. The experimental results show that the proposed computing power measurement model can effectively identify performance differences between nodes in different computing power dimensions. Under the same monitoring data conditions, the proposed multi-cluster monitoring strategy reduces cross-cluster network traffic by 89.2% to 93.7% compared to Prometheus federated monitoring. Compared to the default Karmada+Kubernetes scheduling strategy, the proposed computing power scheduling strategy reduces the average completion time of batch tasks by 15.36% and the maximum completion time by 21.88%, while ensuring a high level of multi-dimensional resource load balancing.

Key words: Computational power measurement, computational resource scheduling, multi-cluster monitoring, Kubernetes, Karmada

目录

摘要.....	I
ABSTRACT.....	III
第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状与发展趋势.....	2
1.2.1 算力度量的研究现状.....	3
1.2.2 基于 Kubernetes 的算力调度研究现状.....	4
1.3 本文的主要工作.....	6
1.4 本文的组织结构.....	7
第二章 相关技术与理论基础.....	8
2.1 Kubernetes 容器编排框架.....	8
2.1.1 Kubernetes 概述.....	8
2.1.2 Kubernetes 调度.....	9
2.2 Karmada 多集群编排系统.....	11
2.2.1 Karmada 架构.....	11
2.2.2 Karmada 操作流程.....	12
2.2.3 Karmada 调度.....	12
2.3 Prometheus 监控技术.....	13
2.3.1 Prometheus 简介.....	13
2.3.2 Node Exporter.....	14
2.3.3 Nvidia GPU exporter.....	15
2.4 其他相关技术.....	15
2.4.1 Docker 容器技术.....	15
2.4.2 NVIDIA device plugin.....	17
2.5 本章小结.....	18
第三章 多集群异构算力调度系统总体分析.....	19
3.1 应用场景分析.....	19

3.2 需求分析.....	22
3.2.1 系统功能性需求分析.....	22
3.2.2 系统非功能性需求分析.....	23
3.3 本文待解决的关键问题及对应的解决方案.....	24
3.4 本章小结.....	24
第四章 系统设计.....	25
4.1 系统总体设计.....	25
4.1.1 系统整体架构设计.....	25
4.1.2 系统功能模块.....	26
4.2 多维细粒度算力度量模块设计.....	28
4.2.1 算力指标体系构建.....	28
4.2.2 算力度量.....	31
4.3 资源监控模块设计.....	35
4.3.1 单集群资源监控.....	35
4.3.2 多集群资源监控.....	36
4.4 资源接入与限制模块设计.....	39
4.4.1 GPU 资源接入.....	39
4.4.2 网络带宽资源限制.....	40
4.4.3 磁盘 IO 资源限制	41
4.5 算力调度模块设计.....	42
4.5.1 算力调度模型构建.....	42
4.5.2 多目标优化算力调度.....	46
4.6 本章小结.....	50
第五章 系统实现与测试.....	51
5.1 系统部署.....	51
5.1.1 系统部署拓扑.....	51
5.1.2 系统硬件环境.....	51
5.1.3 系统软件环境.....	52

5.1.4 关键软件安装.....	53
5.2 系统核心功能实现.....	54
5.3 系统测试.....	60
5.3.1 系统功能测试.....	60
5.3.2 算力度量有效性测试.....	61
5.3.3 多集群监控性能测试.....	63
5.3.4 算力调度效果测试.....	66
5.4 本章小结.....	71
第六章 总结与展望.....	72
6.1 总结.....	72
6.2 展望.....	73
致谢.....	74
参考文献.....	75
附录.....	78
A 作者在硕士研究生期间参加的科研项目及成果.....	78
B 图版.....	79
C 表版.....	81

第一章 绪论

1.1 研究背景及意义

在当前科技飞速发展的时代，算力资源已成为科学研究、工业设计和金融分析等多个关键技术领域创新与进步的重要基础。随着大数据、人工智能和云计算技术的广泛应用，新兴产业和智能应用如智能驾驶^[1]、元宇宙^[2]、以及基于人工智能的内容生成（Artificial Intelligence Generated Content, AIGC）^[3]等陆续涌现，对计算资源的需求呈现出前所未有的增长趋势。如图 1-1 所示，以 AIGC 产业为例，OpenAI 在 2018 年将其用于深度学习训练的单个 Kubernetes 集群节点从 500 个增加到了 2500 个；2021 年初，为有效支持 GPT-3、CLIP、DALL·E 等大模型应用运营，OpenAI 又进一步将 Kubernetes 集群扩展到了 7500 个节点^[4]。如此迅猛的扩展不仅反映了对算力总量的广泛需求，同时也凸显了在提升算力使用效率及智能化调度方面不断探索与优化的重要性。

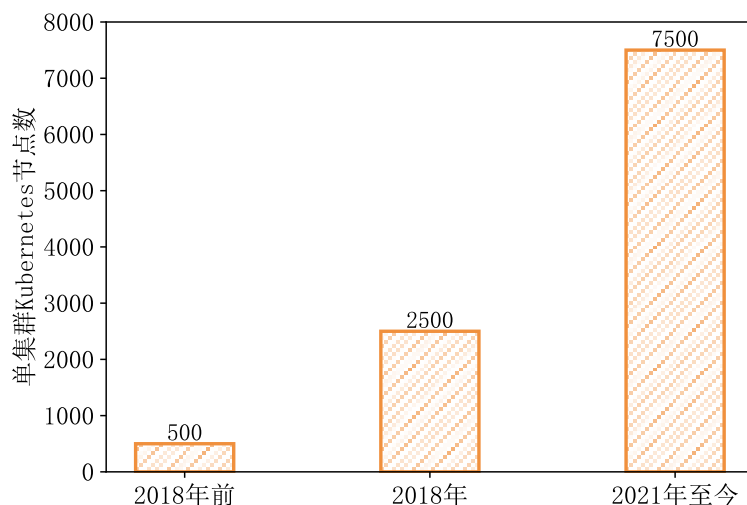


图 1-1 OpenAI 单集群 Kubernetes 历年节点数

当前，各国都在推广算力基础设施的布局与发展。2020 年，美国国家科学技术委员会发布了《开拓未来先进计算生态系统：战略计划》，计划构建和升级国家级高性能算力设施，包括数据中心、超级计算等^[5]。2021 年，国家发改委联合多个部门印发了《全国一体化大数据中心协同创新体系算力枢纽实施方案》，提出布局算力网络国家枢纽节点，并启动“东数西算”战略工程^[6]。同年，欧盟委员会发布《2030 数字指南针：欧洲数字十年之路》，提出要建设安全、高性

能的可持续数字化基础设施，加强云基础设施和算力的布局^[7]。2023 年，国家发改委等部门联合印发《关于深入实施“东数西算”工程加快构建全国一体化算力网的实施意见》，提出要促进多元异构算力融合发展、构建跨区域算力调度体系等战略^[8]。

在产业界，各互联网公司根据自身业务打造算力调度平台，并提供构建算力调度平台的技术路线和思路。2023 年，中国移动研究院等公司在《分布式异构智能算力的管理和调度技术研究报告》中指出，在异构算力管理和调度层面，多以容器技术基于 Kubernetes 定制化研发和实现对任务的高效编排和调度，为上层功能模块提供算力支撑^[9]。同年，为了解决传统云原生存在多云限制和地域限制，华为云 UCS 发布了多集群管理工具 Karmada，提供了多云算力的统一管理和调度能力^[10]。2024 年，中泰证券在《支持异构算力纳管的大数据云原生平台的研究与实践》中基于 Kubernetes 构建大数据算力平台，并使用 Karmada 构建统一的多集群控制平面，实现了多集群的统一管理和调度^[11]。同年，360 智汇云基于 Karmada 进行了二次开发，实现了多集群环境下闲置算力的分享^[12]。

由此看来，随着国家的支持和推广，算力基础设施得以扩大和不断完善，致使各数据中心算力节点分散，节点之间缺乏有效协同，任务与算力资源之间的分配和调度机制存在缺陷，导致任务执行效率低、资源利用率不均衡。在此背景下，对算力及其调度机制的深入研究显得尤为重要，这直接关系到计算资源的有效利用，也在降低能耗、促进技术创新、加快科学研究进度等方面提供了潜在价值。同时，在产业界中，以 Kubernetes 打造算力底座，Karmada 实现多集群管理和调度为本文提供了实现算力调度系统的启发，并在此基础上优化任务与算力资源的匹配，以提高任务的执行效率和资源利用率。

1.2 国内外研究现状与发展趋势

近年来，各行业正向数字化转型，衍生出智能驾驶、元宇宙、人工智能内容生成等新兴产业的相继问世，对算力质量、算力规模的需求量正呈现指数级别的增长，各国都在实施大规模的算力基础设施建设^[13]，为各行业数字化赋能。随着国家重点工程“东数西算”^[14]的启动，对算力设施的布局使得算力资源呈

现较大的异构性，业界正需要一种能够正确标识和准确度量算力的标准，使得任务能够和与之兼容的算力互相匹配，以最大程度利用算力资源，使异构算力资源能够得到有效分配。在满足算力有效兼容的基础上，为了保证算力资源对外提供服务的稳定性，应避免算力资源因局部负载过高而瘫痪，导致算力资源的可用性降低。在算力调度系统层面，业界普遍使用基于 Kubernetes 的解决方案实现算力的管理与调度^[15-17]，因此，本文从算力度量与基于 Kubernetes 的算力调度两个方面介绍国内外研究现状。

1.2.1 算力度量的研究现状

算力是计算机设备所具备的计算性能和能力，广义的算力包括通用算力、智能算力、存储能力和网络能力^[18]，通用算力是指以中央处理器（Central Processing Unit, CPU）和动态随机存取存储器（Dynamic Random Access Memory, DRAM）的组合，为基础任务提供基本运算环境的计算能力；智能算力是指以图形处理器（Graphics Processing Unit, GPU）^[19]、张量处理单元（Tensor Processing Unit, TPU）^[20]、应用特定集成电路（Application Specific Integrated Circuit, ASIC）^[21]、现场可编程门阵列（Field Programmable Gate Array, FPGA）^[22]等为代表的面向人工智能智能芯片的计算能力；存储能力和网络能力则分别表示物理磁盘和网卡的性能。算力度量是通过某种标准对算力节点实际性能进行衡量的方法。

随着算力网络背景的提出，目前已经存在算力度量方法研究的相关工作。Emma P G^[23]等人提出使用每条指令的时钟周期数（Clocks Per Instruction, CPI）来衡量 CPU 的性能，直到今天，CPI 仍被许多行业作为衡量 CPU 性能的方式。柴若楠^[18]等人从基础算力、智能算力、存储能力和网络能力四个维度将算力资源的静态指标与动态指标结合，度量算力节点的综合算力；使用 CPU 空闲率、GPU 空闲率、磁盘剩余量、网络吞吐量作为动态指标，这些动态指标属于容量指标而不是性能指标，此方式在通用任务场景下能够有效标识算力节点的算力，在调度过程中能够保证系统的稳定性，但未能标识算力节点的真实性能。祝淑琼^[24]等人提出一种面向物联网端侧设备的算力度量架构，分别从计算、存储、通信、功耗和电源五个维度进行了算力的标识，考虑了芯片类型、运算精度、

内存大小、磁盘容量、网络协议类型和带宽、功耗、设备充电状态与剩余电量等指标，并在仿真实验下验证了其度量方法的有效性。乔楚^[25]使用算力单元的数量（如内核数、虚拟机数、容器数、计算单元数）站在宏观的角度来衡量数据中心总体算力水平。夏天豪^[26]等人基于强化学习技术构建马尔科夫决策过程来对计算任务的算力需求进行量化，该方法针对任务而不是针对算力节点。Li J^[27]等人描述了一个算力网络（Computing Power Network, CPN）框架，分析了 CPN 的不同角色及其对 CPN 计算能力资源的关注点，从运营商的角度提出了一个算力资源模型，并使用计算任务的完成时间来衡量 CPN 的计算能力。

综上所述，目前在算力度量的研究中存在一些问题。首先，度量标准并不统一，不同场景下缺乏一致的度量体系，这降低了度量方法在通用场景下的适应性。其次，在节点级别的算力度量中，将如 CPU 空闲率和磁盘剩余量等资源容量指标纳入度量，可能会使度量结果与节点的实际性能存在较大的偏差。这种偏差在执行算力调度时可能导致任务与算力资源的匹配度不高，从而影响任务的执行效率。

1.2.2 基于 Kubernetes 的算力调度研究现状

在云计算领域，Kubernetes 已成为一个重要的里程碑，自从 Google 在 2014 年发布 Kubernetes 以来，衍生出了微服务架构，实现了容器化应用的自动部署、编排和管理。随着高性能计算、大数据、机器学习训练任务迁移上云^[28]，以及大模型应用的横空出世，对大规模算力资源的集中管理、自动化分配和优化提出了新的挑战。在此背景下，Kubernetes 展示出其无与伦比的适应性和可扩展性，成为了解决这些挑战的关键工具。然而，Kubernetes 默认调度策略在面对任务复杂性和算力资源的动态性时存在较大的局限性^[29]，例如无法对 GPU 资源进行有效分配，导致 Pod 独占显卡，仅支持对 CPU 和内存资源的管理和分配，无法感知除 CPU、内存以外的资源等。因此，涌现了诸多使用先进调度策略来实现 Kubernetes 中资源的高效分配。本文从提高任务执行效率、高效资源利用以及联合前两者的多目标优化三方面的国内外研究现状进行分析。

（1）针对提高任务执行效率的研究现状

Menouer T^[30]等人提出一种全新的 Kubernetes 调度策略 KCSS，其目的是通

过尽可能高效地调度用户提交的任务来降低任务时间和提高功耗方面的性能，对于每个新提交的任务，KCSS使用多标准决策分析技术，根据云基础设施状态以及用户的需求来选择最佳节点，替代了默认的 Kubernetes 调度策略。Han R^[31]等人在边缘云场景下，提出了 EdgeTuner 调度算法，用于优化动态边缘云工作负载的资源分配和调度，以减少作业延迟，其核心组件是一个用于运行时调度算法调优的深度强化学习（Deep Reinforcement Learning, DRL）代理，以及一个集群模拟器，用于加速漫长的 DRL 训练过程。

（2）针对高效资源利用的研究现状

Zhong Z^[32]等人提出了一种异构任务分配策略，用于在具有弹性算力资源的基于 Kubernetes 云计算基础设施中实现经济高效的容器编排，该策略具有以下特点：支持异构作业的配置、通过自动缩放算法调整集群大小以关闭未被充分利用的虚拟机实例，并重新分配相关任务而保持任务执行进度的重调度机制。刘志彬^[33]等人提出一种 CPU/GPU 异构资源调度策略，自定义 Kubernetes 调度器的过滤和打分阶段，保证任务类型与算力资源匹配的同时减少 GPU 资源碎片，有效避免了 GPU 资源竞争。李华东^[34]等人提出了面向非 GPU 应用提出了 MBCGT 算法，引入了实时监控，把动态的 CPU、内存、网络带宽和磁盘 IO 纳入评价指标，优化多维资源的负载均衡，降低多维资源的碎片化。孙毅^[15]等人面向 CPU/GPU 组成的异构算力，针对具有优先级的混合类型 Pod，采用了改进的遗传算法来解决异构平台的算力分配问题，提升了集群的负载平衡性能，同时考虑了 CPU、内存和 GPU 三个负载指标。

（3）针对多目标优化的研究现状

Fu Y^[35]等人面向机器学习任务引入任务的进度率和同一节点间任务的争用率来优化 Kubernetes 调度器，进度率通过训练任务当前迭代次数与总迭代次数的比值估算，并部署机器学习任务进行实验，改善任务的完成时间以及资源利用率，但这种计算进度率的方式在非机器学习任务中无法获取，存在一定的局限性。在深度学习任务中，Liu Z^[36]等人指出粗粒度调度机制忽略了与 GPU 数量或 GPU 内存以外的信息，导致 DL 任务性能下降；同时，深度学习任务刚启动时会导入依赖，但不会消耗资源，若以此作为评判依据导致任务执行效率降低，

因此,作者设计 GPU 嗅探器和平衡感知调度器,提出了 KubeFBS 调度器,从细粒度的视角为任务分配 GPU 资源,提高集群负载均衡和任务的执行速度。王壮^[37]等人在 Kubernetes 上实现了 GPU 的资源隔离和共享,并基于时间片实现任务的抢占,提升了 GPU 训练任务的执行效率和 GPU 资源的利用率。

综上所述, Kubernetes 作为算力调度系统的底座,在面对各种复杂的场景下默认调度器显得不够灵活和高效,尽管其设计初衷是为了广泛适用于多种类型的工作负载,但在特定的多类型任务场景下,标准的调度策略可能无法满足这些任务的特定需求。上述研究关注点主要从提高任务执行效率、保持集群负载均衡、减少资源碎片等来优化 Kubernetes 调度策略。但是,在优化任务执行效率时和资源分配时,仅考虑了 CPU、GPU、内存、磁盘 IO、网络带宽中 1 到 4 个指标,没有将所有算力指标全部纳入,这在算力调度系统中面向多类型任务的调度时无法达到最优的调度效果。

1.3 本文的主要工作

由 Kubernetes 和 Karmada 打造的异构算力调度系统中面临调度多类型任务时任务执行效率低、资源负载不平衡等问题。

因此,本文针对以上问题,通过多维细粒度的方法度量节点的实际性能,并在多集群环境下引入高效的资源监控策略,进而构建一个多目标优化算力调度模型,以优化默认的调度策略。本文主要做了以下工作:

(1) 提出了一种多维细粒度算力度量模型。首先,从节点细粒度硬件参数出发,构建多维度的算力度量指标体系。其次,采用 AHP 确认各维度下算力指标的权重,并构建 TOPSIS 多属性综合评价模型对各维度下的指标进行综合评价,以此作为节点各维度下的算力值,为后续调度决策提供调度依据。

(2) 设计了一种高效的多集群监控方案。由 Kubernetes 和 Karmada 构建的算力调度平台中,会涉及到多集群的管理和调度,业界普遍采用 Prometheus 联邦监控方案实现多集群监控,这种监控方式会带来极大的跨集群网络流量。因此,本文在 Prometheus 监控的基础上,引入一中高效的多集群监控方案,旨在减少多集群监控产生的跨集群网络流量。

(3) 设计了一种多目标优化算力调度策略,以提高多类型任务的执行效率,

并保证全局多维资源的负载平衡。默认的 Kubernetes 仅支持对 CPU 和内存两种资源的申请，且在进行调度时以资源的申请量而非资源的实时状态进行决策，这会导致节点产生资源碎片。因此，本文在现有的资源申请模式上，加入 GPU、磁盘 IO、网络带宽资源。设计全局资源负载平衡适应度函数，根据任务类型和算力值设计了任务执行时间适应度函数，建立算力调度模型，并使用 NSGA-II 算法进行求解，得出最佳调度方案。最后通过 Karmada API 向系统批量提交任务。实验结果表明，本文设计的算力调度策略能够有效提高任务执行效率，并保证一定程度的全局资源负载平衡。

(4) 在真实服务器上基于 Kubernetes 和 Karmada 重构了多集群异构算力调度系统，部署了本文设计的算力调度模型并对系统进行实验和测试，以验证本文设计的调度策略的最终效果。

1.4 本文的组织结构

后续章节具体安排如下：

第二章，对本文涉及到的相关关键技术的概念、架构和原理进行介绍。主要介绍 Kubernetes、Karmada 容器编排和管理工具，Prometheus、Node Exporter、GPU Exporter 监控生态以及其他诸如 Docker、NVIDIA Device Plugin 等。

第三章，多集群异构算力调度系统总体分析。首先分析算力调度系统的应用场景，并指出基于 Kubernetes 和 Karmada 的算力调度系统中存在的不足；其次对系统进行需求分析；最后指出本文待解决的关键问题。

第四章，系统设计。介绍系统的整体架构及相关功能的设计细节。

第五章，系统实现与测试。首先介绍系统最终的部署拓扑结构、系统所需的软硬件环境，并对关键软件安装进行说明。其次，介绍系统关键功能模块的实现细节。最后对系统关键模块进行验证性和测试，并对结果进行分析。

第六章，总结与展望。对全文的内容进行总结，指出本文在异构算力调度系统中的特色和创新，指出本系统存在的不足之处及未来的优化和改进方向。

第二章 相关技术与理论基础

本章旨在深入探讨本研究所依托的核心技术与理论基础。通过对 Kubernetes 容器编排技术、Karmada 多集群管理系统、Prometheus 监控技术、Docker 容器技术、Nvidia Device Plugin 设备插件等相关技术进行详细分析，建立夯实的知识基础，从而更好地理解后续的研究设计、实验方法及科学意义。

2.1 Kubernetes 容器编排框架

2.1.1 Kubernetes 概述

Kubernetes 作为一个开源的、强大的、灵活的云计算平台，已经在容器化应用的部署、扩展和管理方面引起了云计算领域革命性的变化。随着软件开发向微服务架构倾斜，大数据计算、人工智能训练等任务迁移上云，Kubernetes 提供了一种高效、自动化管理的解决方案来处理成百上千的服务部署以及计算任务的调度问题。

如图 2-1 所示，从架构上看，Kubernetes 架构由控制平面和工作节点两部分构成，各自承担着不同但互补的角色，其本质上是一个主从架构的分布式平台，用于容器应用的自动编排、扩展和管理。

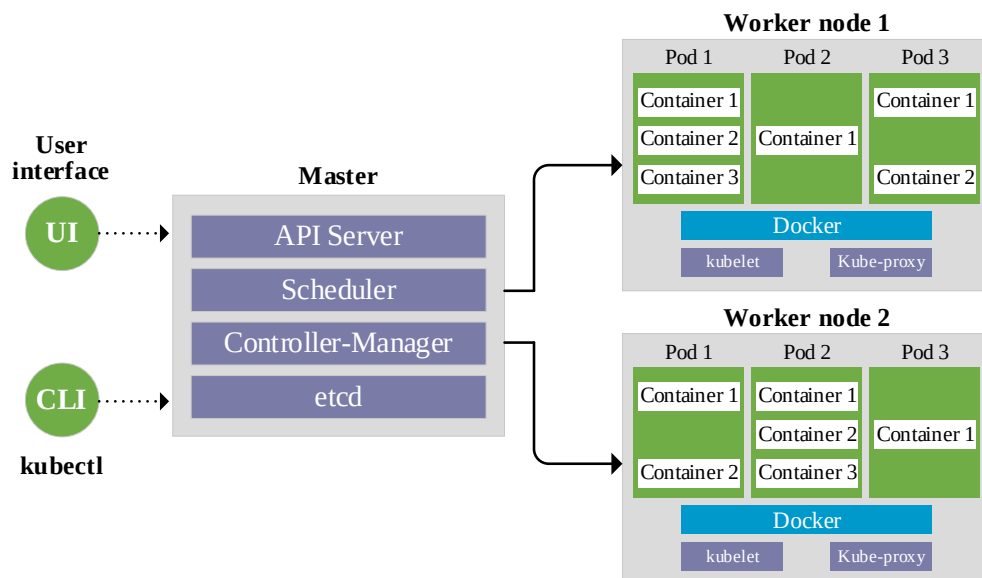


图 2-1 Kubernetes 架构

Kubernetes 控制平面是 Kubernetes 架构中的关键组成部分，主要负责

Kubernetes 集群的全局管理。它是 Kubernetes 集群的大脑，负责做出一系列决策，以及相应集群内发生的各种事件。控制平面主要由以下组件组成：

(1) **API Server**: Kubernetes API 唯一的入口，负责处理 REST 请求，是各种组件进行交互的枢纽。

(2) **Scheduler**: Kubernetes 的调度器，当有新的 Pods 发出调度请求后，Scheduler 会根据 Pods 请求的资源量和集群的资源剩余情况等数据，根据一系列调度算法将待调度 Pods 调度到最合适的节点上运行。

(3) **Controller Manager**: 该组件负责运行控制器进程，包括 Deployment 控制器、Job 控制器、Namespace 控制器等，负责监听集群的状态，并采取措施使当前状态向期望状态转变。

(4) **Etc**: Kubernetes 后端持久化存储数据库，保存整个集群的状态数据。

Kubernetes 工作节点是运行容器化应用的实例，部署有以下组件：

(1) **Kubelet**: 该组件以代理的形式运行在工作节点上，负责管理运行节点上 Pods 的生命周期、资源监控、节点状态检查、容器启停等。

(2) **Container Runtime**: 负责运行容器，例如 Docker、Containerd 等。

(3) **Kube Proxy**: 负责网络代理和负载均衡的关键组件，便于 Pods 间以及外部流量 Pods 之间的通信，其所用主要体现在服务发现和负载均衡、管理网络规则、支持不同的网络模型、实现 Service 抽象等。

Kubernetes 的腾空出世标志着云计算进入了一个全新的时代。作为一个新的云计算开源平台，其不仅提供技术上的创新和优势，也为云原生、算力网络等新领域提供了技术上的启发。

2.1.2 Kubernetes 调度

Kubernetes Scheduler 是 Kubernetes 平台的核心组件之一，负责调度新创建或未指定运行节点的 Pods，指定将 Pods 调度到哪个节点上运行。Scheduler 组件确保了 Kubernetes 集群的高资源利用率和应用的高可用性。Kubernetes 的调度流程主要分为两个环节：Filtering 和 Scoring。在 Filtering 环节，Scheduler 会检查集群中的所有节点，排除不符合节点资源需求（如 CPU 和内存的请求和限制、节点亲和性/反亲和性、Pod 亲和性和反亲和性等）的节点。在 Scoring 阶段，

Scheduler 会对筛选出来的节点进行打分，通过各种打分策略的组合来评估每个节点，最后选择得分最高的节点与待调度 Pod 进行绑定运行。

虽然 Kubernetes 提供了灵活的默认调度器，但在某些场景下，原生调度器可能无法满足用户需求，因此，Kubernetes 为了满足用户的特定需求，允许自定义调度器，以提供特定的调度逻辑。目前最新版本的 Kubernetes 支持多种方式自定义调度器，其中，由 Kubernetes SIG Scheduling 负责开发和维护的 Kubernetes 调度框架是最简单、使用最多的自定义调度方式。如图 2-2 所示，Kubernetes 调度框架开放了调度周期和绑定周期、多个阶段的自定义插件点，包括优先级队列排序、过滤阶段、绑定阶段等。

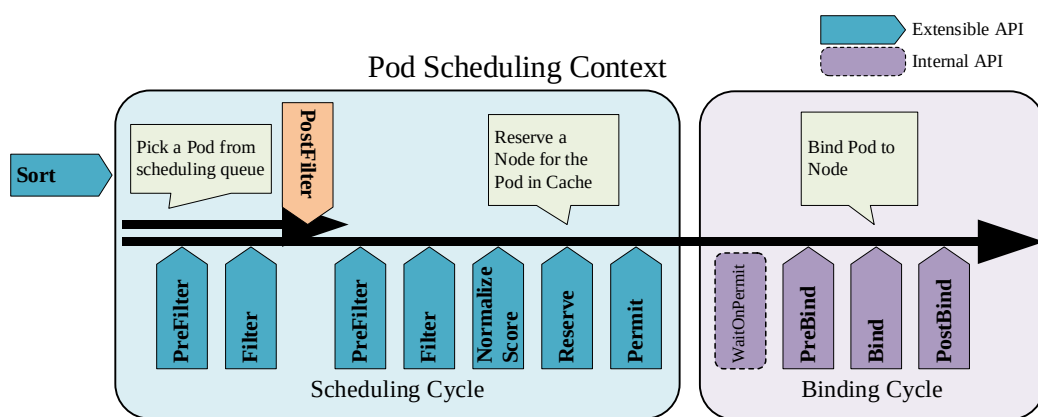


图 2-2 Kubernetes 调度框架扩展点

在 Kubernetes 调度器开源社区中，根据特定的需求和场景，涌现了多种调度器。例如，在大数据、科学计算、人工智能训练任务等批处理场景下的协同调度器 Coscheduling，其确保一组 Pods 始终作为一个整体被调度，如果集群中没有足够的资源同时满足该组 Pods 的资源需求，则这组 Pods 将会等待，直到有足够资源后一起调度，避免部分 Pods 调度成功而其他 Pods 调度失败的情况，极大提高了高性能计算场景下的资源利用率。

Kubernetes Scheduler 是体现 Kubernetes 容器化应用自动编排能力的核心，其通过一系列高效的调度算法，确保 Pods 能够在最合适的节点上运行，优化了资源的利用率，提高了容器化应用的可靠性。但是，随着 ChatGPT、大模型等新兴应用的产生，Kubernetes Scheduler 需要不断引入更多的特性以支持新型应用的调度编排，以更好地支持云原生应用的发展。

2.2 Karmada 多集群编排系统

2.2.1 Karmada 架构

Karmada (Kubernetes Armada) ^[38] 是一个基于 Kubernetes 的多集群管理项目，实现跨不同 Kubernetes 集群的工作负载的无缝分配、部署和管理。Karmada 体现了高可用性和灾难性恢复的原则，通过支持分布式系统的同时保证系统的弹性和可扩展性。

如图 2-3 所示，Karmada 由控制平面中中心和分布式数据平面组成。其架构的核心是 Karmada API Server，类似于 Kubernetes API Server，为所有客户端交互提供 RESTful API。ETCD 作为底层的持久化层，保证控制平面状态的持久性和一致性。

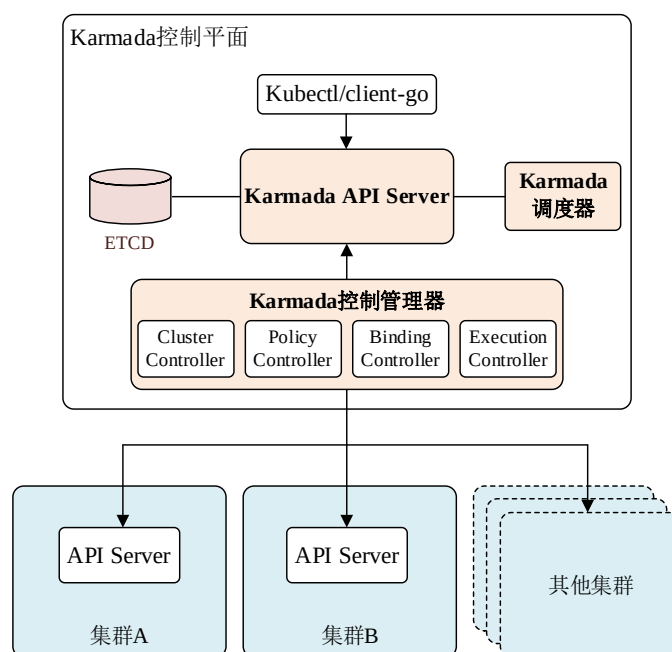


图 2-3 Karmada 架构

Karmada 控制平面包含一系列控制器，每个控制器负责特性的操作领域。集群控制器 (Cluster Controller) 是 Karmada 的一个核心组件，用于持续监控已注册集群的状态，并负责同步集群状态，提供基础的集群监控和能力的统一视图。策略控制器 (Policy Controller) 处理跨集群的工作负载分布策略，智能解读诸如调度约束、亲和性规则和优先级等规则，作出明智的工作负载放置决策。在策略控制器决策之后，绑定控制器 (Binding Controller) 将工作负载绑定到特

定集群。将高级调度决策转化为可操作的绑定对象，知道工作负载的部署方式和位置。执行控制器（Execution Controller）负责实现绑定阶段的决策，管理跨集群资源的生命周期，包括创建、更新和删除资源等。

数据平面由多个 Kubernetes 集群本身组成，每个集群作为一个独立实体，拥有自己的 API Server。Karmada 能够将这些集群编织成一个统一的多集群设置，形成一个不依赖于集群配置统一性的分布式系统。

用户交互层通过 Kubernetes 命令行工具 kubectl 进行访问，该工具通过 client-go 库增强了编程交互能力。用户可以通过该界面执行各种操作，如注册集群、部署应用和查询系统状态等。

2.2.2 Karmada 操作流程

Karmada 的操作流程包含以下几个阶段：

集群注册：初始阶段，将各个 Kubernetes 集群注册到 Karmada，标记各个 Kubernetes 集群为中心化管理的对象

策略定义：用户可以定义跨集群部署策略，详细说明工作负载分布的标准，这一阶段对于建立系统的期望状态至关重要。

资源调度：利用定义好的策略，Karmada 的调度机制将工作负载分配给合适的集群，调度算法考虑了资源可用性、政策约束和集群健康等因素。

资源绑定和执行：调度完成后，将资源绑定到目标集群进行部署。该阶段中的绑定和执行控制器发挥了重要作用，确保用户定义的策略得到实现。

状态同步：Karmada 持续监控和同步所有集群的状态，这一步是维持每个集群的实际状态和 Karmada 中心视图一致性的关键。

2.2.3 Karmada 调度

Karmada 的调度系统是其核心特性之一，允许用户按照预定义的策略在多个 Kubernetes 集群中调度和管理工作负载。调度过程遵循一系列的步骤，其中包括策略定义、调度决策、绑定和执行。

用户首先需要定义一个或多个调度策略，这些策略描述了工作负载在不同集群中的部署偏好。策略可以基于多种因素，如集群的地理位置、性能、成本、可用性和法律合规要求等。策略也可以定义特定的服务水平协议（Service Level

Agreement, SLAs), 如响应时间和高可用性。

Karmada 调度器根据定义的策略做出决策, 会考虑如表 2-1 中的因素。

表 2-1 影响 Karmada 调度决策的因素

因素	解释
资源需求	工作负载需要的资源量, 如 CPU、内存等
资源亲和性	工作负载与特定资源或其他工作负载的亲和性或反亲和性
集群能力	集群是否具备满足工作负载需求的能力
负载均衡	在集群之间均匀分配工作负载以避免任何单点过载
高可用性	工作负载的副本跨不同集群的分布, 以提高整体系统的鲁棒性

调度决策完成后, 绑定控制器将工作负载与选定的集群进行绑定。这个过程涉及到将工作负载规范转换为在目标集群中可执行的具体配置。执行控制器根据绑定结果在目标集群中创建、更新或删除资源。负责确保在多个集群中实现和维护工作负载的期望状态。

2.3 Prometheus 监控技术

2.3.1 Prometheus 简介

Prometheus 是一个开源的、功能强大的、高效的监控和警报套件^[39], 被广泛用于存储实时的度量指标并提供警报功能, 是云原生生态中不可或缺的一部分, 为 Kubernetes 集群及应用提供了强大的监控能力。

Prometheus 架构如图 2-4 所示, 主要包含 Prometheus Server、Alertmanager、Exporter、Push Gateway、Grafana。

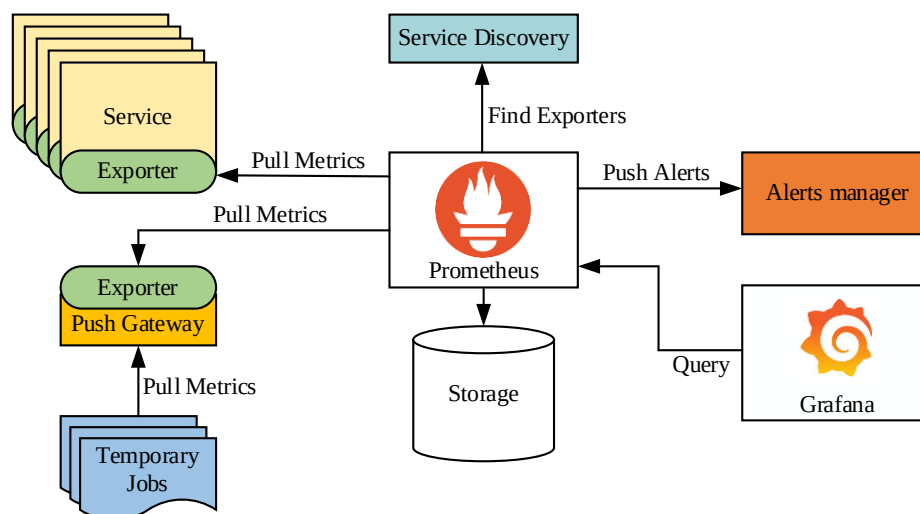


图 2-4 Prometheus 架构

Prometheus Server 是 Prometheus 架构中的核心组件，负责数据抓取、存储，以及执行时间序列数据的查询，通过 HTTP 协议定期从目标中抓取指标数据，并将其存入时间序列数据库中。Prometheus Server 提供一种查询语言 PromQL (Prometheus Query Language)，用于向时间序列数据库中查询数据。

Alertmanager 负责处理 Prometheus Server 发出的警报。负责将警报进行去重、分组，并将其路由到正确的接收对象。同时，Alertmanager 还支持警报的沉默、抑制、时间安排，提供了灵活的警报管理功能。

Exporter 是 Prometheus 中的第三方组件，对于不能直接暴露指标给 Prometheus 的服务，Prometheus 开源社区提供了大量的 Exporter，用于抓取特定服务的指标，并暴露给 Prometheus。常见的 Exporter 包括 Node Exporter、Nvidia GPU exporter、MySQL Exporter 等，极大扩展了 Prometheus 的监控场景。

对于某些运行较短的任务，直接交由 Prometheus Server 抓取指标可能效果不显著，这些临时任务将它们的数据推送至 Pushgateway 中间件，Prometheus Server 之后便可以从中抓取指标。

在可视化操作方面，Prometheus 提供自带的 Web UI 界面，用于执行 PromQL 查询和查看警报状态。若有更复杂的可视化需求，可以将 Prometheus 与 Grafana 集成，Grafana 提供丰富的可视化面板与实时的监控数据展示。

2.3.2 Node Exporter

Node Exporter 是一个专门用于收集主机硬件和操作系统级指标并暴露给 Prometheus 的组件。通过这些指标，用户可以实时了解主机的健康、性能、资源使用情况等状态信息。

在指标收集方面，Node Exporter 可以收集的指标覆盖硬件和操作系统级别，包括 CPU、内存、磁盘和网络使用情况、系统负载等。在部署方面，Node Exporter 以单个静态编译的二进制文件提供，可以独立于其他组件部署，且支持各种 Linux 操作系统版本。在配置方面，Node Exporter 默认开启了大量指标收集器，用户可以通过自定义禁用或启用特定的收集器，以适应不同的监控需求。在兼容性方面，Node Exporter 可以与 Prometheus 无缝衔接，并且其暴露的指标数据可供其他监控系统使用。

Node Exporter 是 Prometheus 监控生态中不可或缺的一部分，为主机和系统级别的监控提供了丰富的指标。通过对这些指标进行深入分析和监控，可以提高系统的稳定性和性能提供强有力的指导。

2.3.3 Nvidia GPU exporter

在当今的计算环境下，GPU 已经成为加速各种高性能计算任务执行效率的重要资源。近年来，高性能计算、AI 训练任务不断地集成到云计算平台中，为了有效监控和管理 GPU 资源，Nvidia GPU exporter^[40]应运而生。

Nvidia GPU exporter 是一个开源的监控工具，用来从 Nvidia GPU 中导出各种 GPU 性能指标到 Prometheus 中。借助 Nvidia GPU exporter，集群维护人员可以实时观察 GPU 资源的使用情况和健康状态，这对于确保高性能计算任务稳定运行至关重要。

Nvidia GPU exporter 的监控核心是充分利用了 NVIDIA 管理库 (Nvidia Management Library, NVML)，可以收集包括 GPU 时钟速度、GPU 利用率、显存使用情况、温度变化、功耗、风扇转速等指标。Nvidia GPU exporter 的部署相对简单，它可以作为容器运行在提供 Nvidia GPU 支持和 Nvidia GPU 驱动环境的主机上，并与 Prometheus 紧密集成，使得 GPU 设备和 GPU 任务运行状态在 Kubernetes 环境中变得更加透明。

Nvidia GPU exporter 作为一个现在计算环境下不可或缺的工具，其重要性将随着 GPU 在各种高性能计算任务中作用的增加而增加。通过提供精确、实时的监控数据，Nvidia GPU exporter 使得 GPU 集群的维护变得更便捷，优化了资源利用率。

2.4 其他相关技术

2.4.1 Docker 容器技术

Docker 是云原生生态中一种革命性的开源的容器技术，允许开发人员在各种环境下快速部署和原理容器应用，常作为 Kubernetes 运行时工具，以支撑 Kubernetes 对容器的高效部署和管理。Docker 通过提供一个额外的抽象层和自动化操作平台，极大简化了应用的分发与执行。

在 Docker 架构中，有以下核心对象：

(1) 容器：在 Docker 的视角下，容器可比拟成轻量级的虚拟机，但和虚拟机不同，容器共享主机的内核，而不需要额外提供整个操作系统。每个容器都可以对应用的执行代码、库和依赖项进行隔离和独立管理、运行。

(2) 镜像：镜像是创建容器的基础模板。镜像是不可变的文件集合，包含有关应用所需的代码、运行时、工具以及库的所有信息。

(3) Dockerfile: Dockerfile 是一个文本文件，是创建 Docker 镜像所需步骤的集合，开发人员可以通过编写 Dockerfile 来构建镜像。

(4) Docker Daemon: Docker Daemon 是 Docker 的后台服务，即守护进程，负责管理 Docker 对象，如镜像、容器、网络 and 卷等。

(5) Registry: Registry 是存储 Docker 镜像的服务，允许使用人员上传和下载镜像。最常用的 registry 是 Docker Hub，这是一个公共 registry，同时，用户可以设置自己的私有 registry。

(6) 客户端：Docker 客户端是用户与 Docker Daemon 进行交互的平台，用户可以通过客户端使用一系列 Docker 命令对容器进行操作和管理。

如图 2-5 所示，Docker 基于 C/S 架构实现，包括客户端、Docker Daemon、Registry。用户通过 Docker 客户端与守护进程进行交互，守护进程负责创建、运行和监视容器状态，并与 Registry 交互以拉取或推送镜像。容器通过 Docker Daemon 管理的镜像实例化而来，并在 Docker Host 上运行。

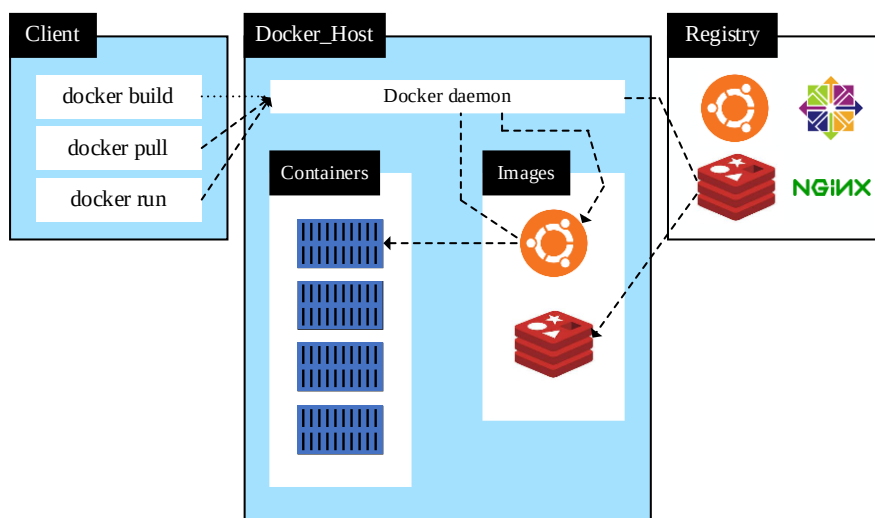


图 2-5 Docker 架构及工作原理

Docker 的应用场景非常广泛。在微服务架构中，Docker 常作为微服务的运行时支持，每个服务可以打包在独立的容器中运行；在开发和测试环境下，Docker 保证开发、测试和生产环境的一致性，极大程度降低了兼容性问题；在高性能计算场景下，Docker 容器可以在多租户的环境中隔离运行，适用于高性能和大规模计算；同时，Docker 与 CI/CD 工具链的高效集成，使得自动化部署变得简单可靠。

Docker 作为一种高效、轻量级的容器化解决方案，已成为现代软件工程实践中不可或缺的一部分。通过简化部署流程和加强环境控制，提升开发和运维的协同效率，降低运维难度，加快了 DevOps 的发展。

2.4.2 NVIDIA device plugin

在高性能计算和机器学习任务中，GPU 扮演着越来越重要的角色。NVIDIA 作为 GPU 技术的领导者，为 Kubernetes 容器编排平台提供专门的设备插件 NVIDIA device plugin^[41]，以便 Kubernetes 能够支持和使用 GPU 资源。

NVIDIA device plugin 利用 Kubernetes 设备插件框架，实现了 Kubernetes 识别和管理节点 NVIDIA GPU 资源。该插件遵循 Kubernetes 设备插件的 API 规范，通过注册自身至 Kubernetes 的 Kubelet，将节点上的 GPU 资源作为可调度资源公开。不仅确保了节点能够正确报告其 GPU 资源，而且允许在创建 Pod 时请求特定数量的 GPU 资源。如图 2-6 所示，新创建的 Pod 向 Scheduler 申请扩展资源 nvidia.com/gpu 资源，Scheduler 将 Pod 与 Node 进行绑定后通知 Kubelet 中的 Device Plugin Manager，Device Plugin Manager 向 NVIDIA device plugin 申请分配 GPU IDS，NVIDIA device plugin 在所管理的 GPU 资源中为 Pod 分配运行时环境，并由 Kubelet 创建 Docker 容器。

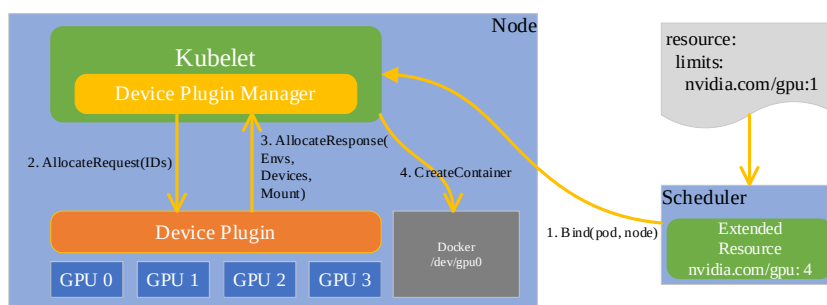


图 2-6 Nvidia device plugin 工作原理

NVIDIA device plugin 的关键特性包括其自动化资源管理、版本兼容性和集群资源共享。它自动化了 GPU 在 Kubernetes 中的发现和注册过程，无需人工干预即可将节点上的 GPU 资源整合到 Kubernetes 集群中。与此同时，NVIDIA device plugin 支持多个版本的 NVIDIA 驱动，能够在不同版本的系统中灵活运行，提升了系统的兼容性。

在云计算环境下的高性能计算和机器学习领域，NVIDIA device plugin 显得尤为重要。对于需要大量并行处理能力的任务，如科学计算、数据分析和深度学习训练等，GPU 提供了必不可少的计算资源。通过 NVIDIA device plugin，这些计算密集型任务可以在 Kubernetes 环境中充分使用 GPU，实现快速计算和推理。

尽管 NVIDIA device plugin 已经在多个方面展现了其强大的能力，但在实际部署和运维过程中仍面临许多挑战。包括与 Kubernetes 的无缝集成、持续的性能优化、跨不同环境之间的兼容性问题。未来的发展方向可以进一步提升其自动化功能、提升在 Kubernetes 环境中充分利用 GPU 资源，以更高效地管理 GPU 资源和提升 GPU 资源的利用率。

2.5 本章小结

本章深入剖析实现系统所需的关键技术和理论基础，涵盖从容器编排、多集群管理、资源监控、GPU 资源管理的知识。首先对 Kubernetes 的核心组件和操作机制进行细致讲解，特别是对调度器和调度框架的功能进行了精确的分析，同时，介绍 Karmada 多集群管理工具的操作流程和调度原理。接着转向云原生监控领域，详细介绍 Prometheus 监控方案，并强调 Node Exporter 和 Nvidia GPU exporter 的作用与重要性。继而，本章展开对系统中设计的其他技术进行探讨，包括 Docker 容器化技术、针对 NVIDIA GPU 专门为 Kubernetes 设计的 NVIDIA Device Plugin。每项技术的介绍都旨在为实际系统实现提供坚实的技术支撑，通过全面的技术铺垫，奠定了实现一个稳固、可靠和高效系统的基础。

第三章 多集群异构算力调度系统总体分析

3.1 应用场景分析

随着云计算技术的快速发展，政府、企业、研究机构等越来越依赖云平台来处理复杂的超算任务。在此背景下，异构计算资源的有效管理和调度成为提升计算效率、降低成本的关键。传统的资源调度策略往往难以应对异构环境中的多样化计算需求，特别是在面临节点异构性差异显著，以及无法准确评估异构资源实际性能的情况下，这一问题尤为突出。在以 Kubernetes 和 Karmada 构建的算力调度系统中，存在以下几个关键挑战。

(1) 算力度量是评估计算资源性能的重要手段，主要用于量化地描述算力节点执行任务的能力。然而，在多数调度系统中，调度决策往往依赖于资源的可用性和使用量，忽视了任务特性与算力资源之间的匹配程度，进而导致任务执行效率不佳。目前的算力度量研究存在以下问题：度量标准不统一、度量方法在不同场景下的适应性较差以及节点级别的度量结果不能有效反映节点的实际性能。因此，亟需设计一个的算力度量模型，准确识别节点的性能，并应用于调度工作中，以提高任务与节点性能的匹配度。

(2) 为解决数据中心分散部署，算力资源无法统一管控的问题，需要将多个集群整合起来，实现多集群统一的管理和调度。因此，对于多集群场景，业界采用 Karmada 多云容器编排项目实现多集群的管理，Karmada 进行多集群管理的同时提供了基本的调度功能。但是，Karmada 内置的调度器仅支持集群级别的调度，对节点级别的调度存在较大的局限性。对于现有版本的 Karmada，节点级别的调度需要借助 Kubernetes 调度器，以两级调度的方式完成任务到节点的分配。

如图 5-19 所示，在 Karmada 控制平面提交任务时，如果将任务调度到某个节点执行，需要在 Karmada 控制平面手动配置成员集群的优先级，在满足集群资源容量足够的情况下，Karmada 调度器会将任务调度到优先级最高的集群中，再由 Kubernetes 调度器将任务调度到节点上执行，完成任务的分配过程。该调度机制会导致局部的成员集群负载过高，从而影响系统的稳定运行。

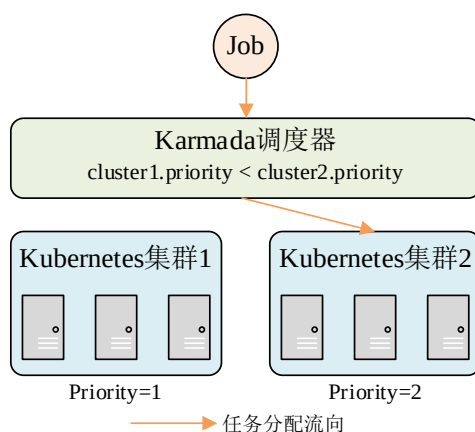


图 3-1 Karmada 与 Kubernetes 默认调度策略

(3) Kubernetes 中调度的最小单位为 Pod，在执行调度时，Kubernetes 以 Pod 申请的资源的静态状态而非实际状态作为调度依据，容易产生较多资源碎片，导致资源利用率低。如图 3-2 所示，本文在 Kubernetes 集群上提交了一个 Spark PI 任务，该任务由 1 个 Driver 和 5 个 Executor 组成，每个 Executor 被封装在一个独立的 Pod 中，并申请了 12000 毫核 CPU 和 600Mi 内存资源。通过监控整个任务执行周期的真实资源使用情况，发现 Pod 的实际资源使用量始终低于其向 Kubernetes 申请的资源量，这意味着大量资源被锁定而未被实际使用，产生较多资源碎片。由于调度器未能准确感知 Pod 的实际资源使用情况，容易导致集群中某些节点过载，而其他节点资源闲置。资源分配的不合理可能会影响任务的执行效率。

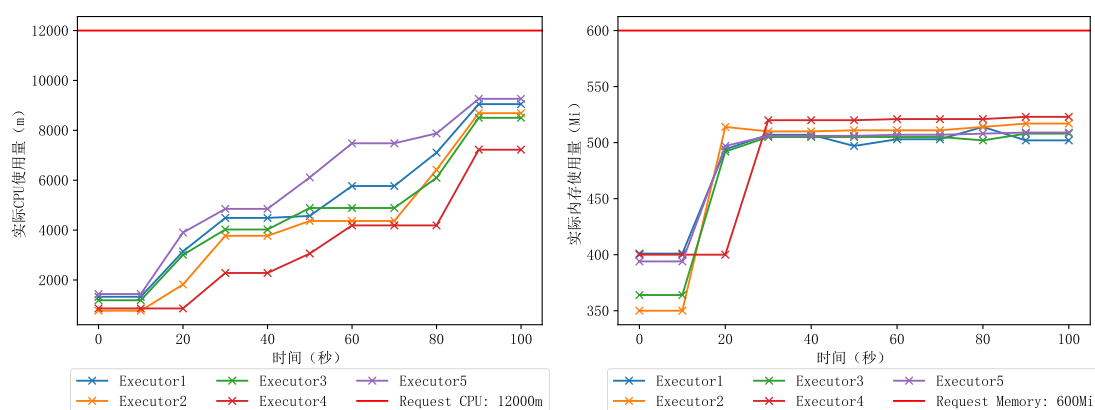


图 3-2 Pod 实际资源使用情况

(4) Kubernetes 默认调度策略只考虑 CPU 和内存，忽略了 GPU、磁盘 IO、网络带宽等资源对系统负载的影响，造成除 CPU 和内存外，其他资源的负载不平衡，导致系统稳定性降低。

针对以上指出的基于 Kubernetes 和 Karmada 搭建的算力调度系统中存在任务与节点性能匹配度低下、在多集群场景下节点级别的调度导致局部负载较高,负载不平衡、调度工作往往以静态资源状态作为决策依据,且忽略了 GPU、磁盘 IO、网络带宽等对资源负载平衡的影响,导致负载不平衡,进而使系统稳定性降低的问题,本文重构了一个多集群异构算力调度系统并对默认调度策略进行优化。在算力度量层面,从多维度、细粒度的角度出发,全面度量节点性能,为算力调度决策提供重要依据;在算力调度方面,基于 Kubernetes 和 Karmada 实现多集群算力调度解决方案,从全局的角度监控实时资源状态,并对任务进行全局调度,以提高任务执行效率、减少资源碎片化、保证全局的负载平衡等。

本文基于 Kubernetes 和 Karmada 重构了多集群异构算力调度系统,设计了如图 3-3 的应用场景。存在一个由多个数据中心组成的多集群异构算力系统,其中一个数据中心作为全网资源的控制平面管控全网的算力资源,并对外向用户提供任务提交的入口。用户向控制平面提交任务后,控制平面依据任务特性、任务申请的资源量等信息,结合全局资源的实时状态并根据相应的调度策略将任务分配给算力节点,任务执行完成后,向用户返回最终的执行结果。在进行资源分配时考虑最小化任务的执行时间,并保证一定的资源负载平衡,提高任务的处理效率和保持系统的稳定性。

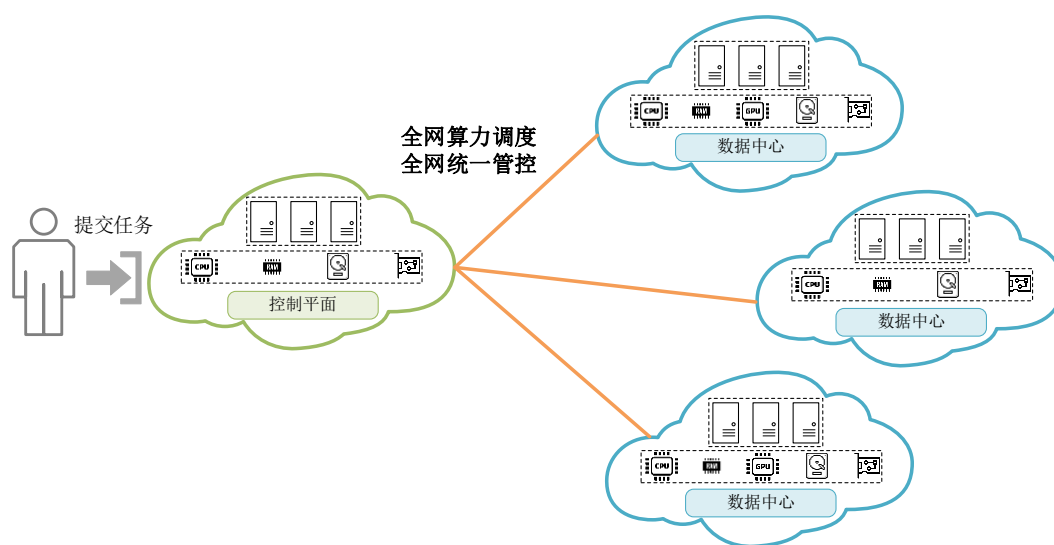


图 3-3 多集群异构算力调度系统应用场景

3.2 需求分析

3.2.1 系统功能性需求分析

随着东数西算、算力网络等背景相继提出，企业开始构建自身的算力调度系统以支撑自身业务的发展。设计高效的算力调度系统是实现资源最大化利用和提升服务质量的关键。本文通过分析图 3-3 的应用场景，深入探讨基于 Kubernetes 和 Karmada 的算力调度系统的设计需求及其功能性描述。系统的核心功能需求如下：

（1）算力度量：为了有效支持算力调度，本系统需对节点多维度的算力资源性能进行准确度量。从节点的底层硬件参数出发，构建合理的算力度量指标体系，采用科学的方法对指标体系进行度量，估计节点的实际性能，为算力调度提供可靠的量化依据。

（2）单集群监控：Kubernetes 的资源调度依赖于静态资源状态，该机制将 Pod 的资源申请量视为调度决策的评估标准。这种设计无法有效感知节点实时资源使用状态，可能导致资源利用率不高和资源碎片化等问题。针对这一问题，本系统需要在各集群内实施实时资源监控方案，通过实时监控节点的资源状态，为调度决策提供更加准确的数据支持，以实现资源的高效利用。

（3）多集群监控：由 Karmada 管理的多 Kubernetes 集群架构中，控制平面集群对成员集群的资源状态了解有限。为确保在控制平面集群中有效部署任务，需在单集群监控基础上，进一步设计多集群资源监控策略，加强控制平面对成员集群资源状况的认知，为多集群的资源调度和优化提供决策数据支持。

（4）资源接入与限制：在 Kubernetes 基础架构上实现任务部署的基本功能后，系统将遇到处理多种类型任务对多样化资源的需求挑战。Kubernetes 的原生设计主要聚焦于 CPU 和内存资源的调度，对于 GPU、磁盘 IO 及网络带宽的管理和分配提供的支持较为有限。这种局限性可能导致资源的低效利用和潜在的资源争用问题。为了解决这一问题，本系统需将 GPU 资源接入 Kubernetes 集群，并针对磁盘 IO 以及网络带宽引入管理和限制机制，通过精细化的控制和限制策略，确保磁盘 IO 及网络带宽等资源可以被按需合理分配，减少资源浪费，提高资源利用效。

(5) 算力调度模型构建：本系统的算力调度模型为一个多目标优化问题，基于全局监控数据、任务特性及其资源申请信息构建多目标优化模型。该环节将综合考虑任务的资源需求、任务大小等因素，设计出合理的调度模型、适应度函数和约束条件等，为实现资源调度的最优化提供理论模型。

(6) 多目标优化算力调度：在算力调度模型的基础上，本系统将采用多目标优化算法对调度问题进行求解，旨在找到最优的资源调度方案。求解得到的最优调度方案将通过 Karmada 在控制平面集群中进行实施，减小任务的执行时间，并保持较高的全局负载均衡，提高任务的执行效率，实现资源的高效分配。

3.2.2 系统非功能性需求分析

本文将从可靠性需求、性能需求、可用性需求、系统健壮性四个方面对系统非功能性需求进行介绍。

(1) 可靠性需求：算力调度系统在满足实现基本的功能之外，还要保证系统能在各种不可预料因素下正常运行，同时还要保证提交到系统的任务能够正常运行。不可预料因素包括对硬件配置的更改、系统下架与维护、扩展算力节点等。

(2) 性能需求：性能需求结合系统的功能性需求，从三个方面进行分析。第一，算力度量的结果要能够有效识别异构节点之间的性能差异。第二，对于跨集群监控产生的流量，在能够收集到相同监控数据的条件下，尽可能减少产生的跨集群流量以及资源消耗。第三，在算力调度层面，和默认的调度策略相比，要能够有效减少批量任务的执行时间，并保证一定程度的资源负载平衡。

(3) 可用性需求：在实现资源管理模块的功能时，确保 GPU 资源接入、磁盘 IO 管理和网络带宽限制能够在不同环境下正常执行，如不同的操作系统环境、软件版本、Docker 镜像等。

(4) 系统健壮性：算力调度系统在长时间运转的过程中，会出现诸如节点崩溃、断电、宕机等不可预料的故障问题。当局部节点出现以上故障后，保证平台的其他节点能够正常工作，且正在执行的任务能够重新调度到其他节点继续执行。

3.3 本文待解决的关键问题及对应的解决方案

针对目前基于 Kubernetes 和 Karmada 实现的多集群算力调度系统中存在任务与节点性能匹配度低、全局资源监控性能低、任务执行效率低、系统负载不平衡等问题，本文待解决的关键问题如下：

(1) 在算力调度场景下，有效衡量算力节点的真实性能是算力调度策略的基础，决定了任务是否和算力资源有效匹配。现有的算力度量方法存在度量维度单一，粒度不够细致等问题，无法反映节点的实际性能。

(2) 在基于 Kubernetes 实现的算力调度系统中，默认调度策略只考虑了节点资源的可用性，且以 Pod 申请的静态资源状态作为调度依据，容易产生较多资源碎片，导致任务执行效率和资源利用率低。

(3) 在基于 Karmada 的多集群算力调度场景下，默认调度策略只能进行集群级别的调度，节点级别的调度存在较大的局限性。

面对上述存在的问题，本文基于 Karmada 和 Kubernetes 重构出多集群异构算力调度系统，从细粒度、多维度的角度度量节点的算力，并设计高效的多集群监控框架，为调度决策提供决策依据和数据源。根据任务特性、节点算力值以及节点实际资源使用状态，构建多目标优化算力调度模型，并采用多目标优化算法求解出最佳调度方案，在控制平面实现全局调度，以减少批量任务的执行时间，并保证较高的全局资源负载平衡。

3.4 本章小结

本章从基于 Kubernetes 和 Karmada 的多集群异构算力调度系统的应用场景、需求分析、待解决的关键问题等方面进行了阐述。首先指出现有多集群算力调度系统存在的局限性和不足之处，继而提出本系统的应用场景。其次，根据应用场景及存在的问题，分析系统的功能性需求和非功能性需求。最后指出需要解决的关键问题以及对应的解决方案。

第四章 系统设计

4.1 系统总体设计

根据第三章对多集群算力调度系统的需求以及关键问题的分析，设计一个多集群异构算力调度系统，并对其进行优化，以提高系统稳定性、系统资源利用率以及任务执行效率。

4.1.1 系统整体架构设计

系统总体架构如图 4-1 所示。系统主体架构由 Kubernetes 容器编排框架和 Karmada 多集群管理器构成。

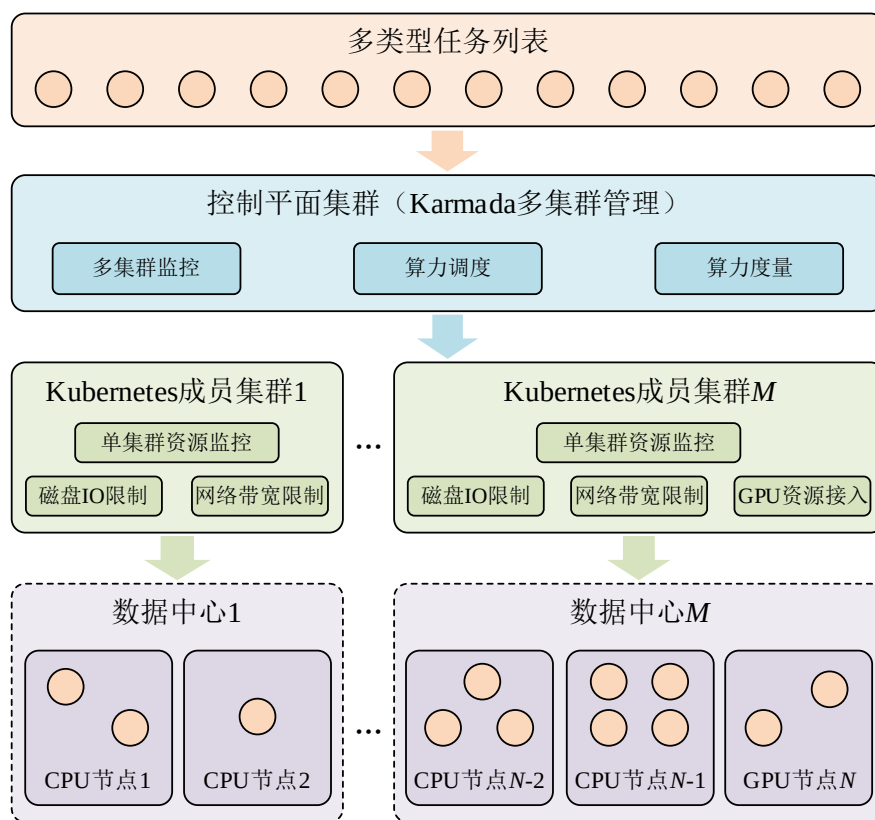


图 4-1 系统总体架构图

系统底层由多个来自不同数据中心的异构算力节点组成，一部分是 GPU 节点，各节点之间的 CPU、GPU、内存、磁盘、网卡存在异构性，每个数据中心的节点组成一个 Kubernetes 成员集群，各成员集群之上是一个部署了 Karmada 多集群管理器的控制平面集群。每个成员集群中部署了单集群监控，用于监控各节点实时的资源状态，基于单集群监控设计多集群监控策略，将监控数据上

报给控制平面集群；在控制平面集群根据多维细粒度算力度量指标度量节点各维度的实际性能；控制平面集群接收来自用户批量提交的多类型任务，根据任务特性、任务申请的资源量、多集群监控提供的节点状态信息，执行多目标算力调度策略，得出最佳调度方案，根据调度方案将任务提交给 Karmada，由 Karmada 将任务调度给各个 Kubernetes 成员集群，每个 Kubernetes 成员集群再将任务调度到指定的算力节点执行。

4.1.2 系统功能模块

系统功能模块划分如图 4-2 所示。系统分为多维细粒度算力度量模块、资源监控模块、资源接入与限制模块、算力调度模块 4 个基本功能模块以及算力指标体系构建、算力度量、单集群资源监控、多集群资源监控、GPU 资源接入、网络带宽资源限制、磁盘 IO 资源限制、算力调度模型构建、多目标优化算力调度 9 个子模块。

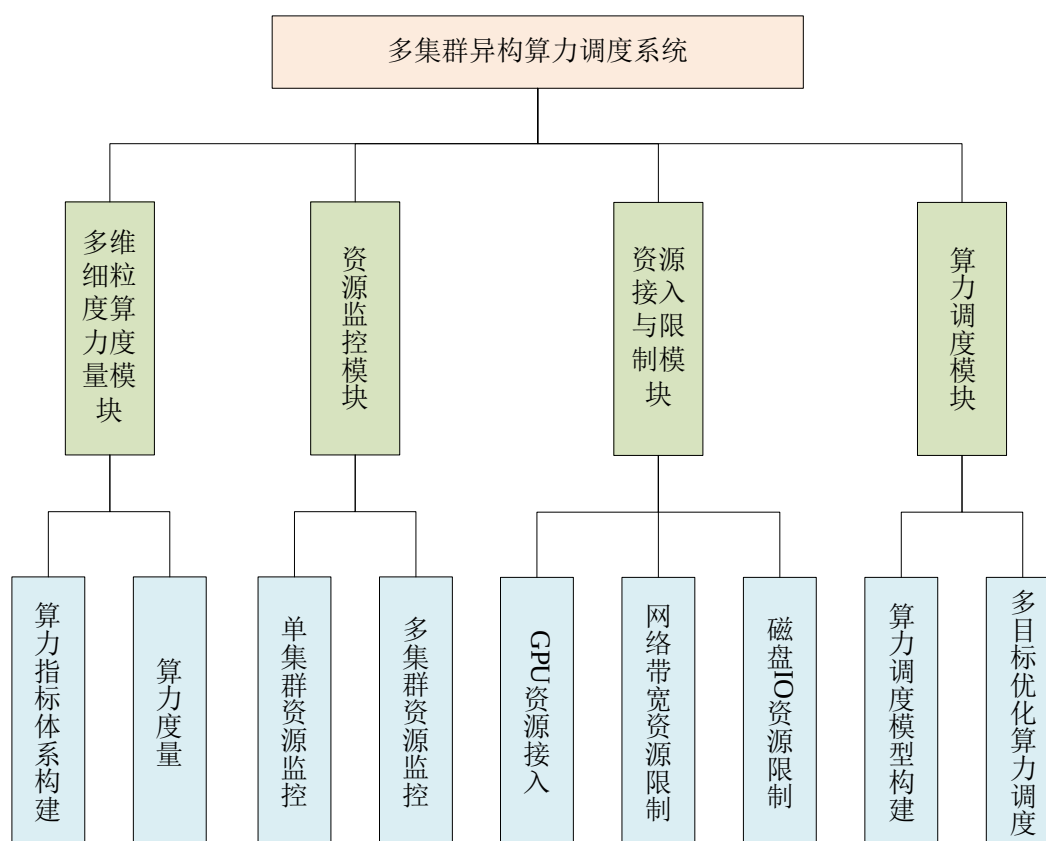


图 4-2 系统功能模块图

下面将详细分析基于云原生架构的异构算力调度系统各模块之间的业务逻辑。

辑关系以及各模块的功能。

(1) 系统业务逻辑

系统基础架构由 Kubernetes 和 Karmada 支撑，Kubernetes 作为底层的算力调度工具，Karmada 作为控制平面在控制集群中管理多个成员集群，包括资源监控、任务部署等。在每个成员集群中，实时监控各节点的资源状态信息（包括 CPU、GPU、磁盘、网络等），并将监控数据上报给控制集群，控制集群根据从监控数据中取出符合算力度量指标的数据，度量多维度（通用算力、智能算力、磁盘能力、网络能力）的算力值，作为算力调度的依据。任务则通过 Karmada 以声明式的方式批量提交，在提交任务的时候，除了声明对 CPU 和内存的使用量之外，还要声明对磁盘 IO 带宽、网络带宽的使用量，同时需要声明是否需要使用 GPU 资源。当向系统提交批量任务时，系统在控制平面根据任务申请的资源量、任务大小以及各节点资源使用情况构建调度模型，求解出最佳调度方案后，根据最佳调度方案以 Karmada 作为入口进行任务的分配。

(2) 多维细粒度算力度量模块

该模块根据多种原则构建出多维度（通用算力、智能算力、磁盘能力、网络能力）的算力度量指标体系，根据多维度的算力指标度量出能体现节点真正真实性能的多维度算力值，给算力调度模块提供决策支持。

(3) 资源监控模块

算力资源监控模块包括单集群和多集群的资源监控，本文选择的单集群监控是 Prometheus、Node Exporter、GPU Exporter 组合的监控解决方案。在单集群监控的基础上设计高效的多集群监控，并最大化减少多集群监控产生的跨集群网络流量及资源消耗。该模块主要用于给之后的算力调度决策提供实时的节点监控数据，同时当算力资源出现异常时提供线索，便于排查。

(4) 资源接入与限制模块

针对 Kubernetes 无法识别 GPU 资源，以及无法对磁盘 IO 和网络带宽资源进行精细化的分配和管理，扩展 Kubernetes 的资源管理机制，设计相关策略限制 Kubernetes 中 Pod 使用的磁盘 IO 和网络带宽量，在 Kubernetes 原有的 CPU、内存资源管理的基础上，实现限制 Pod 申请 CPU、内存、磁盘 IO、网络带宽资

源、需要 GPU 资源的任务能够使用 GPU。

(5) 算力调度模块

算力调度模块包含算力调度模型构建与多目标优化算力调度。当在控制平面提交批量任务时，全局调度器根据任务申请的资源量、全局节点的资源状态构建满足调度目标的算力调度模型，并采用多目标优化算法对调度问题进行求解，得出最佳调度方案后使用 Karmada API 接口批量提交任务，减少批量任务的执行时间，并保持较高的全局资源负载均衡。

4.2 多维细粒度算力度量模块设计

本文从算力节点细粒度硬件参数出发，分别从通用算力、智能算力、存储能力和网络能力四个维度构建算力度量指标体系，采用 AHP 权重计算方法确认各维度指标的权重，并构建多属性综合评价 TOPSIS 模型对各维度下的指标进行综合评价，得出各维度下的算力值。算力度量模块总体流程如图 4-3 所示。

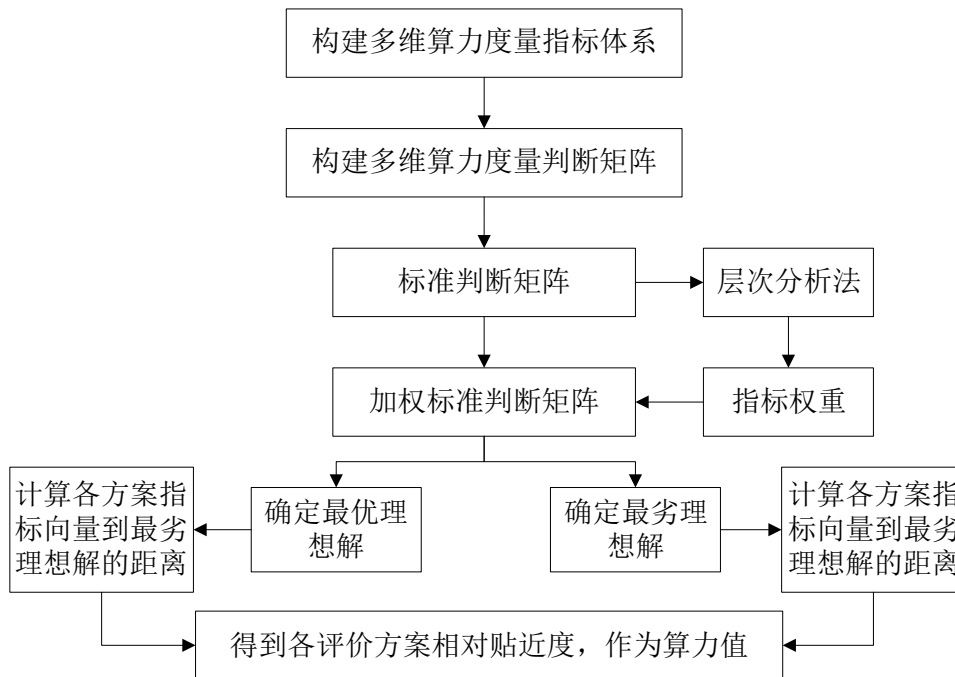


图 4-3 算力度量流程

4.2.1 算力指标体系构建

(1) 构建原则

在构建算力度量指标体系时，须遵循若干关键原则以确保指标体系的有效

性和实用性。目前,业界在算力度量方面存在一定程度的分散和不统一,主要集中在细粒度性、全面性、科学性、实施性和独立性方面的综合设计。

①细粒度性:从底层硬件性能参数出发,对算力节点的核心性能进行度量。

②全面性:指标体系应包括对通用能力、智能能力、存储能力和网络能力的全面覆盖,实现对计算资源、存储资源、网络资源的综合性度量。

③科学性:指标的设立需基于深入其原理,并能有效映射算力节点的实际性能,避免无效或误导性的度量标准。

④实施性原则:度量指标必须清晰明确,确保在实际应用中易于获取,避免产生难以解释或难以采集的数据。

⑤独立性原则:各项指标应具有明确区分,互相之间保持独立,以防止度量过程中的重复和不准确的现象。

(2) 构建过程

基于以上原则,本文收集硬件厂商提供的硬件参数,并对参数进行分析,确立最终的算力指标体系,分别从通用算力、智能算力、磁盘能力和网络能力4个维度来进行分析。

①通用算力指标:通用算力指标包括CPU和内存两个硬件的性能,从CPU和内存两个硬件参数出发,构建通用算力维度下的算力指标。

以英特尔官网提供的 Intel Xeon E-2468 处理器参数^[42]为例,英特尔目前在CPU规格中公布了内核数、总线程数、频率、缓存、总线速度、TDP 功耗等参数信息。其中内核数、总线程数之间的关系是总线程数为内核数和单核支持的超线程数相乘的结果,两者属于容量指标,不属于性能指标的范畴;CPU 频率是衡量 CPU 执行周期的速度的指标,以赫兹为单位,1 赫兹表示每秒执行一个周期,CPU 频率指 CPU 每秒能够进行的时钟周期数,频率越大,每秒可以执行的时钟周期越大,属于性能指标;CPU 缓存通过预取和存储近期访问的数据来减少 CPU 与主内存之间的数据交换次数,从而显著提高性能,分为一级缓存、二级缓存和三级缓存,属于性能指标;CPU 总线速度指连接 CPU 与主板其他组件的总线的数据传输速率,总线速度影响数据传输的快慢,进而影响计算机的整体性能,属于性能指标;TDP 表示 CPU 在工作时的最大热量散发量,提供了

一个基准，帮助散热系统设计者确保散热解决方案能够处理 CPU 在最大负荷下产生的热量，保证系统稳定运行不过热，属于能耗指标。

内存硬件参数则包括内存容量、内存频率、内存位宽等指标。内存容量表示主机可以同时处理的数据总量，更大的内存容量允许计算机同时运行更多的程序和进程，如果内存不足会导致程序崩溃，对于满足内存容量的程序来说，主机内存容量的大小不是限制任务执行效率的瓶颈，因此，内存大小属于容量指标；内存频率和 CPU 频率类似，指内存执行其操作的速度，高频率的内存能够更快地处理数据，属于性能指标；内存位宽指内存模块和内存控制器之间一次能够并行传输数据的位数，位宽直接影响每个时钟周期内可以传输的数据量，从而影响总的数据传输速率，属于性能指标。

②智能算力指标：智能算力指标涵盖 GPU 和显存相关的参数。主要包括 GPU 频率、核心数量、一级缓存、二级缓存、显存频率、显存位宽、显存容量。GPU 频率同 CPU 频率类似，频率越高，GPU 处理任务的速度越快，属于性能指标；核心数量是指内部的处理单元数量，GPU 的优势在于其能够对任务并行处理，每个核心都能同时执行计算任务，核心数量越多，GPU 同时处理数据的能力就越强，属于性能；一级缓存和二级缓存和 CPU 缓存类似，也属于性能指标；显存频率、显存位宽、显存容量与主机内存频率、内存位宽、内存容量类似，前两者属于性能指标，后者属于容量指标。

③存储能力指标：存储能力指标主要包括磁盘相关的性能指标。现有的磁盘生产商大多只公布了磁盘的容量指标，而没有公布其性能指标，因此，采用 fio 磁盘测试工具测量 Read IOPS、Write IOPS、Read 延迟以及 Write 延迟，作为磁盘的性能指标。

④网络能力指标：网络能力指标主要包括网卡相关的性能指标。同磁盘类似，网卡生产商只公布了网卡的带宽，没有提供相关的性能参数，网卡带宽指网卡在单位时间内能够同时处理的最大数据量，在不受 DDos 攻击或不将其用来处理大规模流量等特殊场景下，普通任务一般不会占用整张网卡的带宽，因此，网卡带宽属于容量指标。在网络环境下，网卡延迟和网卡的丢包率会影响网络传输任务的传输时间，因此，使用 ping 命令来测量网卡的延迟和丢包率，作为

网卡的性能指标。

(3) 算力指标体系的构建

基于以上构建过程，构建了如图4-4的算力度量指标体系。通用算力维度下的指标包括CPU频率、CPU一级缓存、CPU二级缓存、CPU三级缓存、内存频率和内存位宽。智能算力维度下的指标包括GPU频率、CPU核心数、GPU一级缓存、GPU二级缓存、显存频率和显存位宽。存储能力维度下的指标包括磁盘Read IOPS、Write IOPS、Read延迟和Write带宽。网络能力维度则包括网卡丢包率和网卡延迟。

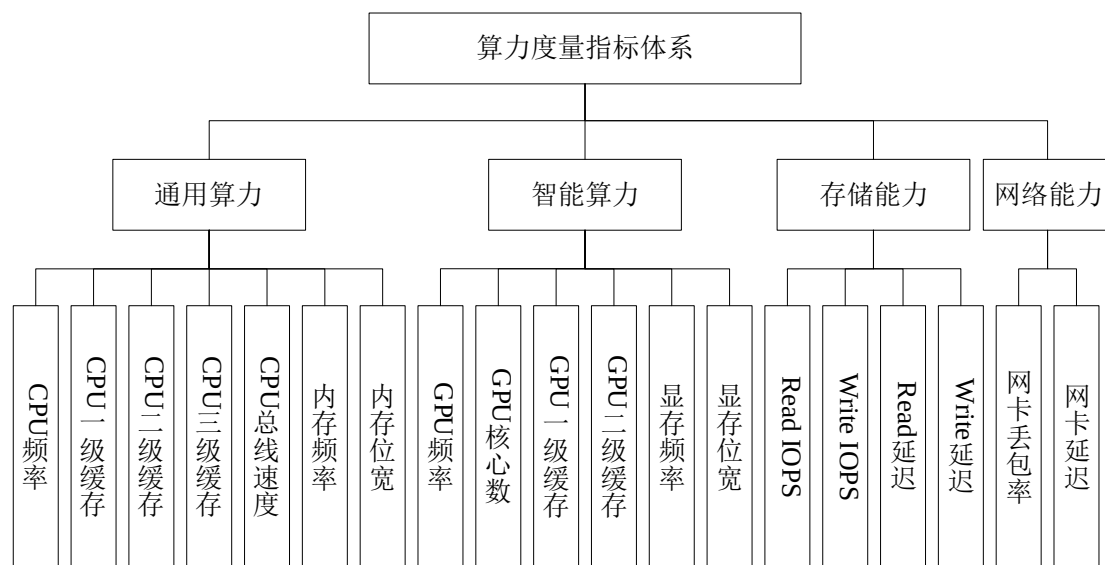


图 4-4 算力度量指标体系

4.2.2 算力度量

本文根据各维度下的算力指标，度量各维度的算力值。算力度量分为两大步骤，第一步需确认各维度下指标的权重，第二步建立TOPSIS模型求解各维度下的算力值。

(1) 基于AHP的算力指标权重计算

AHP (Analytic Hierarchy Process, AHP) 是一种结构化的、量化的决策分析方法。该方法通过将复杂的决策问题分解为不同的层次结构 (如目标、准则、方案等)，在每一层次内对各因素进行成对比较，通过计算比较矩阵的最大特征值对应的特征向量来确定各因素的相对重要性或权重。层次分析法的核心是建立一个层次结构模型，然后通过成对比较的方式，收集判断矩阵，最后利用数

学方法计算出各层次因素对总目标影响的权重。

使用 AHP 计算各维度下指标的权重分为以下 3 个过程：

①构建成对判断矩阵。将每个维度下的指标进行两两比较，得到成对比较矩阵 **A**：

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad (4-1)$$

其中， a_{ij} 表示第 i 项指标相对于第 j 项的重要性，通常采用 1-9 的标度来评估指标之间的相对重要性。

②计算权重向量。该步骤的目的是为了量化指标相对于该维度下算力的重要性。为了确保每个指标的相对重要性得到适当的比较，将判断矩阵进行归一化，得到归一化矩阵 **B**，**B** 中元素 b_{ij} 的计算方式为：

$$b_{ij} = \frac{a_{ij}}{\sum_{k=1}^n a_{kj}} \quad (4-2)$$

其中， a_{ij} 是判断矩阵中的元素， n 是判断矩阵的维度，即指标的个数。根据归一化矩阵 **B** 计算权重向量 **W**，其中每个元素表示为：

$$w_i = \frac{\sum_{j=1}^n b_{ij}}{n} \quad (4-3)$$

③执行一致性检验。该步骤是为了确保决策过程的逻辑性和合理性，有助于发现和纠正决策过程中的偏差或错误。计算一致性指标 CI ，用于衡量判断矩阵的一致性程度，计算方式为：

$$CI = \frac{\lambda_{\max} - n}{n - 1} \quad (4-4)$$

其中， $\lambda_{\max}$ 是矩阵 **A** 的最大特征值。计算一致性比率 CR ，用于评估判断矩阵的一致性是否在可接受，计算方式为：

$$CR = \frac{CI}{RI} \quad (4-5)$$

其中， RI 是随机一致性指数（见表 4-1），根据判断矩阵的阶数 n 选择对应的值。如果 $CR < 0.1$ ，则认为判断矩阵的一致性是可接受的；如果 $CR \geq 0.1$ ，则需要不断调整判断矩阵，直到满足要求为止。

表 4-1 随机一致性指数

阶数 (n)	1	2	3	4	5	6	7	8	9
随机一致性指数 (RI)	0.00	0.00	0.58	0.90	1.12	1.24	1.32	1.41	1.45

(2) 基于 TOPSIS 的算力度量

TOPSIS (Technique for Order Preference by Similarity to Ideal Solution, TOPSIS) 是一种多属性综合评价模型。这种方法基于将决策选项与理想解及负理想解进行比较的原理。理想解代表所有属性或标准上的最佳可能性能, 而负理想解则代表所有属性上的最差可能性能。TOPSIS 认为, 最优的决策选项是那些与理想解最为相似 (即距离最近), 而与负理想解最为不同 (即距离最远) 的选项。TOPSIS 能够明确显示出各个选项与理想情况的相对接近度, 从而帮助决策者做出更加明智的选择。

算力指标分为智能算力、通用算力、存储能力和网络能力四个维度, 在每个维度下, 构建 TOPSIS 模型对各维度的指标进行综合评价, 基于 TOPSIS 的算力度量步骤如下:

①构建算力决策矩阵

假设有 m 个算力节点和 n 个指标, 决策矩阵 $\mathbf{D}$ 可以表示为:

$$\mathbf{D} = \begin{bmatrix} d_{11} & d_{12} & \cdots & d_{1n} \\ d_{21} & d_{22} & \cdots & d_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ d_{m1} & d_{m2} & \cdots & d_{mn} \end{bmatrix} \quad (4-6)$$

其中, 每一行代表一个算力节点, 每一列代表一个指标, d_{ij} 表示第 i 个节点在第 j 个指标的数值。

②归一化决策矩阵

为了便于比较, 需要将决策矩阵转换为无量纲的形式, 使其处于相同的量级。由于存在正向、负向两种类型的指标, 需要分别对不同类型的指标进行归一化处理。

对于正向指标 s_{ij}^+ , 归一化的公式如下:

$$s_{ij}^+ = \frac{d_{ij} - \min(d_j)}{\max(d_j) - \min(d_j)} \quad (4-7)$$

对于负向指标 s_{ij}^- ，归一化的公式如下：

$$s_{ij}^- = \frac{\max(d_j) - d_{ij}}{\max(d_j) - \min(d_j)} \quad (4-8)$$

其中， $\max(d_j)$ 和 $\min(d_j)$ 分别是该指标在节点中的最大值和最小值。

③构建加权决策矩阵

加权决策矩阵 $\mathbf{V} = [v_{ij}]_{m \times m}$ 通过将决策矩阵中的每个元素乘以相应指标的权重来构建。其中 v_{ij} 为 $\mathbf{V}$ 中的元素，计算方式如下：

$$v_{ij} = s_{ij} \times w_j \quad (4-9)$$

其中， w_j 为第 j 项指标的权重，由层次分析法计算得出。

④计算正负理想解

正负理想解分别代表了所有备选方案在各指标上的最优和最差表现，通过比较每个备选方案与这两个理想解的距离评估方案的优劣。正理想解是指在所有评价指标上都取得最有性能值的假想方案，对于每个评价指标，正理想解的值是所有备选方案在该指标上的最佳值，反之，负理想解同理。

正理想解 A_j^+ 的计算公式为：

$$A_j^+ = \max_i v_{ij} \quad (4-10)$$

负理想解 A_j^- 的计算公式为：

$$A_j^- = \min_i v_{ij} \quad (4-11)$$

⑤计算每个节点与正负理想解的距离

计算节点与正负理想解的欧式距离，用于度量每个节点的算力，其能够反映每个节点与理想情况之间的相对贴近度。对于每个节点 i ，其到 A_j^+ 的欧式距离 D_i^+ 可以通过以下方式计算：

$$D_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - A_j^+)^2} \quad (4-12)$$

到 A_j^- 的欧氏距离通过以下方式计算：

$$D_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - A_j^-)^2} \quad (4-13)$$

⑥计算相对贴进度

相对贴进度是指每个节点与正理想解的接近程度相对于正理想解和负理想

解总和的接近程度，是用来衡量每个节点与最优方案的接近程度和与最差方案的远离程度的综合指标。对于每个节点，其相对贴近度 C_i 的计算方式如下，以此作为最终度量的算力值：

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-} \quad (4-14)$$

4.3 资源监控模块设计

为了维护系统稳定运行并获得节点 CPU、GPU、磁盘 IO 带宽、网络带宽实际使用率等关键信息，设计一种基于 Prometheus 的资源监控方案。初步方案在单个集群内部署，以获得详细的监控数据。在此基础上，借鉴 Prometheus 联邦监控策略，设计一个多集群监控框架，整合来自多个成员集群的监控数据，并以粗粒度形式向控制平面集群上报，实现多集群全局监控功能。

4.3.1 单集群资源监控

单集群资源监控采用 Prometheus 提供的监控套件，包括 Prometheus 服务、Node Exporter、GPU Exporter 和 Grafana。单集群监控架构如图 4-5 所示。

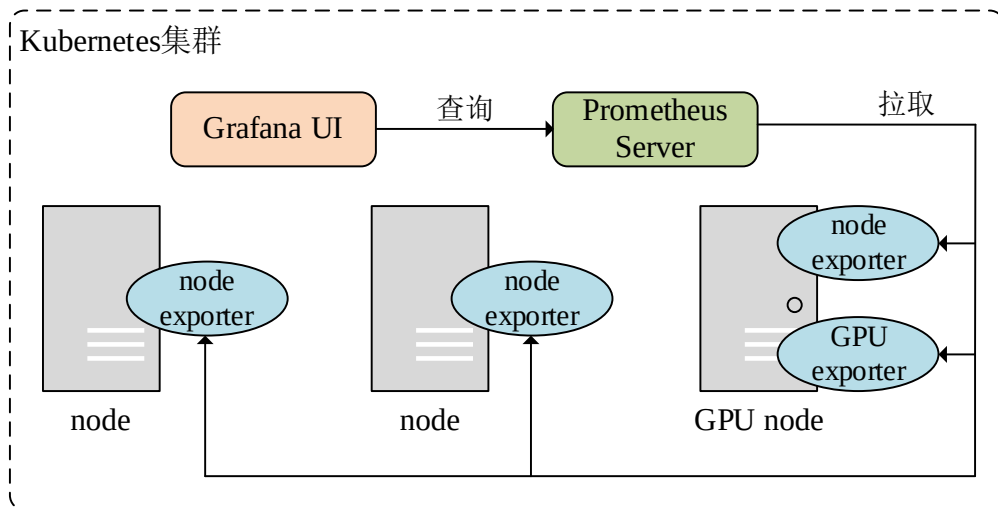


图 4-5 单集群资源监控架构

每个节点上部署 Node Exporter 代理和 GPU Exporter 代理（GPU 节点），用于采集主机、操作系统及 GPU 相关指标；Prometheus 服务定期从 Node Exporter、GPU Exporter 拉取监控数据，存储在本地时序数据库中；Grafana 前端工具定期向 Prometheus 服务查询监控数据，可视化各种资源状态，供管理员查看。

4.3.2 多集群资源监控

多集群资源监控受 Prometheus 联邦监控机制而设计。Prometheus 联邦允许一个 Prometheus 服务从一个或多个其他 Prometheus 服务中抓取数据。这种机制通常用在不同层级或不同地理位置上构建更加复杂的监控架构，联邦特性允许用户建立一个层次化的监控系统，其中高层级的 Prometheus 实例可以抓取来自下层实例的聚合数据，从而实现多集群的监控。然而，这种联邦监控机制会将细粒度的监控数据全部上报给上层的 Prometheus，带来极大的网络资源消耗，且加剧上层 Prometheus 的负载。

本文以扩大监控数据粒度、压缩上报数据量的方式，设计一个多集群资源监控框架（Multi-cluster Monitor, MCM），MCM 架构如图 4-6 所示。

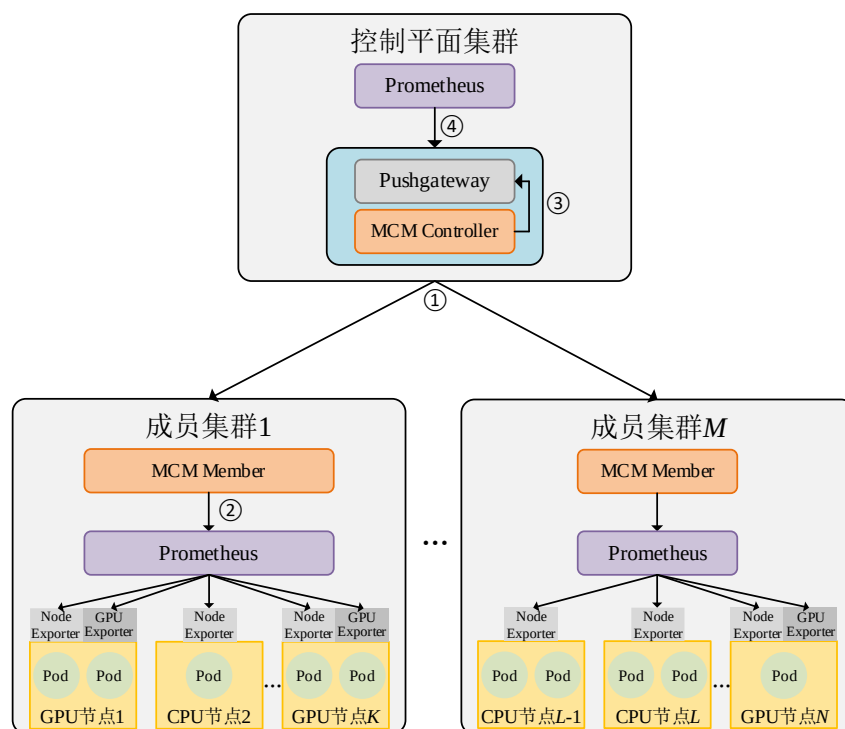


图 4-6 多集群资源监控架构

MCM 依托于 Prometheus 生态系统中的核心组件构建，涵盖 Prometheus 服务、Node Exporter、GPU Exporter 以及 Pushgateway 等。在各个成员集群内部，Prometheus 服务负责拉取由 Node Exporter 和 GPU Exporter 采集到的数据并存储在本地数据库中，以便进行查询和分析。控制平面集群的 Prometheus 服务则负责存储从成员集群聚合而来的监控数据以及该集群自身的监控数据。此外，全

局调度器能够利用控制平面集群内 Prometheus 的数据支持进行调度决策。Node Exporter 和 GPU Exporter 作为部署在各节点上的监控代理，主要功能是公开每个节点在硬件、操作系统以及 GPU 层面相关的监控指标。Pushgateway 则作为一个位于控制平面集群中的中间件，其作用是缓存成员集群上报的数据，并向控制平面的 Prometheus 暴露各成员集群的监控数据。

在多集群监控框架 MCM 中，涉及到两个关键组件：MCM Controller 和 MCM Member。MCM Controller 负责从成员集群中拉取数据，添加集群编号标签后将数据推送到 Pushgateway。MCM Member 的职责是使用 PromQL 从每个集群的 Prometheus 中汇总数据，提高监控数据的粒度，并向控制平面集群上报数据。MCM Controller 和 MCM Member 之间传输的数据使用 gzip 进行压缩。具体工作流程如图 4-7 所示：

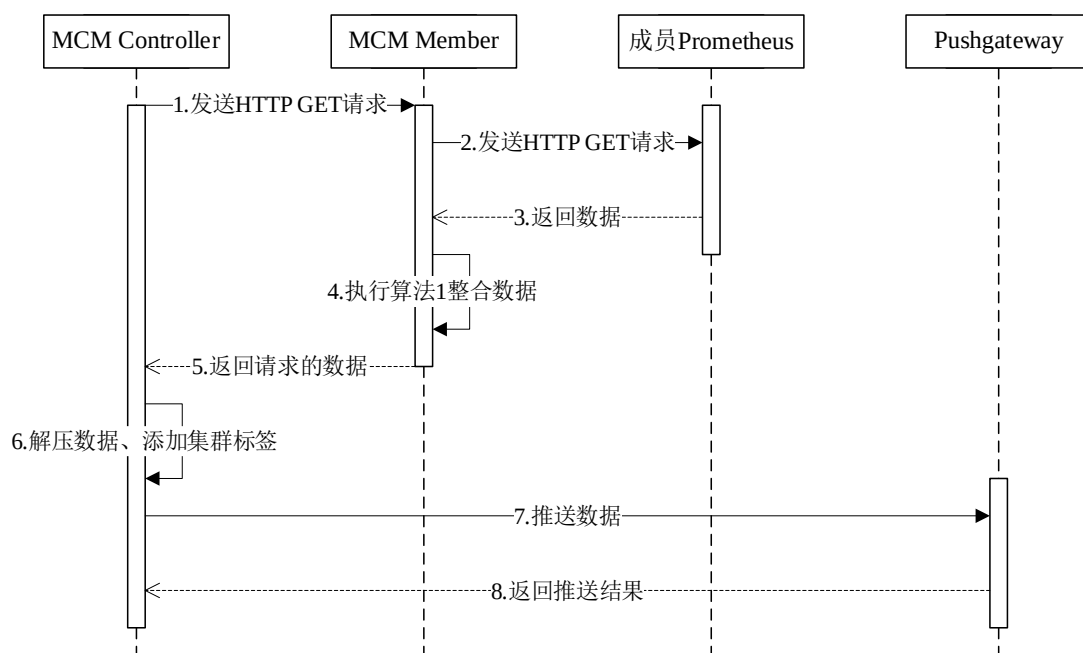


图 4-7 多集群资源监控时序图

当 MCM Controller 需要获取数据时，会向 MCM Member 发送请求，MCM Member 收到请求后，通过 HTTP GET 使用 PromQL 的方式向 Prometheus 汇总监控数据，执行算法 1 整合监控数据后，将最终的数据发送给 MCM Controller，MCM Controller 收到数据后对其进行解压缩，为数据添加源集群标签，并以

HTTP POST 的方式将数据推送至 Pushgateway。控制平面的 Prometheus 服务定期从 Pushgateway 中抓取数据，将其存储到本地数据库中，供调度决策、全局监控需求使用。

对于控制平面集群，主要关注主机粗粒度的监控数据，如 CPU 总量、CPU 使用量等，而不关注细粒度的主机数据，如每个 CPU 核的使用量、每个物理磁盘块的总量等。在成员集群中，使用 Prometheus 支持的查询语言 PromQL 来对细粒度数据进行汇总，需要的监控数据信息如表 4-2 所示：

表 4-2 多集群监控指标信息

指标	解释
<i>node_cpu_total</i>	CPU 总量（核）
<i>node_cpu_usage</i>	已使用的 CPU 量（核）
<i>node_memory_total</i>	内存总量（Mi）
<i>node_memory_usage</i>	已使用的内存量（Mi）
<i>node_disk_total</i>	磁盘总量（Mi）
<i>node_disk_usage</i>	已使用的磁盘量（Mi）
<i>node_disk_io_utilization</i>	磁盘 IO 使用率
<i>node_network_bandwidth_utilization_rate</i>	网络带宽使用率
<i>node_gpu_count</i>	GPU 数量
<i>node_gpu_utilization_rate</i>	GPU 使用率
<i>node_gpu_memory_total</i>	GPU 内存总量
<i>node_gpu_memory_usage</i>	已使用的 GPU 内存量

Prometheus 中的指标数据模型由指标名、不定数量的标签键值对以及指标值组成，指标模型可以表示为：

$$metric_name\{label_name=label_value,\dots\}metric_value \quad (4-15)$$

其中，*metric_name* 为指标名，*label_name* 为指标携带的标签，*metric_value* 为指标对应的具体数值。

可以看出，每个指标附带了多个标签数据，这些标签数据在单个集群内用于定位指标。但在多集群场景下，只关心指标来自哪个集群、哪个节点；同时，诸如 CPU 核心数、GPU 数量等指标在不改变节点硬件配置的情况下通常不会发生变化。因此，为了减少 MCM Member 和 MCM Controller 之间的数据传输量，设计了算法 1，对监控数据去除无用的标签值、避免传输没有变化的数据。第 4-11 行过滤掉除节点名以外的无效指标，第 12-14 行判断本次查询的数据是否发生变化，如果未变化，则不发送。

算法 1 多集群监控数据整合算法

输入: query_metrics: 执行 PromQL 返回的数据; latest_send_metrics: 最新发送的指标集合

输出: require_send_metrics: 需要发送的数据

```

1: latest_send_metrics ← empty list, require_send_metrics ← empty list
2: for each metric in query_metrics do:
3:   del_label_indexes ← empty list
4:   for each label in metric[labels] do:
5:     if label is not "instance" then:
6:       del_label_indexes.append(label)
7:     end if
8:   end for
9:   for each index in del_label_indexes do:
10:    metric[labels].remove(index)
11:   end for
12:   if metric in latest_send_metrics or
    metric[value] is not latest_send_metrics[metric_name][value] then:
13:     require_send_metrics.push(metric)
14:   end if
15: end for

```

4.4 资源接入与限制模块设计

在现代算力调度系统中，Kubernetes 被广泛作为算力调度框架，以优雅地管理和分配计算资源，任务可以通过声明式方式向 Kubernetes 请求所需的资源。默认情况下，Kubernetes 仅支持申请 CPU 和内存资源，然而，在现代计算机系统中，除了 CPU 和内存之外，还有诸如 GPU、磁盘 IO 带宽和网络 IO 等关键资源，这些资源对于 Kubernetes 是不透明的。因此，需要针对 GPU、磁盘 IO、网络带宽等资源设计专门的管理和限制机制，以便更精准地为 Pod 分配资源，从而最小化资源浪费并提高系统运行效率。

4.4.1 GPU 资源接入

Kubernetes 作为当今最流行的容器编排系统，默认不支持对 GPU 资源的识别和管理。随着 Kubernetes 在云计算领域的快速发展，各类专用硬件资源供应商通过 Kubernetes 暴露的插件 API 实现了其专用硬件的设备插件，使得专用硬件可以作为一种资源暴露给 Kubernetes，供 Kubernetes 使用。

针对 Kubernetes 中对 GPU 的管理，采用 NVIDIA 官方提供的 NVIDIA Device Plugin 实现 GPU 资源接入 Kubernetes。如图 4-8 所示，NFD (Node Feature Discovery) 以 DaemonSet 的方式部署在每个节点上，自动探测每个节点的硬件特性，包括 CPU、GPU 等的具体型号、数量等信息，将探测到的特征以标签

(Labels) 的形式添加到对应节点的元数据中；设备插件根据 NFD 标记的信息，识别出节点上可用的 GPU 后向 Kubelet 注册自己，声明其可以管理的 GPU 资源，通过与 Kubelet 通信，设备插件将节点上可用的 GPU 数量和类型公告给 Kubernetes，并将这些信息作为调度决策的一部分；当 Pod 需要请求 GPU 资源时，设备插件根据 Pod 请求为其分配适当的 GPU 资源，GPU 以挂载的方式供 Pod 使用，确保 Pod 在启动时获得所需的 GPU 资源。

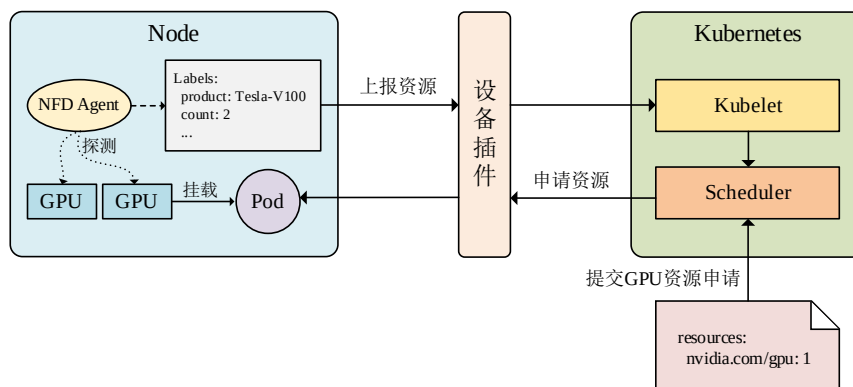


图 4-8 GPU 资源管理架构图

在设备插件中，默认不支持 GPU 共享，会导致显卡独占，即一个 Pod 独占一块 GPU，导致 GPU 资源的利用率极低。NVIDIA 的设备插件中提出一种 GPU Time-Slicing 的资源共享技术，允许多个容器共享同一个 GPU，每个容器获得一段时间的 GPU 使用权，然后将控制权传递给下一个容器，能够极大提高 GPU 的利用率。因此，为解决 Pod 独占 GPU 的问题，需要配置开启设备插件的 Time-Slicing 模式，并设置 GPU 向 Kubernetes 暴露的副本数。

4.4.2 网络带宽资源限制

在 Kubernetes 环境中，对 Pod 网络带宽的管理是提高资源利用率和保证服务质量的重要环节。尽管 Kubernetes 提供了丰富的资源管理功能，如 CPU 和内存的请求与限制，但在网络管理方面的支持却相对薄弱。

为了后续对任务实现精细化的网络带宽资源分配，基于网络带宽管理工具 Wondershaper^[43]设计一个 Pod 网络带宽管理组件，实现限制 Pod 能够使用的网络带宽量。Wondershaper 使用 Linux TC (Traffic Control) 提供的功能来限制网络接口的上传和下载带宽，通过创建排队规则和类别，以及匹配到这些类别的过

滤波器来工作，指定入站和出站带宽限制。使用 `Wondershaper` 限制带宽的示例命令为：`wondershaper <dev> 10000 10000`，表示将名为 `dev` 的网卡的出站带宽和入站带宽限制为 `10000Kbps`，其中 `<dev>` 为网络接口名称，`10000` 指以 `Kbps` 为单位的带宽限制，第一个数字是入站带宽限制，第二个数字是出站带宽限制。

如图 4-9 所示，Pod 网络带宽管理组件由网络带宽控制器和节点代理组成。用户通过 Kubernetes 客户端创建 Pod，网络带宽控制器通过 Kubernetes API 监听 Pod 的创建和更新事件，当有 Pod 创建或更新时，Kubernetes API 通知网络带宽控制器，网络带宽控制器接收到事件后，通过 `calicoctl` 工具获取 Pod 所在节点、网络接口名称、带宽限制参数等信息，并将其发送给节点代理，节点代理收到网络带宽控制器发送的命令后，执行 `wondershaper` 命令生成 TC 规则限制 Pod 网络接口的网络带宽，以实现 Pod 的网络带宽限制。

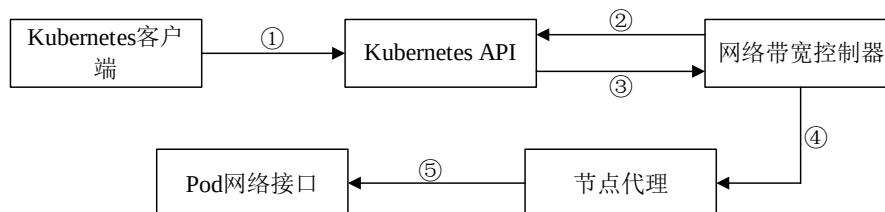


图 4-9 Pod 带宽限制原理

4.4.3 磁盘 IO 资源限制

上一小节中，针对 Kubernetes 只存在对 CPU 和内存资源的管理机制，设计了一个用于限制网络带宽的组件。在磁盘 IO 领域，Kubernetes 同样不支持对磁盘 IO 的限制功能，无法对 Pod 所使用的磁盘 IO 资源进行精细化管理。因此，设计一个能够在 Kubernetes 中限制 Pod 磁盘 IO 的策略变得至关重要。

本文使用 Kubernetes 提供的生命周期钩子来实现对 Pod 使用的磁盘 IO 进行限制。在 Kubernetes 容器中，生命周期钩子允许容器管理器在生命周期中的某些阶段执行操作，例如创建、启动或终止。这些钩子使得在容器的不同阶段自定义操作成为可能，例如初始化设置、资源释放或其他必要的清理工作。Kubernetes 提供了两种主要的生命周期钩子：`postStart` 和 `preStart`。`postStart` 是一个在容器创建后立即执行的钩子，这个钩子指示 Kubernetes 在容器的进程启动后立即执行一个指定的操作或脚本。

使用 `postStart` 钩子实现对 Pod 磁盘 IO 进行限制的工作原理如下：以 Kubernetes 容器启动的动作作为整个过程的触发点，`postStart` 生命周期钩子会在容器启动后立即执行；在 `postStart` 钩子中，执行如图 4-10 所示的两条命令来完成对 Pod 所能使用的磁盘 IO 量限制，第一条命令是限制 Pod 对磁盘的写入带宽，第二条命令是限制 Pod 对磁盘的读带宽；在 `cgroup` 层面，对应的限制（写和读 IO 限制）被应用到容器所在的 `cgroup`，`postStart` 钩子执行完磁盘 IO 限制设置后，容器继续执行其主要的任务或应用。

```
①echo '${DEVICE_MAJOR_MINOR} ${IO_WRITE_LIMIT}' > /sys/fs/cgroup/blkio/blkio.throttle.write_bps_device
②echo '${DEVICE_MAJOR_MINOR} ${IO_READ_LIMIT}' > /sys/fs/cgroup/blkio/blkio.throttle.read_bps_device
```

图 4-10 磁盘 IO 限制命令

部署任务时，需在 YAML 文件中指定如表 4-3 所示的环境变量，其中 `DEVICE_MAJOR_MINOR` 所需的参数可用“`lsblk`”命令查看，另外，需要设置 `securityContext.privileged` 为 `true`，为容器开启特权模式。

表 4-3 限制磁盘 IO 所需的环境变量

环境变量名	解释
<code>IO_WRITE_LIMIT</code>	Pod 能够使用的最大磁盘写带宽
<code>IO_READ_LIMIT</code>	Pod 能够使用的最大磁盘读带宽
<code>DEVICE_MAJOR_MINOR</code>	宿主机硬盘的主设备号和次设备号

4.5 算力调度模块设计

在多集群环境下，业界普遍使用 Karmada 来实现多个 Kubernetes 的管理及算力调度。但是，Karmada 进行调度决策时只能在集群层面进行任务的分发，无法感知每个 Kubernetes 集群下各节点的资源。因此，本文针对在 Karmada 所管理的多 Kubernetes 集群环境下，根据 MCM 全局监控框架提供的全局监控指标以及各节点的多维算力值，在 Karmada 控制平面设计一个全局调度策略，以提高批次任务的执行效率，并保证全局节点的负载平衡。

4.5.1 算力调度模型构建

如图 4-11 所示，调度模型的输入为带有资源请求的批量多类型任务、来自控制平面集群的全局监控数据、节点多维算力值以及来自多集群异构算力节点。调度目标包括最小化批次任务执行时间和保持全局多维资源的负载平衡，在满

足一定的约束条件下，通过多目标优化算法寻求最优解，得出最优调度方案。

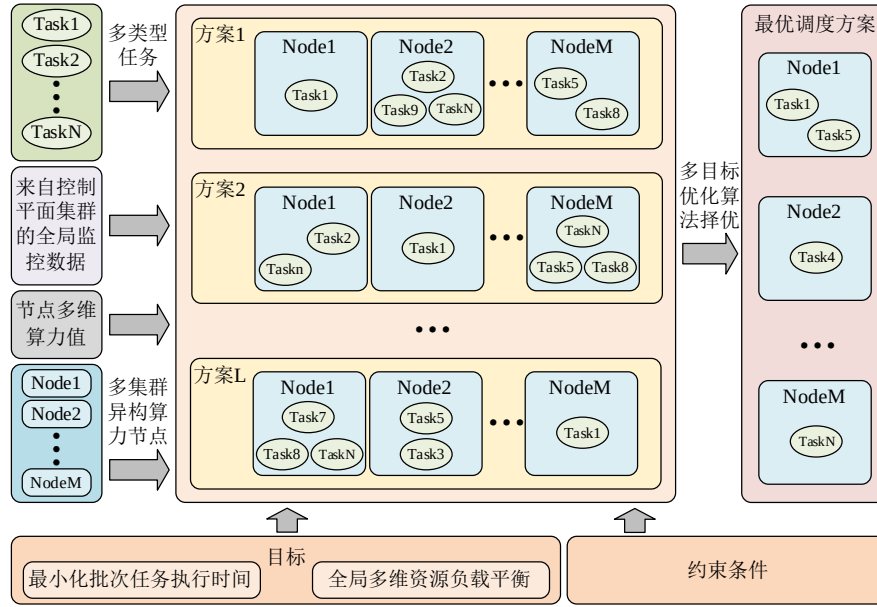


图 4-11 算力调度模型

(1) 调度模型定义

在 Karmada 所管理的多 Kubernetes 集群环境下，假设有 N 个多类型（CPU 密集型、GPU 密集型、磁盘 IO 密集型、网络密集型）任务需要被调度到 M 个异构算力节点上，每个算力节点都有可供其正常运行的资源（包括 CPU、Memory、磁盘、网卡，其中部分节点是 GPU 节点）。

定义 R 表示资源类型集合， $R = \{CPU, GPU, Mem, Disk, Net\}$ ；定义 C 表示算力类型集合， $C = \{General, Intelligent, Storage, Network\}$ ，其中 *General* 表示通用算力，*Intelligent* 表示智能算力，*Storage* 表示存储能力，*Network* 表示网络能力；定义节点集合为 $N = \{n_1, n_2, n_3, \dots, n_M\}$ ，其中 n_j 为第 j 个节点，每个节点可表示为三元组 $\langle p_c, u_r, a_r \rangle$ ，其中 $c \in C$ ， $r \in R$ ， p_c 表示节点在 $c \in C$ 算力类型下的算力值， u_r 和 a_r 分别表示节点在 $r \in R$ 资源类型下的使用率和总量。

定义 E 表示任务类型集合， $E = \{cpu, gpu, disk, network\}$ ；定义任务集合为 $T = \{t_1, t_2, t_3, \dots, t_N\}$ ，其中 t_i 为第 i 个需要被调度的任务，每个任务 t_i 表示为 $t_i = \{size, type, d_r\}$ ，其中 *size* 表示任务大小， $type \in E$ 表示任务类型，任务类型分为 CPU 密集型、GPU 密集型、磁盘 IO 密集型、网络密集型， d_r 表示任务在某个资源类型下所需的资源量。

(2) 调度问题定义

假设 T 中的每个任务是相互独立的, 不存在任务之间的依赖关系。设 x_i 表示第 i 个任务分配到的节点的编号, 每个任务 t_i 可以分配给任意一个节点 n_j , 因此, 将这些待调度任务调度到算力节点上的调度方案可以表示为:

$$X = \{x_1, x_2, x_3, \dots, x_N\} \quad (4-16)$$

其中, 每个元素 x_i 的值是一个介于 1 到 M 之间的整数, 表示对应任务分配的节点编号。

(3) 约束条件设计

在执行 Kubernetes 任务调度时, 可能存在不符合实际情况的调度方案, 因此, 需设计约束条件来规范调度方案的配置, 避免发生调度失败的情况。约束条件有以下三个:

①最大资源限制约束: 为了满足算力节点各维度下的资源限制, 保证任务正常运行, 避免将任务调度到无法满足其资源需求的节点上, 在执行任务调度时, 将 CPU、GPU、内存、磁盘 IO、网络带宽纳入约束指标, 任务申请的资源量不得超过节点剩余的资源量, 约束条件如下所示:

$$\sum_{i=1}^N \left(\frac{d_{i,r}}{a_{r,j}} + u_{r,j} \right) \cdot y_{i,j} \leq 1, \forall j = 1, 2, 3, \dots, M; \forall r \in R \quad (4-17)$$

其中, $d_{i,r}$ 表示任务 t_i 对资源 $r \in R$ 的需求量; $a_{r,j}$ 表示节点 n_j 在 $r \in R$ 下的资源总量; $y_{i,j}$ 是一个决策变量, 如果任务 t_i 分配给节点 n_j , 则 $y_{i,j} = 1$, 否则, $y_{i,j} = 0$ 。

②调度唯一性约束: 执行任务调度时, 确保每个任务只能被调度到一个节点, 避免同一个任务调度到多个算力节点重复执行, 导致资源浪费, 约束条件如下所示:

$$\sum_{j=1}^M y_{i,j} = 1, \forall i = 1, 2, 3, \dots, N \quad (4-18)$$

③GPU 任务约束: 对于 GPU 类型任务, 在没有 GPU 算力的节点上运行会导致任务执行失败, 因此, 对于 GPU 任务, 如果任务的待调度节点是非 GPU 节点, 则将 $y_{i,c}$ 置为 0, 其中 c 表示非 GPU 节点的编号。

(4) 多目标适应度函数设计

算力调度模型的核心是从所有的调度方案中找到一个同时满足多个目标的折中点, 进而找到一个最优的调度方案。因此, 结合本文的调度需求, 需要设

计适应度函数供调度算法寻优，本文考虑了批次任务执行时间以及多维资源负载均衡度作为评价指标。

每个任务的执行时间可以根据任务大小、任务类型、以及对应任务类型的节点算力类型的算力值进行估算。CPU 密集型、GPU 密集型、磁盘密集型和网络密集型任务对应的节点算力类型分别为通用算力、智能算力、存储能力和网络能力。每种算力类型任务执行时间的计算方式如下所示：

$$time_{i,type} = \frac{size_i}{c_j} \quad (4-19)$$

其中， $type \in E$ ， $size_i$ 表示 t_i 的任务大小， c_j 表示节点 n_j 对应任务类型的算力值。

由于算力度量模块中度量的各维度的算力是相互独立的，不在同一个量纲，因此在估算整体的任务执行时间时，需要将任务分类，并以各类别中执行时间最长的任务的平均执行时间作为该批次任务执行时间的适应度函数，整体任务执行时间的适应度函数计算过程如下。计算每个类别中任务执行时间的最大值：

$$Time_{max,type} = \max_{type \in E} \{time_{i,type}\} \quad (4-20)$$

整体任务执行时间的适应度函数表示为：

$$F_T = \text{avg}_{type \in E} \{Time_{max,type}\} \quad (4-21)$$

Kubernetes 默认调度策略仅使用 CPU 和内存作为调度依据，未对 GPU、磁盘 IO 和网络带宽进行考量；且在进行调度时，使用 Kubernetes 默认的静态资源状态作为依据，与节点实际资源使用情况不符。因此，本文从节点实际的资源使用情况出发，以控制平面集群中 Prometheus 的全局监控数据作为数据源，将节点 GPU、磁盘 IO、网络带宽纳入评价指标，作为设计负载适应度函数的基础。

每个节点下有 CPU、内存、磁盘 IO、网络带宽或 GPU 资源，假设任务已成功调度，对于普通节点，节点的综合资源使用率可以表示为：

$$U_j = \frac{1}{4} \left(\frac{\sum_{i=1}^s d_{i,r}^{cpu}}{a_{r,j}^{cpu}} + u_{r,j}^{cpu} + \frac{\sum_{i=1}^s d_{i,r}^{mem}}{a_{r,j}^{mem}} + u_{r,j}^{mem} + \frac{\sum_{i=1}^s d_{i,r}^{disk}}{a_{r,j}^{disk}} + u_{r,j}^{disk} + \frac{\sum_{i=1}^s d_{i,r}^{net}}{a_{r,j}^{net}} + u_{r,j}^{net} \right) \quad (4-22)$$

对于 GPU 节点，节点的综合资源使用率可以表示为：

$$U_j = \frac{1}{5} \left(\frac{\sum_{i=1}^s d_{i,r}^{cpu}}{a_{r,j}^{cpu}} + u_{r,j}^{cpu} + \frac{\sum_{i=1}^s d_{i,r}^{gpu}}{a_{r,j}^{gpu}} + u_{r,j}^{gpu} + \frac{\sum_{i=1}^s d_{i,r}^{mem}}{a_{r,j}^{mem}} + u_{r,j}^{mem} + \frac{\sum_{i=1}^s d_{i,r}^{disk}}{a_{r,j}^{disk}} + u_{r,j}^{disk} + \frac{\sum_{i=1}^s d_{i,r}^{net}}{a_{r,j}^{net}} + u_{r,j}^{net} \right) \quad (4-23)$$

其中， s 为调度到节点 n_j 上的任务数， $d_{i,r}^{cpu}, d_{i,r}^{gpu}, d_{i,r}^{mem}, d_{i,r}^{disk}, d_{i,r}^{net}$ 分别为任务申请的 CPU、GPU、内存、磁盘 IO、网络带宽量； $a_{r,j}^{cpu}, a_{r,j}^{gpu}, a_{r,j}^{mem}, a_{r,j}^{disk}, a_{r,j}^{net}$ 分别为节点 CPU、GPU、内存、磁盘 IO、网络带宽的总量； $u_{r,j}^{cpu}, u_{r,j}^{gpu}, u_{r,j}^{mem}, u_{r,j}^{disk}, u_{r,j}^{net}$ 分别为节点 CPU、GPU、内存、磁盘 IO、网络带宽的使用率。

所有算力节点的平均资源使用率计算方式如下所示：

$$U_{avg} = \frac{\sum_{j=1}^M U_j}{M} \quad (4-24)$$

全局的资源负载均衡使用标准差来衡量，全局的负载均衡标准差计算公式如下所示：

$$S = \sqrt{\frac{1}{N} \sum_{j=1}^N (U_j - U_{avg})^2} \quad (4-25)$$

最大化多维资源负载均衡度目标函数的计算公式如下所示：

$$F_B = \frac{1}{S+1} \quad (4-26)$$

因此，综合公式(4-21)和公式(4-26)，算力调度模块最终的适应度函数如下所示：

$$\begin{cases} F_T = \text{avg}_{type \in E} \{Time_{\max, type}\} \\ F_B = \frac{1}{S+1} \end{cases} \quad (4-27)$$

结合本文的调度目标可知，需要分别最小化和最大化 F_T 和 F_B 。因此， F_T 越小、 F_B 越大，适应度得分越高。

4.5.2 多目标优化算力调度

由上述分析可知，本文的算力调度是一个多目标优化问题，实现算力调度需要一个多目标优化算法来驱动，本文基于 C-TAEA（双档案进化的约束多目标优化算法, Two-Archive Evolutionary Algorithm for Constrained Multiobjective

Optimization) ^[44]算法实现系统的算力调度。C-TAEA 是一种专门设计用于处理带约束的多目标优化问题的方法，旨在解决多目标优化问题，即同时优化多个互相冲突的目标。C-TAEA 使用两个档案来存储解，一个用于可行解，另一个用于不可行解。这使算法能够在搜索空间探索和利用优质解之间保持平衡。

(1) 个体编码方式

基因型是指单独个体的基因组成，通常用二进制编码来表示。在本文的调度问题中，每个基因表示一个任务在算力节点中的调度方案，一个基因型通常由多个基因组成，表示集群中任意一个节点。个体是指基因型的集合，可以看作是一个基因型向量。在本文的调度模型中，一条染色体表示一批任务的调度方案，如果有 N 个任务需要被调度到 Karmada 所管理的某个 Kubenretes 节点上，则一条染色体包含 N 个基因，每个基因表示一个任务的调度位置。

本文以整数序列的形式对任务的调度部署进行编码，编码的长度为单批次任务的数量。如图 4-12 所示，假设当前有 N 个任务需要被调度到 M 个节点上，则一组任务的调度方案就是一个个体，可以表示为 $[1, M, 1, \dots, 2]$ 。

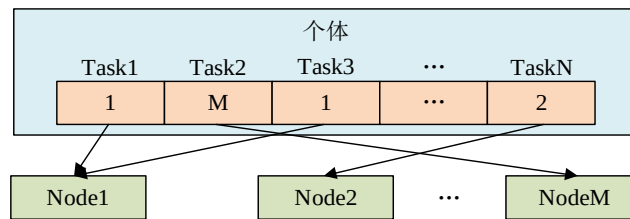


图 4-12 个体编码方式

(2) 初始化种群生成

初始化函数的主要任务是生成算法开始搜索前的初始种群，这个种群由一系列可能的解（个体）组成。C-TAEA 默认随机生成一组初始化种群，随机产生的解很有可能无法满足可行的调度方案。为了提高算法的搜索效率、加速算法的收敛、提升解的质量，以达到更高效的调度效果，根据上一节中的约束条件设计一个满足约束条件的初始化种群算法。

如算法 2 所示，1-19 行定义了生成初始化种群的过程，通过迭代的方式为每个任务找到兼容的节点（包括检查资源的可用性、为 GPU 任务找到具有 GPU 的节点），确保个体是可行的，然后将其添加到初始化种群中。第 20-28 行定义了 IsResSufficient 函数，用于检查节点是否有足够资源来满足一个特定任务的需

求, 考虑了节点的资源使用率。第 29-36 行定义了函数 `IsFeasible`, 用于检查所有任务分配是否都是可行的。

算法 2 满足约束条件的初始化种群算法

输入: `tasks`: 任务集合; `nodes`: 节点集合; `total_resources`: 节点资源总量; `resource_utilization_rate`: 节点资源使用率

输出: `population`: 一个初始种群, 其中每个个体表示一个可行的任务分配方案

```

1: population ← empty set
2: for i ← 1 to populationSize do
3:   individual ← empty list
4:   for each task in tasks do
5:     compatible_node ← empty list
6:     for each node in nodes do
7:       if IsResSufficient(task, node, total_resource[node],
8:         resource_utilization_rate[node]) then
9:         if task.type is not 'gpu' or (task.type = 'gpu' and node.gpu > 0) then
10:          add node to compatible_nodes
11:        end if
12:      end if
13:    end for
14:    selected_node ← get a node randomly selected from compatible_nodes
15:    add(task, selected_node) to individual
16:  end for
17:  if IsFeasible(individual, tasks, total_resources, resource_utilization_rate) then
18:    add individual to population
19:  end if
20: function IsResSufficient(task, node, node_resources, node_utilization)
21:   for each resource_type in {cpu, gpu, memory, disk, network} do
22:     resource_request ← task[resource_type] + node_utilization[node][resource_type] *
23:       node_resources[node][resource_type]
24:     if resource_request > node_resources[resource_type] then
25:       return FALSE
26:     end if
27:   end for
28:   return TRUE
29: end function
30: function IsFeasible(individual, tasks, total_resources, resource_utilization_rate)
31:   for each (task, node) in individual do
32:     if not IsResSufficient(task, node, total_resources[node],
33:       resource_utilization_rate[node]) then
34:       return FALSE
35:     end if
36:   end for
37:   return TRUE
38: end function

```

(3) 选择、交叉和变异

调度算法使用锦标赛选择方法来选择父代个体, 从种群中随机挑选若干个体进行比较, 选出适应度最高的个体。交叉操作采用均匀交叉, 在每个基因位置上独立决定从哪一个父代继承基因, 从而生成多样化的子代个体。变异操作

则采用多项式变异，通过对选定基因位置的小幅度随机扰动来增加种群的多样性，防止算法过早陷入局部最优。

(4) 调度算法流程

调度算法的工作流程如图 4-13 所示。在初始化步骤中，调度对象是一组多类型的任务集合，将任务集合、节点算力值以及与节点相关的监控数据作为算法的输入。在生成初始化种群过程中，对于每个种群的每个个体，通过算法 2 生成满足约束条件的初始化种群。在可行性评估阶段，对每个个体进行可行性检验，如果个体中的任务分配不违反任何节点资源的约束，则认为该个体是可行的，在此基础上，通过交叉、变异和选择操作来进化种群。算法的每一代中，都会使用适应度函数来评估个体的质量，适应度函数包含了两个目标（见公式 (4-27)），适应度高的个体将有更大的机会被保留下来并产生后代。当算法达到设定的代数时终止执行，选择目标 F_T 适应度最高的解作为最终的调度方案。

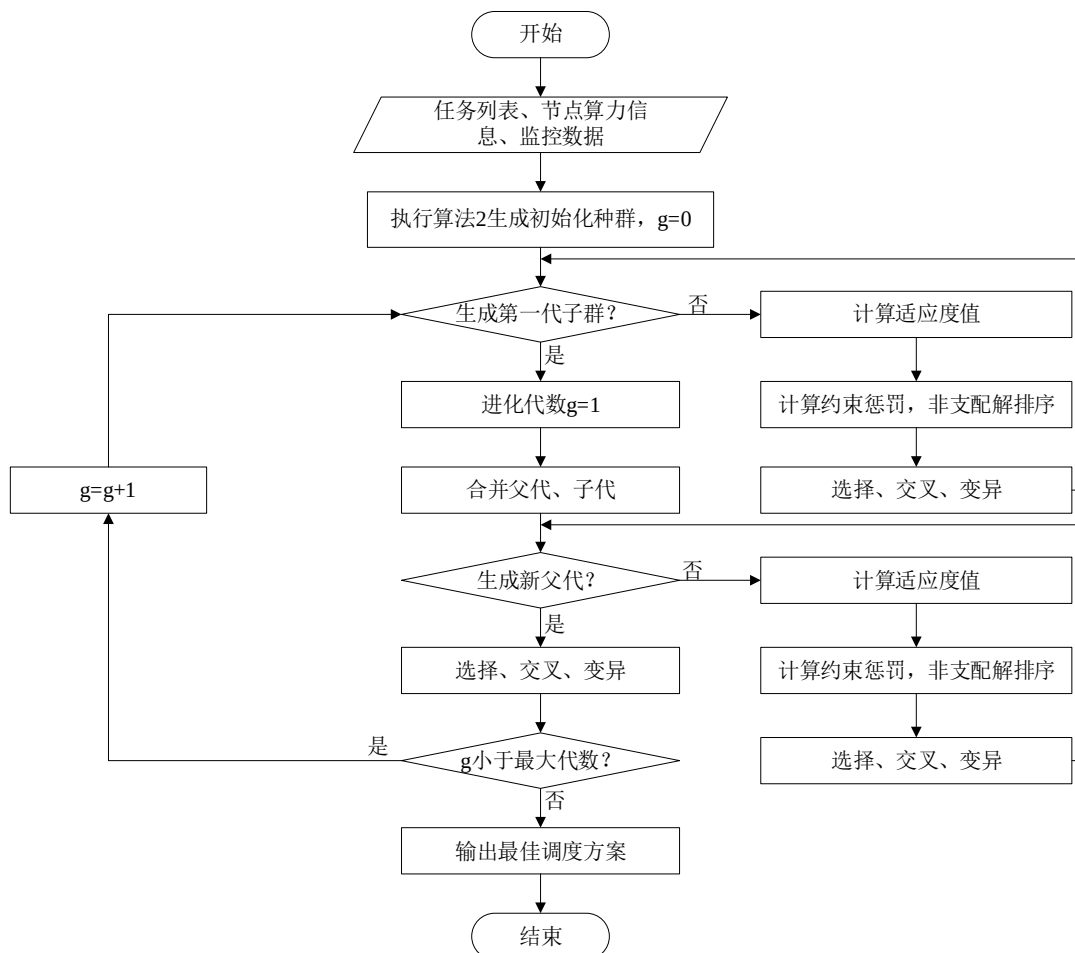


图 4-13 调度算法求解流程图

4.6 本章小结

本章首先分析了多集群异构算力调度系统的总体架构，对系统业务逻辑和功能模块划分进行了介绍。接着对多维细粒度算力度量模块、资源监控模块、资源接入与限制模块、算力调度模块的设计细节进行了详细的介绍。在算力调度模块中，构建了算力度量指标体系，并使用 TOPSIS 综合评价模型度量了节点多维度下的算力，以识别节点间的性能差异；在资源监控模块中，基于 Prometheus 监控技术设计了单集群资源监控，并在此基础上设计了高效的 MCM 多集群资源监控策略，以减少跨集群监控产生的网络流量；在资源接入与限制模块中，将 GPU 资源引入了 Kubernetes 中，使得任务能够访问和使用 GPU 资源，并针对 Kubernetes 资源分配的局限性，引入了磁盘 IO 和网络带宽限制组件，以精细化分配资源；在算力调度模块中，面向多类型任务、算力值、多维资源状态设计了算力调度模型，并采用 C-TAEA 算法求解调度问题，得出任务与节点间的最佳调度方案。基于以上设计，系统能够满足降低任务执行时间和保持资源负载平衡。

第五章 系统实现与测试

5.1 系统部署

本系统运行涉及到许多前沿技术，如 Karmada、Kubernetes、Prometheus、Docker 等，这些技术运行对软硬件环境都有相应的要求，为了实现系统的功能，需要部署复杂的外部环境。本节主要从系统硬件环境、系统软件环境以及系统拓扑三个方面进行系统的运行环境。

5.1.1 系统部署拓扑

为了测试系统功能及性能，构建了如图 5-1 的原型系统。系统由异构的 6 台普通服务器及 3 台 GPU 服务器组成。9 台服务器均分为 3 个 Kubernetes 集群，其中一个集群既作为成员集群，也通过 Karmada 作为控制平面集群管理三个成员集群。成员集群 2 和成员集群 3 中分别有 2 台和 1 台 GPU 服务器。

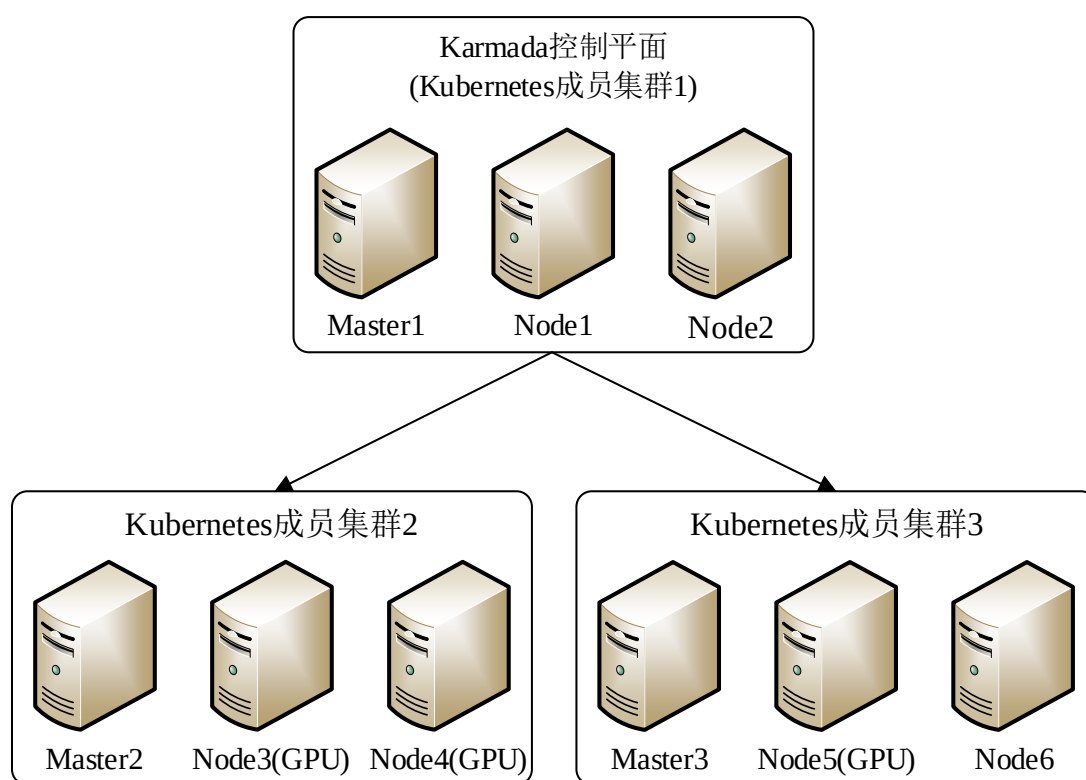


图 5-1 系统部署拓扑图

5.1.2 系统硬件环境

系统各服务器节点的配置信息如表 5-1 所示。其中 Master1、Master2、

Master3 作为 Kubernetes 主节点，不作为任务执行的节点，因此，忽略 3 台 Master 节点的磁盘 IO、网卡带宽性能。

表 5-1 系统硬件环境

集群编号	节点编号	操作系统	CPU	内存	磁盘 IO	网卡带宽	GPU
Member1 (控制平面)	Master1	CentOS7.9	20 核	20GB	-	-	-
	Node1	CentOS7.9	16 核	62GB	300Mi	1000Mi	-
	Node2	CentOS7.9	80 核	125GB	350Mi	1000Mi	-
Member2	Master2	CentOS7.9	20 核	20GB	-	-	-
	Node3	CentOS7.9	24 核	109GB	146Mi	1000Mi	-
	Node4	CentOS7.9	80 核	125GB	63Mi	1000Mi	Tesla V100(32G B)
Member3	Master3	CentOS7.9	10 核	10GB	-	-	-
	Node5	CentOS7.9	80 核	125GB	1000Mi	1000Mi	RTX3090(24GB)
	Node6	CentOS7.9	80 核	156GB	818Mi	1000Mi	RTX2080T i(11GB)

5.1.3 系统软件环境

本系统在开发、运行和测试过程中需要涉及到的软件、测试工具等信息如表 5-2 所示。

表 5-2 系统软件环境

软件	版本	功能
Kubernetes	1.22.8	用于为容器化任务提供基础调度工具
Karmada	v1.8.0	用于管理多个 Kubernetes 集群
Docker	20.10.7	用于给任务提供基础运行环境
Nvidia Driver	535.154.05	用于 Docker 能够识别 GPU
CUDA	12.2	用于提供 GPU 运行时环境
Nvidia Docker	1.0.0-rc95	用于配置 Docker 使用 GPU
Prometheus	v0.72.0	用于搭建系统的基础监控
Node Exporter	v1.7.0	主机指标采集代理
NVIDIA GPU Exporter	0.3.0	GPU 指标采集代理
Grafana	10.3.3	可视化工具
Pushgateway	2.8.0	缓存中间件
NVIDIA Device Plugin	0.15.0-rc.1	用于配置 Kubernetes 使用 GPU
Python	3.8.10	系统主要开发语言
Xshell	7.0077	系统开发工具
tcpdump	4.99.1	网络性能分析工具
Wondershaper	1.2.1	网络带宽限制工具

本系统中，Docker 和 Nvidia Docker 是承载任务的容器化工具，为任务提供运行时环境，Nvidia Docker 是针对 GPU 任务的；Kubernetes 负责对容器化任务

进行编排和管理，为任务提供基础的调度功能；Karmada 负责管理多个 Kubernetes 集群，实现任务跨集群调度的基础功能；Node Exporter 和 GPU Exporter 负责采集主机级别和 GPU 资源的各项指标；Prometheus 负责从 Node Exporter 和 GPU Exporter 拉取监控数据；Grafana 是对接 Prometheus 数据源的可视化工具，将监控数据进行大屏展示；Pushgateway 作为跨集群监控的中间件，用于在 MCM Controller 和 MCM Member 之间缓存跨集群监控数据，并供全局集群的 Prometheus 拉取；NVIDIA Device Plugin 则负责将 GPU 资源暴露给 Kubernetes，使得任务能够在 Kubernetes 环境下使用 GPU 资源；Python 和 Xshell 是系统的主要开发语言和工具；tcpdump 用于对跨集群监控进行性能测试；Wondershaper 是用于限制 Pod 网络带宽的工具。

5.1.4 关键软件安装

本系统开发过程中涉及多种前沿技术，需要提前将系统运行环境配置好。图 5-2 是安装 Docker 和 Kubernetes 的关键步骤。图 5-3 是部署 NVIDIA Device Plugin 设备插件和 Prometheus 监控工具的关键步骤。图 5-4 和图 5-5 是安装 Karmada 和配置多集群管理的关键流程。

<pre>#1.关闭selinux setenforce 0 && sed -i 's/^SELINUX=enforcing\$/SELINUX=permissive/' /etc/selinux/config #2.关闭swap分区 swapoff -a && sed -ri 's/*swap.*#&/' /etc/fstab #3.允许 iptables 检查桥接流量 cat <<EOF sudo tee /etc/modules-load.d/k8s.conf br_netfilter EOF cat <<EOF sudo tee /etc/sysctl.d/k8s.conf net.bridge.bridge-nf-call-ip6tables = 1 net.bridge.bridge-nf-call-iptables = 1 EOF #4.安装时间同步 yum install chrony-3.4-1.el7.x86_64 -y #2.安装依赖及配置源 yum install -y yum-utils yum-config-manager --add-repo http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo #3.安装docker yum install -y docker-ce-20.10.7 docker-ce-cli-20.10.7 containerd.io-1.4.6 #4.安装Nvidia docker(GPU节点) yum install -y nvidia-docker #5.为GPU节点配置docker运行时为nvidia-docker</pre>	<pre>#1.添加Kubernetes源 cat <<EOF tee /etc/yum.repos.d/kubernetes.repo [kubernetes] name=Kubernetes baseurl=http://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64 enabled=1 gpgcheck=0 repo_gpgcheck=0 gpgkey=http://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg http://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg exclude=kubelet kubeadm kubectl EOF #2.安装kubeadm、kubernetes和kubectl yum install -y kubelet-1.22.8 kubeadm-1.22.8 kubectl-1.22.8 --disableexcludes=kubernetes #3.初始化主节点 kubeadm init \ --apiserver-advertise-address=主节点IP \ --control-plane-endpoint=cluster-endpoint \ --kubernetes-version v1.22.8 \ --service-cidr=10.96.0.0/16 \ --pod-network-cidr=192.168.0.0/16 #4.安装网络插件 kubectl apply -f calico.yaml #5.加入node节点 kubeadm join cluster-endpoint:6443 --token xxx --discovery-token-ca-cert-hash sha256:xxx</pre>
--	---

图 5-2 Docker 和 Kubernetes 安装流程

<pre> helm install nvdp nvdp/nvidia-device-plugin \ # 指定安装版本 --version=0.15.0-rc.1 \ # 指定命名空间 --namespace nvidia-device-plugin \ # 如果命名空间不存在, 则创建 --create-namespace \ # 开启gfd插件 --set gfd.enabled=true \ # 加载额外配置 --set-file config.map.config=extra.yaml </pre>	<pre> helm install prometheus-stack prometheus-community/ kube-prometheus-stack \ #指定命名空间 --namespace monitoring \ # 如果命名空间不存在, 则创建 --create-namespace \ # 设置抓取间隔 --set prometheus.prometheusSpec.scrapeInterval=5s \ # 暴露Prometheus服务, 供外部访问 --set prometheus.service.type=NodePort \ # 设置Prometheus暴露的端口 --set prometheus.service.nodePort=30100 \ # 暴露Grafana服务, 供外部访问 --set grafana.service.type=NodePort \ # 设置Grafana暴露的端口 --set grafana.service.nodePort=30101 \ # 部署Node Exporter --set prometheus-node-exporter.enabled=true </pre>
---	--

图 5-3 NVIDIA Device Plugin 和 Prometheus 部署流程

```

# 下载KarmadaCRD资源
wget https://github.com/karmada-io/karmada/releases/download/v1.8.0/crds.tar.gz
# 下载Karmada二进制管理工具
wget https://github.com/karmada-io/karmada/releases/download/v1.8.0/karmadactl-linux-amd64.tgz

# 安装Karmada, 指定镜像地址和组件版本
karmadactl init \
--namespace='karmada-system' \
--port 30000 \
--etcd-image='quay.io/coreos/etcd:v3.5.6' \
--etcd-replicas=1 \
--karmada-aggregated-apiserver-replicas=1 \
--karmada-apiserver-replicas=1 \
--karmada-controller-manager-replicas=1 \
--karmada-kube-controller-manager-replicas=1 \
--karmada-scheduler-replicas=1 \
--karmada-webhook-replicas=1 \
--karmada-aggregated-apiserver-image='docker.io/karmada/karmada-aggregated-apiserver:v1.8.0' \
--karmada-apiserver-image='registry.cn-hangzhou.aliyuncs.com/google_containers/kube-apiserver:v1.22.8' \
--karmada-controller-manager-image='docker.io/karmada/karmada-controller-manager:v1.8.0' \
--karmada-kube-controller-manager-image='registry.cn-hangzhou.aliyuncs.com/google_containers/kube-controller-manager:v1.22.8' \
--karmada-scheduler-image='docker.io/karmada/karmada-scheduler:v1.8.0' \
--karmada-webhook-image='docker.io/karmada/karmada-webhook:v1.8.0' \
--crds='./crds.tar.gz'

```

图 5-4 Karmada 安装流程

```

# 复制成员集群的kube config文件到控制平面集群master节点
scp member1:/root/.kube/config /root/k8scluster/member1_kubeconfig
scp member2:/root/.kube/config /root/k8scluster/member2_kubeconfig
scp member3:/root/.kube/config /root/k8scluster/member3_kubeconfig

# 在控制平面集群master节点以push模式添加成员集群
# 添加成员集群member1
MEMBER_CLUSTER_NAME=$(cat /root/k8scluster/member1_kubeconfig | grep current-context | sed 's:/\n/g' | sed '1d')
karmadactl --kubeconfig /etc/karmada/karmada-apiserver.config join ${MEMBER_CLUSTER_NAME} --cluster-kubeconfig=/root/k8scluster/member1_kubeconfig
# 添加成员集群member2
MEMBER_CLUSTER_NAME=$(cat /root/k8scluster/member2_kubeconfig | grep current-context | sed 's:/\n/g' | sed '1d')
karmadactl --kubeconfig /etc/karmada/karmada-apiserver.config join ${MEMBER_CLUSTER_NAME} --cluster-kubeconfig=/root/k8scluster/member2_kubeconfig
# 添加成员集群member3
MEMBER_CLUSTER_NAME=$(cat /root/k8scluster/member3_kubeconfig | grep current-context | sed 's:/\n/g' | sed '1d')
karmadactl --kubeconfig /etc/karmada/karmada-apiserver.config join ${MEMBER_CLUSTER_NAME} --cluster-kubeconfig=/root/k8scluster/member3_kubeconfig

```

图 5-5 多集群管理配置流程

5.2 系统核心功能实现

本系统核心功能主要有资源监控模块、资源分配模块以及任务分配模块。

其中资源监控模块包括单集群监控和多集群监控，资源分配模块包括 GPU 资源接入、网络带宽限制以及磁盘 IO 限制。各功能具体实现如下所示：

(1) 资源监控模块实现

在单集群内，采用 Node Exporter、GPU Exporter 以及 Prometheus 监控套件实现，在系统软件环境部署时已对其进行了安装。图 5-6 是 Prometheus 成功识别 Node Exporter 和 GPU Exporter 的可视化界面。图 5-7 和图 5-8 是 GPU 和主机级资源状态的可视化面板。

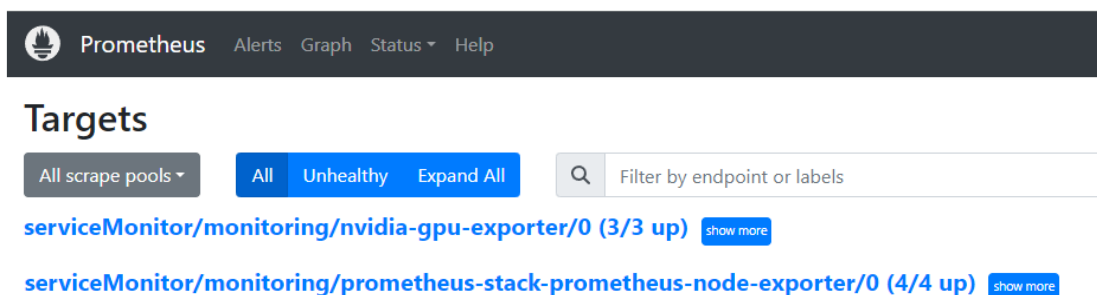


图 5-6 Prometheus 监控指标界面



图 5-7 GPU Exporter 可视化面板

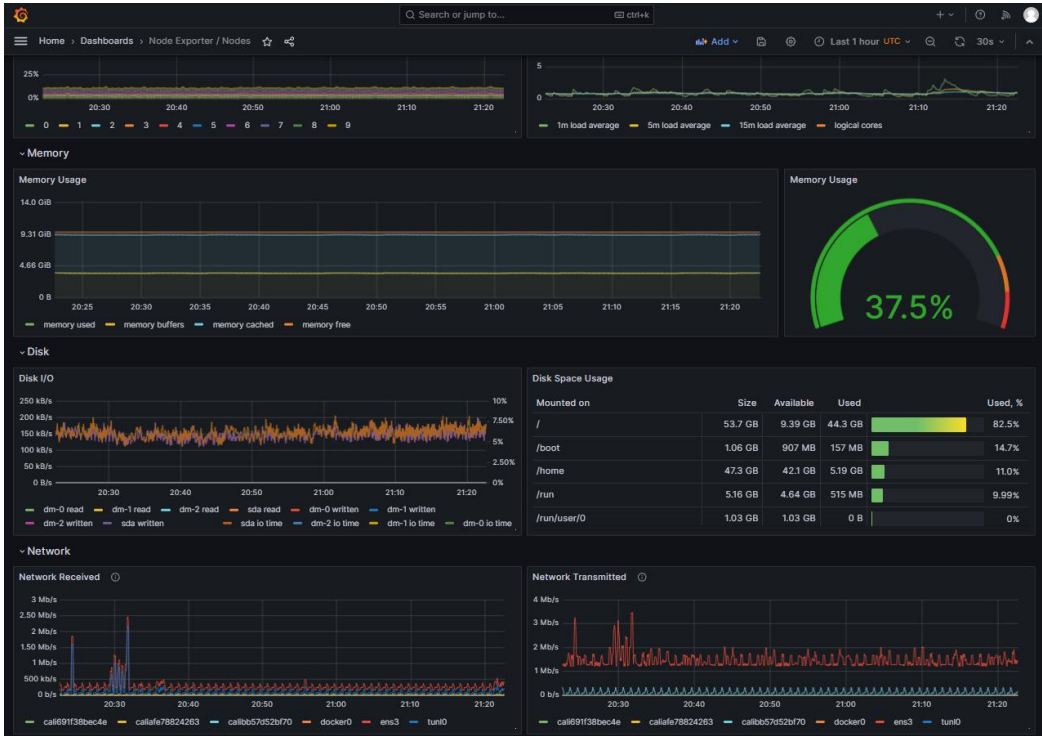


图 5-8 Node Exporter 可视化面板

多集群监控 MCM 框架中涉及到 MCM Member 和 MCM Controller 两个组件，MCM Member 部署在各成员集群中，MCM Controller 部署在控制平面集群中。MCM Member 定期从 Prometheus 查询数据，将数据过滤掉无用标签向 MCM Controller 发送数据，发送前会检查数据是否变化。MCM Controller 接收到数据后为其添加集群标签，并推送到 Pushgateway。图 5-9 和图 5-10 是 MCM 框架的关键代码。

```
# 存储上次发送的数据
latest_metrics = {}
# 从Prometheus查询数据
def fetch_prometheus_data(query):
    response = requests.get('Prometheus地址', params={'query': query})
    return response.json()[0]['data'][0]['result']
# 去除无用标签值，并检查数据是否变化
def clean_and_compare_metrics(metrics):
    global latest_metrics
    cleaned_metrics = {}
    for metric in metrics:
        metric_name = metric['metric']['__name__']
        instance = metric['metric']['instance']
        value = metric['value'][1] # metric value is a list [timestamp, value]
        # 过滤掉除节点名以外的标签
        cleaned_metric = {
            'metric': {'__name__': metric_name, 'instance': instance},
            'value': value
        }
        # 比较数据是否有变化
        if metric_name not in latest_metrics or latest_metrics[metric_name]['value'] != value:
            cleaned_metrics[metric_name] = cleaned_metric
            latest_metrics[metric_name] = cleaned_metric
    return list(cleaned_metrics.values())

# 向MCM Controller发送数据
@app.route('/metrics', methods=['GET'])
def send_metrics():
    # Prometheus的多个查询
    queries = [...]
    metrics_data = []
    for query in queries:
        metrics = fetch_prometheus_data(query)
        cleaned_metrics = clean_and_compare_metrics(metrics)
        metrics_data.extend(cleaned_metrics)
    # 将数据发送到MCM Controller
    controller_url = request.args.get('controller_url')
    if metrics_data: # 如果有数据更新，则发送
        response = requests.post(controller_url, json=metrics_data)
        return jsonify({'status': response.status_code, 'data_sent': metrics_data})
    else:
        return jsonify({'status': 'no_change', 'data_sent': []})
```

图 5-9 MCM Member 关键代码实现

```

@app.route('/receive', methods=['POST'])
def receive_and_push():
    # 接收MCM Member发送的数据
    metrics_data = request.get_json()
    # 为每个指标添加集群标签
    for metric in metrics_data:
        metric['metric']['CLUSTER_LABEL'] = 'my_cluster'
    # 将带有集群标签的数据推送到Pushgateway
    response = requests.post(PUSHGATEWAY_URL,
                             data=json.dumps(metrics_data))
    if response.ok:
        return Response("Data pushed to Pushgateway successfully.", status=200)
    else:
        return Response(f"Failed to push data to Pushgateway. Status code: {response.status_code}", status=500)

```

图 5-10 MCM Controller 关键代码实现

图 5-11 是由 MCM Controller 成功推送数据到 Pushgateway 后的可视化面板。

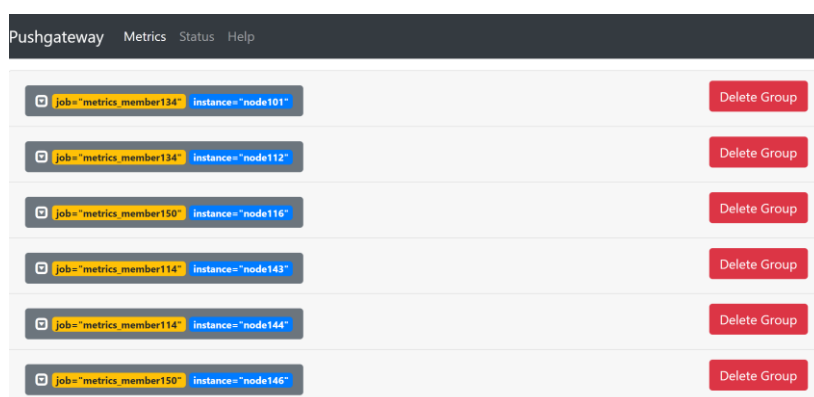


图 5-11 Pushgateway 中缓存的跨集群监控数据

(2) 资源接入与限制模块实现

对于 GPU 资源，在系统软件环境中已安装了 NVIDIA 设备插件，这里主要修改设备插件相关配置，实现 GPU 共享，避免出现独占情况。图 5-12 为 NVIDIA 设备插件修改后的配置，配置 gfd 来周期性发现 GPU 资源，设置 replicas 为 100，向 Kubernetes 报告 100 个 GPU 副本。

```

version: v1
flags:
  ...
plugin:
  ...
  deviceListStrategy: "envvar"
  deviceIDStrategy: "uuid"
gfd:
  oneshot: false
  noTimestamp: false
  outputFile: /etc/kubernetes/node-feature-discovery/features.d/gfd
  sleepInterval: 60s
sharing:
  timeSlicing:
    resources:
      - name: nvidia.com/gpu
        replicas: 100

```

图 5-12 NVIDIA 设备插件配置

网络带宽限制模块包含网络带宽控制器和节点代理两个组件，节点代理以 DaemonSet 的形式部署。网络带宽控制器实时监听 Pod 的创建事件，当有 Pod 创建时使用 calicoctl 工具获取 Pod 所在节点名和其对应的网络接口名并将其发送给节点代理，节点代理收到通知后执行命令限制 Pod 能够使用的网络带宽量。网络带宽限制模块的关键代码实现如图 5-13 和图 5-14 所示。

```
def get_pod_network_details(pod_name, namespace):
    # 使用 calicoctl 获取 Pod 的网络接口名和所在节点名
    cmd = ['calicoctl', 'get', 'workloadendpoints', '-o', 'json', '--namespace', namespace]
    result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True)
    endpoints = json.loads(result.stdout)
    for ep in endpoints:
        if ep['metadata']['labels']['projectcalico.org/podName'] == pod_name:
            return ep['spec']['node'], ep['spec']['interfaceName']
    return None, None

def apply_bandwidth_limit(node_name, interface_name, egress, ingress):
    # 向节点代理发送带宽限制请求
    url = f"http://{node_name}.node.agent:8080/limit"
    data = {'interface': interface_name, 'egress': egress, 'ingress': ingress}
    try:
        response = requests.post(url, json=data)
        response.raise_for_status()
    except requests.RequestException as e:
        print(f"Failed to apply bandwidth limit: {e}")

def watch_pods():
    # 监控 Pod 创建事件并应用带宽限制
    w = watch.Watch()
    for event in w.stream(v1.list_pod_for_all_namespaces):
        if event['type'] == 'ADDED':
            pod = event['object']
            annotations = pod.metadata.annotations
            if 'request-egress-network-bandwidth' in annotations and 'request-ingress-network-bandwidth' in annotations:
                node_name, interface_name = get_pod_network_details(pod.metadata.name, pod.metadata.namespace)
                if node_name and interface_name:
                    apply_bandwidth_limit(node_name, interface_name, annotations['request-egress-network-bandwidth'],
                                          annotations['request-ingress-network-bandwidth'])
```

图 5-13 网络带宽控制器关键代码实现

```
@app.route("/apply-bandwidth-limit", methods=['POST'])
def apply_bandwidth_limit():
    # 解析JSON格式的请求数据
    data = request.json
    interface_name = data.get('interface_name')
    egress_limit = data.get('egress_limit')
    ingress_limit = data.get('ingress_limit')
    if not interface_name or not egress_limit or not ingress_limit:
        return jsonify({"error": "Missing required parameters"}), 400
    try:
        # 清除现有的带宽限制
        subprocess.run(['wondershaper', 'clear', interface_name], check=True)
        # 应用新的带宽限制
        subprocess.run(['wondershaper', interface_name, egress_limit, ingress_limit], check=True)
        return jsonify({"message": "Bandwidth limits applied successfully"}), 200
    except subprocess.CalledProcessError as e:
        return jsonify({"error": str(e)}), 500
```

图 5-14 网络带宽节点代理关键代码实现

(3) 全局调度器实现

全局调度器根据调度模型得出的最佳调度方案将任务分配到与之对应的节点上执行。在控制平面集群中, 使用 `kubectl --kubeconfig` 的方式向 Karmada 提交批量任务。全局调度器监听到任务创建后, 通过 Kubernetes API 读取任务, 并根据最佳调度方案为任务添加 `job.spec.template.spec.node_name` 为需要调度的节点名, 更新任务元数据。之后为任务创建一个 `PropagationPolicy`, 指定节点所在的成员集群名, 实现任务的调度。该模块关键代码实现如图 5-15 所示。

```
# 更新Job的nodeName并创建PropagationPolicy
def update_job_and_create_policy(job_name, namespace, target_node, target_cluster):
    k8s_batch_api = client.BatchV1Api()
    try:
        # 从 Kubernetes API 读取 Job
        job = k8s_batch_api.read_namespaced_job(name=job_name, namespace=namespace)
        # 更新 Job 的 nodeName
        job.spec.template.spec.node_name = target_node
        # 提交 Job 更新
        k8s_batch_api.patch_namespaced_job(name=job_name, namespace=namespace, body=job)
        print(f'Job {job_name} updated to run on node {target_node}.')

        # 创建对应的 PropagationPolicy
        create_propagation_policy(job_name, namespace, target_cluster)
    except ApiException as e:
        print(f'Error updating job: {e}')

# 创建 Karmada PropagationPolicy
def create_propagation_policy(job_name, namespace, target_cluster):
    policy = {
        "apiVersion": "policy.karmada.io/v1alpha1",
        "kind": "PropagationPolicy",
        "metadata": {
            "name": f"{job_name}-policy",
            "namespace": namespace
        },
        "spec": {
            "resourceSelectors": [
                {"apiVersion": "batch/v1", "kind": "Job", "name": job_name}
            ],
            "placement": {
                "clusterAffinity": {
                    "clusterNames": [target_cluster]
                }
            }
        }
    }
    headers = {'Content-Type': 'application/json'}
    url = f"{KARMADA_API_URL}/apis/policy.karmada.io/v1alpha1/namespaces/{namespace}/propagationpolicies"
    # 发送请求到 Karmada API 创建 PropagationPolicy
    response = requests.post(url, json=policy, headers=headers,
                             auth=HTTPBasicAuth(KARMADA_USERNAME, KARMADA_PASSWORD),
                             verify=False)
```

图 5-15 全局调度器关键代码实现

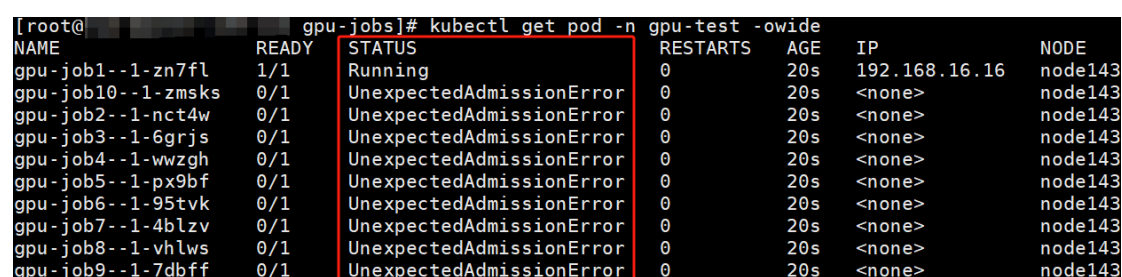
5.3 系统测试

本系统主要包括四部分的测试，分别为系统功能测试、算力度量方法的有效性测试、多集群监控性能测试和算力调度效果测试。

5.3.1 系统功能测试

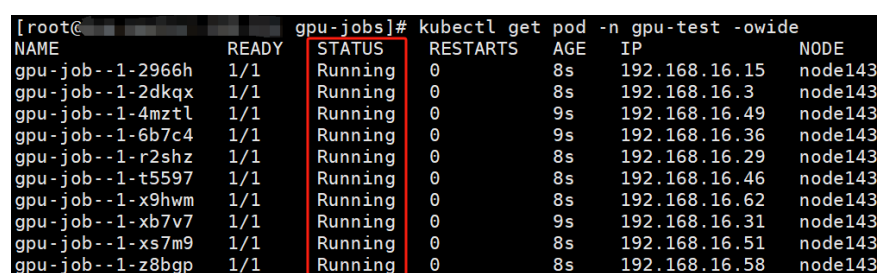
(1) GPU 接入与共享功能测试

Kubernetes 默认不支持访问 GPU 资源，且 Nvidia Device Plugin 默认不支持 GPU 共享，导致显卡独占。图 5-16 和图 5-19 是是否配置 GPU 共享时部署 GPU 任务的测试结果。未配置 GPU 共享时，每块 GPU 被一个 Pod 独占，极大浪费了 GPU 资源；配置 GPU 共享后，一块 GPU 可以同时被多个 Pod 访问，提高了 GPU 资源的利用率。



NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
gpu-job1--1-zn7fl	1/1	Running	0	20s	192.168.16.16	node143
gpu-job10--1-zmsks	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job2--1-nct4w	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job3--1-6grjs	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job4--1-wwzgh	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job5--1-px9bf	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job6--1-95tvk	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job7--1-4blzv	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job8--1-vhlws	0/1	UnexpectedAdmissionError	0	20s	<none>	node143
gpu-job9--1-7dbff	0/1	UnexpectedAdmissionError	0	20s	<none>	node143

图 5-16 未配置 GPU 共享时部署 GPU 任务测试结果



NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
gpu-job--1-2966h	1/1	Running	0	8s	192.168.16.15	node143
gpu-job--1-2dkqx	1/1	Running	0	8s	192.168.16.3	node143
gpu-job--1-4mzt1	1/1	Running	0	9s	192.168.16.49	node143
gpu-job--1-6b7c4	1/1	Running	0	9s	192.168.16.36	node143
gpu-job--1-r2shz	1/1	Running	0	8s	192.168.16.29	node143
gpu-job--1-t5597	1/1	Running	0	8s	192.168.16.46	node143
gpu-job--1-x9hwm	1/1	Running	0	8s	192.168.16.62	node143
gpu-job--1-xb7v7	1/1	Running	0	9s	192.168.16.31	node143
gpu-job--1-xs7m9	1/1	Running	0	8s	192.168.16.51	node143
gpu-job--1-z8bgp	1/1	Running	0	8s	192.168.16.58	node143

图 5-17 配置 GPU 共享时部署 GPU 任务测试结果

(2) 资源限制功能测试

Kubernetes 默认不支持对磁盘 IO 和网络带宽资源的限制，部署任务时无法为任务精细化分配以上两种资源。本文针对以上局限性实现了为 Pod 限制磁盘 IO 和网络带宽的功能。

图 5-19 是资源限制功能的测试结果。分别部署了一个申请了磁盘 IO 为 1M 和一个申请了网络带宽为 3M 的应用，两个应用运行期间的磁盘 IO 和网络带宽使用量分别为 1M 和 3M 左右，符合 Pod 对资源的申请需求。

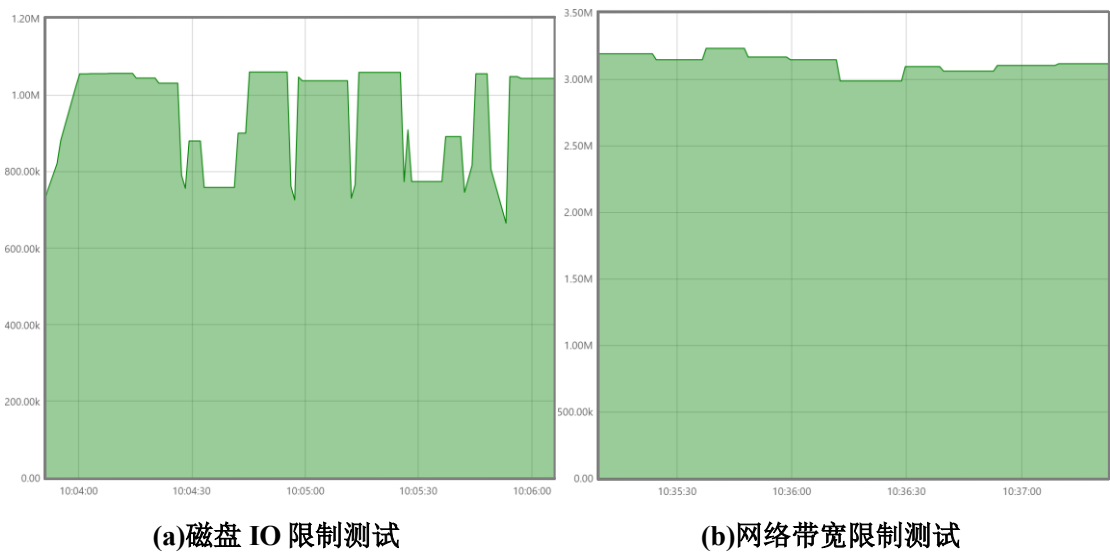


图 5-18 资源限制测试结果

(3) 调度功能测试

如图 5-19 所示，在 Karmada 控制平面提交一批任务后，任务能够根据调度策略被部署到相应的节点上正常执行。

[root@ ~]# kubectl --kubeconfig /etc/karmada/karmada-apiserver.config get job -n jobs -owide									
NAME	COMPLETIONS	DURATION	AGE	CONTAINERS	IMAGES	SELECTOR			
cpu-job1	0/1	2m6s	2m6s	cpu-job1	/cpu-job-gobrot:v0.1	controller-uid=64dbc4ca-d3c0-473f-8426-5327a9486733			
cpu-job2	0/1	2m6s	2m6s	cpu-job2	/cpu-job-gobrot:v0.1	controller-uid=88b91488-27ca-435f-bb26-9e5d2d410285			
cpu-job3	0/1	2m6s	2m6s	cpu-job3	/cpu-job-gobrot:v0.1	controller-uid=d0d1c509-8779-49a8-8159-2f82689275f3			
cpu-job4	0/1	2m6s	2m6s	cpu-job4	/cpu-job-gobrot:v0.1	controller-uid=24741b2a-alc3-4637-9758-3b69e470974c			
cpu-job5	0/1	2m6s	2m6s	cpu-job5	/cpu-job-gobrot:v0.1	controller-uid=cf357390-376e-4e1a-a0c7-a9462b1a18b4			
cpu-job6	0/1	2m6s	2m6s	cpu-job6	/cpu-job-gobrot:v0.1	controller-uid=1b5bf691-fce4-4f3e-bc51-71febfcd51a			
cpu-job7	0/1	2m6s	2m6s	cpu-job7	/cpu-job-blender:v0.1	controller-uid=ceal4bb2-c330-4750-941b-a4509f6bd4ac			
cpu-job8	0/1	2m6s	2m6s	cpu-job8	/cpu-job-blender:v0.1	controller-uid=dal9f9dd-7bf2-4082-8565-f2f5994e5d66			
cpu-job9	0/1	2m6s	2m6s	cpu-job9	/cpu-job-blender:v0.1	controller-uid=0bad723c-ad3f-44d8-b883-6f8434cb9d12			
disk-job1	0/1	2m5s	2m6s	disk-job1	/disk-job:v0.1	controller-uid=33b7c8fb-1d20-45b4-ab29-3ccfa927144a			
disk-job2	0/1	2m5s	2m6s	disk-job2	/disk-job:v0.1	controller-uid=930fcb47-6a62-49d7-aeb3-1305d99a1818			
disk-job3	0/1	2m5s	2m6s	disk-job3	/disk-job:v0.1	controller-uid=18b99101-a38f-4eb4-9686-bec401effc06			
disk-job4	0/1	2m5s	2m6s	disk-job4	/disk-job:v0.1	controller-uid=91234eeb-e78d-4b71-8d8e-5fac9a9d6e24			
disk-job5	0/1	2m5s	2m6s	disk-job5	/disk-job:v0.1	controller-uid=33551eal-0175-4e7a-9a88-a99757e93bd7			
disk-job6	0/1	2m5s	2m6s	disk-job6	/disk-job:v0.1	controller-uid=9f060848-f918-46c8-9598-209456205f12			
disk-job7	0/1	2m5s	2m6s	disk-job7	/disk-job:v0.1	controller-uid=1ac6abdd-c4f7-4426-ac54-4aaee982f6a6			
disk-job8	0/1	2m5s	2m6s	disk-job8	/disk-job:v0.1	controller-uid=a7f32961-6b80-40e2-8c10-2e44163e0d70			
gpu-job1	0/1	2m5s	2m6s	gpu-job1	/gpu-job-dekm:v0.1	controller-uid=ca570b01-6122-40de-a77f-27a596a022h8			
gpu-job2	0/1	2m5s	2m6s	gpu-job2	/gpu-job-openstl:v0.1	controller-uid=49ef86ef-0e3b-46b4-b2al-5e4a91bc6e55			
gpu-job3	0/1	2m4s	2m5s	gpu-job3	/gpu-job-openstl:v0.1	controller-uid=2f1c099e-70db-40ca-9f77-4c21fca740b			
gpu-job4	0/1	2m4s	2m5s	gpu-job4	/gpu-job-cifar:v0.1	controller-uid=41911e52-f6e2-4ef6-bdda-cf90ae7301b5			
gpu-job5	0/1	2m5s	2m5s	gpu-job5	/gpu-job-cifar:v0.1	controller-uid=352f932c-e083-4a6d-8cd6-125fe486a0e9			
network-job1	0/1	2m4s	2m5s	network-job1	/network-job-download-file:v0.2	controller-uid=64055b46-c067-4386-9b5b-18f49c7484bd			
network-job2	0/1	2m4s	2m5s	network-job2	/network-job-download-file:v0.2	controller-uid=8959a8c2-04fb-4e76-b73e-c276724bf2fc			
network-job3	0/1	2m4s	2m5s	network-job3	/network-job-download-file:v0.2	controller-uid=c0045a37-285c-456a-82f7-561433958611			
network-job4	0/1	2m3s	2m5s	network-job4	/network-job-download-file:v0.2	controller-uid=10ba3123-bbca-44f4-9b14-a483800c4724			
network-job5	0/1	2m3s	2m5s	network-job5	/network-job-download-file:v0.2	controller-uid=b8652615-82a3-44ca-9b3f-cb6cb4157627			
network-job6	0/1	2m4s	2m5s	network-job6	/network-job-download-file:v0.2	controller-uid=2e6e46de-6ccf-4b40-b51c-ba4a4c37cb81			
network-job7	0/1	2m3s	2m5s	network-job7	/network-job-download-file:v0.2	controller-uid=06be06b9-b598-4eb1-8ba7-474548e9e764			

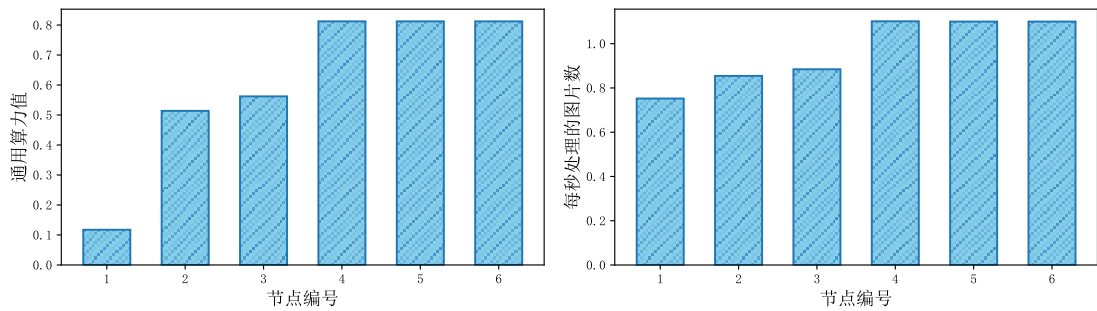
图 5-19 任务运行过程

5.3.2 算力度量有效性测试

本节使用相关测试工具来模拟各维度下的工作负载，通过工作负载的执行结果与算力度量模块度量的算力值对比，以证明本文提出的算力度量模型的有效性。使用 TensorFlow Benchmarks^[45]基准测试项目模拟 CPU 密集型和 GPU 密集型任务，使用 fio 模拟磁盘密集型任务，使用 iperf3 模拟网络密集型任务。

如图 5-20，对于通用算力，节点的处理能力和算力值相比有相同的趋势，随着算力节点编号增加，两者都呈正向增长的趋势，对于实验中的节点，算力

值能够有效区分节点在通用算力下的性能。

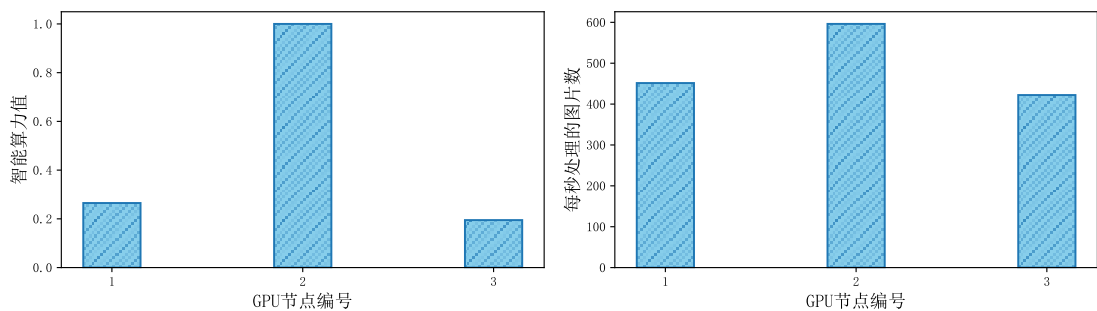


(a)通用算力度量结果对比

(b)通用算力处理能力对比

图 5-20 通用算力有效性实验结果

如图 5-21 所示, 智能算力和通用算力类似, 对于实验中的三个节点中, 算力值与节点性能能够匹配, 度量的算力值能够有效区分三个节点的智能算力方面的性能。

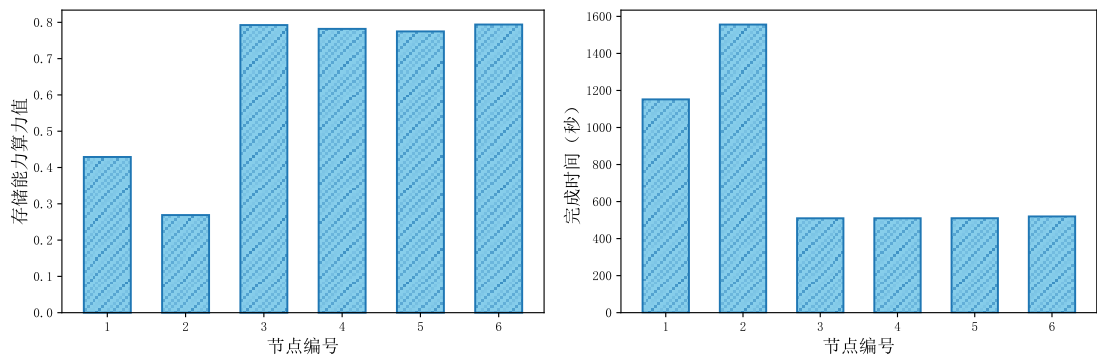


(a)智能算力度量结果对比

(b)智能算力处理能力对比

图 5-21 智能算力有效性实验结果

图 5-22 和图 5-23 分别是存储能力和网络能力维度下的测试结果, 两者的结果都表明在各算力维度下的算力值能够有效区分节点对应维度下的性能。



(a)存储能力度量结果对比

(b)存储能力处理能力对比

图 5-22 存储能力有效性实验结果

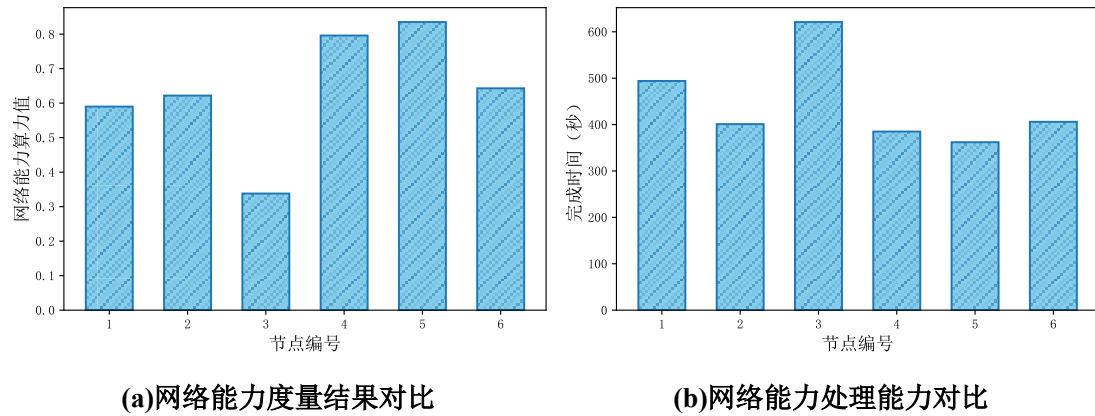


图 5-23 网络能力有效性实验结果

综上所述，在本文的实验环境中，本文提出的多维细粒度算力度量模型在各维度下的算力值能够与节点性能相匹配，证明了本文提出的算力度量模型的有效性。

5.3.3 多集群监控性能测试

本节通过测试跨集群监控产生的跨集群平均流量、每次抓取数据的网络流量、跨集群监控组件消耗的 CPU 和内存资源三方面的内容，并与 Prometheus 联邦跨集群监控方案相比，以验证本文设计的跨集群监控框架的性能优势。

(1) 跨集群网络流量测试

使用 tcpdump 工具测试跨集群监控的网络流量，测试脚本如图 5-24 所示，其中 TARGET_IP1 和 TARGET_IP2 是两个成员集群的入口 IP 地址，在控制集群中，在 Prometheus 服务所在的节点测试 Prometheus 联邦的网络流量，在本文设计的监控组件 MCM Controller 所在的节点测试 MCM 的网络流量。

```
#!/bin/bash
DURATION=150 # 捕获时长(秒)
TARGET_IP1="172.16.x.xxx" # 目标IP地址1
TARGET_IP2="172.16.x.xxx" # 目标IP地址2
CAPTURE_FILE="capture.pcap" # 捕获文件名
# 捕获两个IP地址的流量
timeout $DURATION tcpdump -i <dev> \((host $TARGET_IP1 or host $TARGET_IP2)\) -w $CAPTURE_FILE
# 计算总流量(字节)
TOTAL_BYTES=$(tcpdump -r $CAPTURE_FILE -nn -q | awk '/$TARGET_IP1|$TARGET_IP2/{sum += $NF} END {print sum}')
# 计算平均流量(字节/秒)
AVG_BYTES_PER_SEC=$(echo "$TOTAL_BYTES / $DURATION" | bc)
echo "Total Bytes: $TOTAL_BYTES"
echo "Average Bytes/s: $AVG_BYTES_PER_SEC"
```

图 5-24 跨集群监控流量测试脚本

为了清晰地展示实验结果，本文先对该部分实验图表中关键参数进行定义。

“PF”指 Prometheus Federate，是利用 Prometheus 实现的传统跨集群联邦监控方案；“MCM”表示本文设计的多集群监控框架；而“MCMND”则表示在

MCM框架中，未采取避免未变化的数据发送的实现方式。实验环境，成员集群由两个集群构成，每个成员集群中的节点均部署了监控代理，具体配置见表 5-3。实验中“监控节点数”的增加是按照表 5-3 中节点的顺序逐步增加的。例如，当监控节点数为 1 时，实验仅涉及 Node1；而当监控节点数增至 2 时，则包括 Node1 和 Node2。

表 5-3 多集群监控实验节点配置

集群编号	节点编号	监控代理	
		Node Exporter	NVIDIA GPU Exporter
Cluster1	Node1	√	
	Node2	√	√
	Node3	√	
	Node4	√	√
Cluster2	Node5	√	√
	Node6	√	

如图 5-25 所示，在能够获得相同监控数据的前提下，本文设计的监控方案显著降低了跨集群的网络流量速率。当监控的节点数为 1 时，MCM 产生的跨集群流量速率为 0.3Kb/s，MCMND 产生的跨集群流量速率为 0.35Kb/s，PF 产生的网络流量速率为 1.28Kb/s。与 Prometheus 联邦机制相比，监控单节点的情况下，MCMND 和 MCM 产生的网络流量速率分别降低了 0.93Kb/s 和 0.98Kb/s，分别降低了 72.7%和 76.6%的网络流量。本文设计的两种 MCM 监控方案的网络流量都低于 Prometheus 联邦。如果监控节点数设置为 2、4、6，当使用 MCM 监控方案时，与 Prometheus 联邦相比，分别降低 89.2%、92.5%和 93.7%，这种趋势在跨多个集群的时候的情况下最为明显。

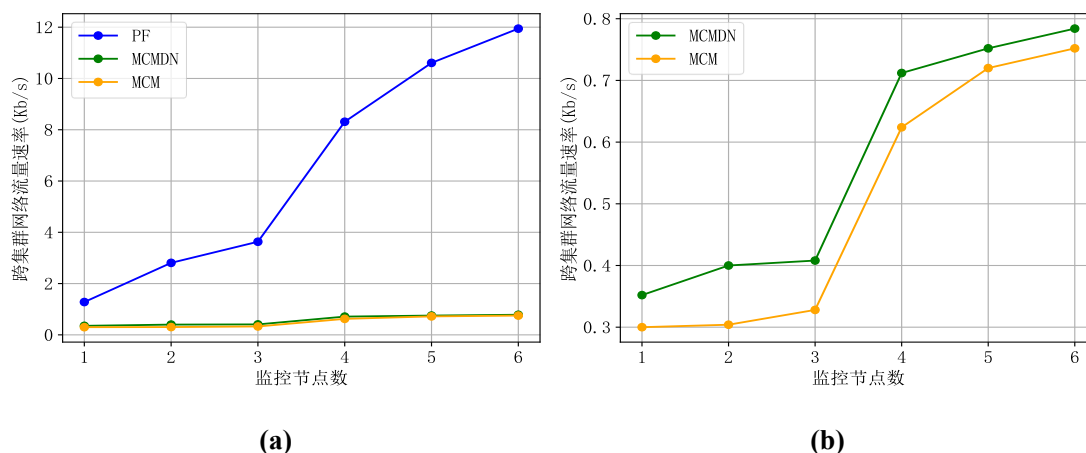


图 5-25 跨集群网络流量速率

图 5-25 表明, 无论实验中有多少个被监控的节点, 本文设计的两种监控方法都比 Prometheus 联邦效果更好。Prometheus 联邦的机制是由全局集群的 Prometheus 服务从各个集群中拉取监控数据到全局集群。本文的策略是把拉取的任务放在各个成员集群中, 经过 PromQL 聚合后主动发送给控制集群, 减少发送数据, 进而极大程度上减少跨集群的网络流量。此外, 如图 5-25(b)所示, MCMDN 的网络流量更低, 因为没有变化的监控数据不会进行发送。

MCM 根据指定的抓取间隔从各成员集群收集数据, 数据以周期性的时间间隔传输 (与 Prometheus 联邦相同)。因此, 本文还想知道在监控过程中单次抓取时使用了多少跨集群网络带宽。图 5-26 显示了每次抓取的跨集群网络流量大小的结果。Prometheus 联邦每次抓取的跨集群网络流量经历了从 19.25Kb (1 个节点)、54.52Kb (3 个节点)、179.14Kb (6 个节点) 的线性增长, 1 个和 6 个监控节点之间的差异为 159.89Kb, 高出 830.6%。这是因为 Prometheus 联邦从成员集群中拉取的数据会附带所有的原始标签, 方便在全局集群识别拉取的数据, 这些策略显著增加了跨集群的消耗。MCM 和 MCMND 两种方法的网络流量都随着被监控节点数量的增加而略有增长, 当监控节点从 1 增加到 6 时, MCM 和 MCMND 的每次抓取的跨集群网络流量分别为 5Kb/5.27Kb 和 11.45Kb/12.45Kb, 增长率为 129%和 136%, 都显著低于 Prometheus 联邦。

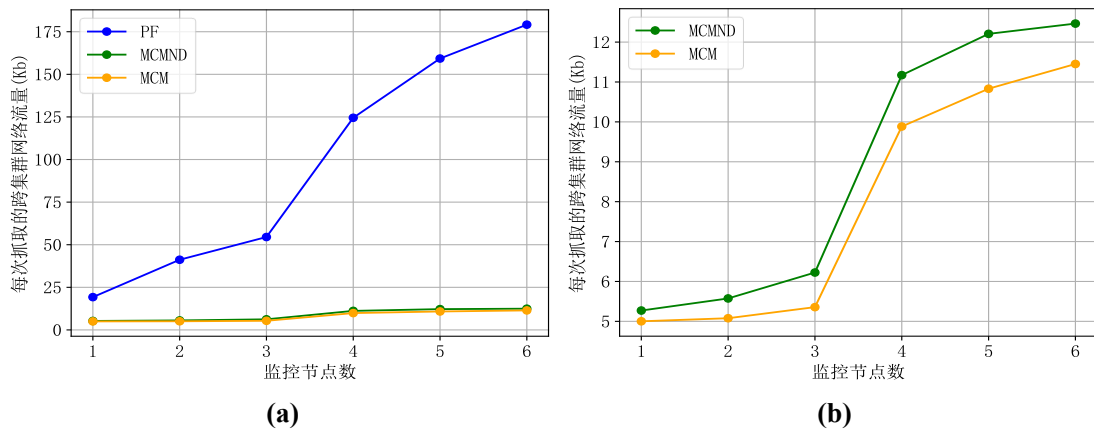


图 5-26 每次抓取的跨集群网络流量

(2) 资源消耗测试

为了更好地了解 MCM 的工作效率, 本文测量了 MCM 的资源使用情况, 以了解需要多少 CPU 和内存。MCM 组件的 CPU 使用情况如图 5-27(a)所示, MCM

组件的 CPU 使用量随着被监控节点的增加逐步增长, 随着被监控节点数量的增加, MCM Controller 的 CPU 使用量的增长率显著低于 MCMND Controller, 这是因为 MCM Member 中有避免未变化的数据重复发送的机制, Controller 收到 Member 发送的数据后需要进行解压, 且 MCM Controller 收到的数据量小于 MCMND Controller。当被监控节点的数量为 2 和 3 时, MCM Member 的 CPU 消耗量略大于 MCMND Member, 这是因为在 MCM Member 比 MCMND Member 多了判断监控数据是否变化的相关逻辑。如图 5-27(b)所示, MCM 和 MCMND 两种方法的内存使用量几乎相同, MCM Controller 的内存消耗量约为 12MiB, MCM Member 的内存消耗量约为 20MiB。

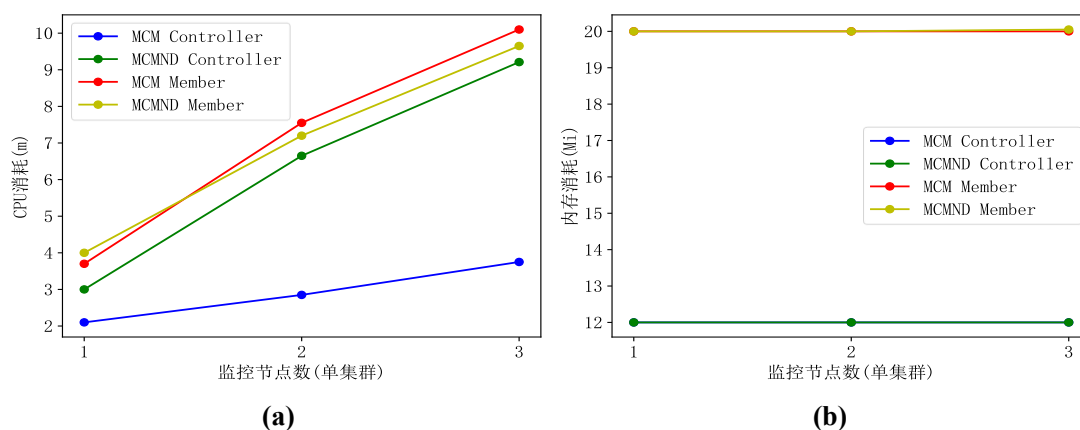


图 5-27 多集群监控资源消耗情况

总体来看, 在能获得本系统需要的跨集群监控数据的条件下, 本文设计的多集群监控框架和 Prometheus 联邦相比, 显著降低了跨集群网络流量, 减轻了跨集群网络的负载。同时, MCM 消耗较少的 CPU 和内存资源, 能够保证系统其他功能的正常执行。

5.3.4 算力调度效果测试

算力调度测试包括任务完成时间、全局负载平衡度两方面的测试。向系统中部署的任务包括 CPU 密集型、GPU 密集型、磁盘密集型和网络密集型任务。使用 Gobrot^[46]和 Blender^[47]项目生成 CPU 密集型任务; 本文复现了文献[48-50]中的源码, 并设置了不同超参数 (如 Epochs、Batch Size 等), 作为 GPU 密集型任务; 使用 fio^[51]和 GUN Wget^[52]分别模拟磁盘密集型任务和网络密集型任务。实验中两种多目标优化算法涉及到的参数见表 5-4。

表 5-4 算法参数信息

参数	NSGA-II	C-TAEA
种群大小	300	-
迭代次数	300	300
交叉概率	0.7	0.7
变异概率	0.2	0.2
参考方向矩阵	-	approach="das-dennis", n_dim=2, n_partitions=36

(1) 任务完成时间测试

本文在 Karmada 控制平面集群提交了 29 个任务，任务类型涵盖 CPU 密集型任务、GPU 密集型任务、磁盘密集型任务和网络密集型任务。如图 5-28 所示，对于提交的 29 个任务，采用 Karmada 和 Kubernetes 结合的默认调度策略，任务的平均完成时间为 620.76 秒；采用 NSGA-II 算法作为调度策略时，任务的平均完成时间分别为 559.90 秒，与默认调度策略相比，减少了 9.80%；本文使用 C-TAEA 算法作为最终的调度策略，在该策略下，任务的平均完成时间为 525.41 秒，与默认和 NSGA-II 调度策略相比，分别减少了 15.36%和 6.16%。

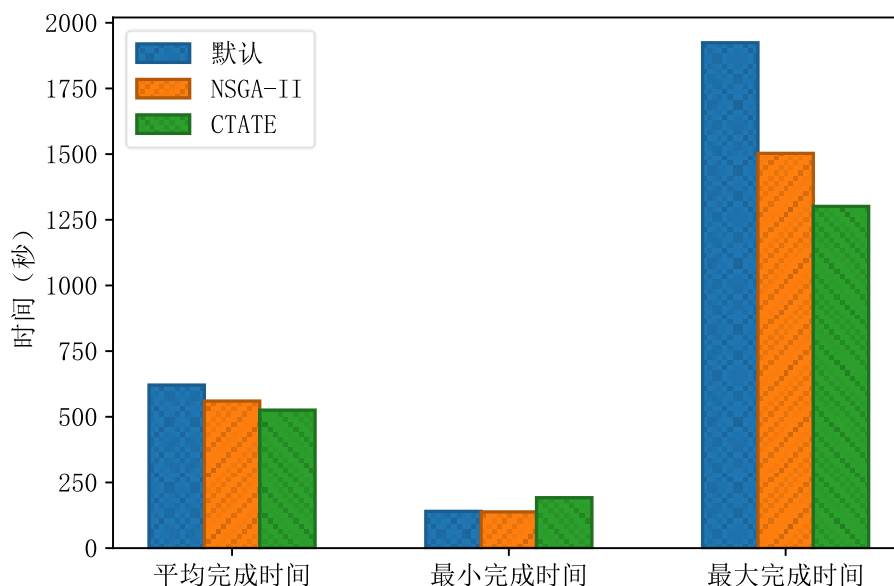


图 5-28 不同调度策略下任务平均、最小、最大完成时间对比

三种调度策略下，任务平均完成时间分别为 620.76 秒、559.90 秒和 525.41 秒，与默认调度策略相比，NSGA-II 和 C-TAEA 调度策略下分别减少了 9.80%和 15.36%；最小完成时间分别为 140 秒、138 秒和 192 秒，默认调度策略与 NSGA-II 调度策略基本一致；最大完成时间分别为 1924 秒、1503 秒和 1301 秒，如果

以批次任务的最大完成时间作为该批次任务的 **Makespan**，和默认调度策略相比，在 NSGA-II 和 C-TAEA 调度策略下分别减少了 21.88%和 32.38%，C-TAEA 和 NSGA-II 调度策略相比，减少了 13.44%。

将 29 个任务分为 G_1 - G_6 六组，分别表示默认调度策略下任务完成时间大于 NSGA-II 调度策略、小于 NSGA-II 调度策略、大于 C-TAEA 调度策略、小于 C-TAEA 调度策略，以及 C-TAEA 调度策略相比于 NSGA-II 调度策略，任务完成时间小于和大于 NSGA-II。如表 5-5 所示，满足 G_1 - G_6 的任务数量分别为 19、10、19、10、15、14 个。从数量上看， G_1 和 G_3 组占据总任务数量的 65.52%，说明本文的调度模型在多目标优化算法的驱动下能够有效增加任务与节点性能之间的兼容性，本文设计的调度策略相比于默认调度策略占据了显著的优势。

表 5-5 任务完成时间对比分组

组别	任务编号集合	说明
G_1	2, 4, 6, 7, 10, 11, 12, 13, 14, 15, 16, 18, 19, 21, 22, 23, 25, 28, 29	默认调度策略相比于 NSGA-II 调度策略
G_2	1, 3, 5, 8, 9, 17, 20, 24, 26, 27	
G_3	7, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 25, 26, 27, 29	默认调度策略相比于 C-TAEA 调度策略
G_4	1, 2, 3, 4, 5, 6, 8, 9, 19, 28	
G_5	8, 9, 12, 15, 16, 17, 18, 20, 21, 22, 23, 24, 26, 27, 29	NSGA-II 调度策略相比于 C-TAEA 调度策略
G_6	1, 2, 3, 4, 5, 6, 7, 10, 11, 13, 14, 19, 25, 28	

如图 5-29 所示，结合表 5-5 中的任务组别可知，超过一半的任务在两种多目标优化算法驱动的调度策略下的运行时间比默认调度策略少。对于 G_1 中的任务，任务完成时间最小减少了 0.50%，最大减少了 54.25%，平均减少了 16.87%；对于 G_2 中的任务，任务完成时间最小增加了 0.30%，最大增加了 126.02%，平均增加了 17.50%；对于 G_3 中的任务，任务完成时间最小减少了 1.37%，最大减少了 49.67%，平均减少了 19.72%；对于 G_4 中的任务，任务完成时间最小增加了 5.65%，最大增加了 190.91%，平均增加了 45.83%。由此来看，NSGA-II 和 C-TAEA 调度策略相比于默认调度策略，能够有效减少批次任务的总体执行时间，而 G_2 和 G_4 中出现任务完成时间增加高达 126.02%和 190.91%，这是因为本文的另一个调度目标是最大化全局负载平衡，调度策略为了满足该目标，会将某些计算量较大的任务调度到性能较差的节点上，以平衡全局的负载。

图 5-29 中， G_5 中的任务完成时间在 C-TAEA 调度策略下小于 NSGA-II， G_6

中的任务完成时间在 C-TAEA 调度策略下大于 NSGA-II。可以看出, 编号为 8、9、18、20、21、22 等计算量较大的任务在 C-TAEA 算法驱动下的调度策略的执行效率相比于 NSGA-II 更具优势, C-TATE 算法相比于 NSGA-II 算法, 能够进一步提高任务与节点性能的兼容性, 加快批次任务的整体执行速度。

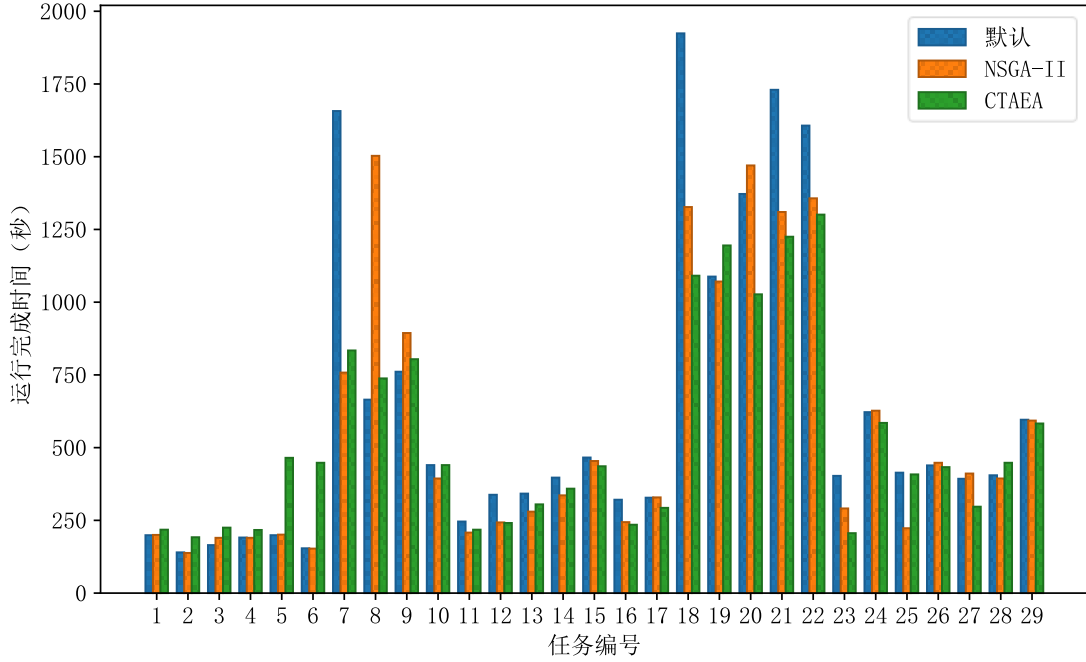


图 5-29 不同调度策略下任务完成时间对比

(2) 负载均衡测试

分别采用默认调度策略以及两种多目标算法驱动的调度策略在 Karmada 控制平面提交 29 个任务, 从提交任务之前, 到任务执行, 再到所有任务执行完毕, 使用 Prometheus 采集了整个过程中各节点的 CPU 使用率、GPU 内存使用率、内存使用率、磁盘 IO 使用率、网络带宽使用率五个指标。使用各节点多维资源使用率的标准差来衡量全局负载均衡度, 计算过程如下。

步骤一: 计算各节点在每个资源维度下的综合资源利用率 U_j , 计算方式如公式(5-1)所示:

$$U_j = \begin{cases} \frac{1}{4}(u_j^{cpu} + u_j^{mem} + u_j^{disk} + u_j^{net}), & type_j = CPU \\ \frac{1}{5}(u_j^{cpu} + u_j^{gpu} + u_j^{mem} + u_j^{disk} + u_j^{net}), & type_j = GPU \end{cases} \quad (5-1)$$

其中, $u_j^{cpu}, u_j^{gpu}, u_j^{mem}, u_j^{disk}, u_j^{net}$ 分别表示节点 j 在 CPU、GPU、内存、磁盘 IO、

网络带宽维度下的使用率， $type_j$ 表示节点 j 是 CPU 节点还是 GPU 节点。

步骤二：计算各节点的平均资源使用率，参考公式(4-24)。

步骤三：使用标准差计算全局负载均衡标准差，参考公式(4-25)。

步骤四：计算全局负载平衡度 B ，计算方式如公式(5-2)所示：

$$B = \frac{1}{S+1} \quad (5-2)$$

其中， S 是由步骤三计算出的全局负载平衡标准差。

图 5-30 是分别在默认调度策略、NSGA-II 调度策略、C-TAEA 调度策略下任务执行过程中的全局负载均衡度变化情况，提交任务之前，全局负载平衡度处在一个较高的值，任务开始执行后，三种调度策略下的全局负载平衡度都迅速降低，直到任务执行完毕后恢复到初始状态。在任务执行期间，NSGA-II 和 C-TAEA 调度策略下的全局负载平衡度始终大于默认调度策略。

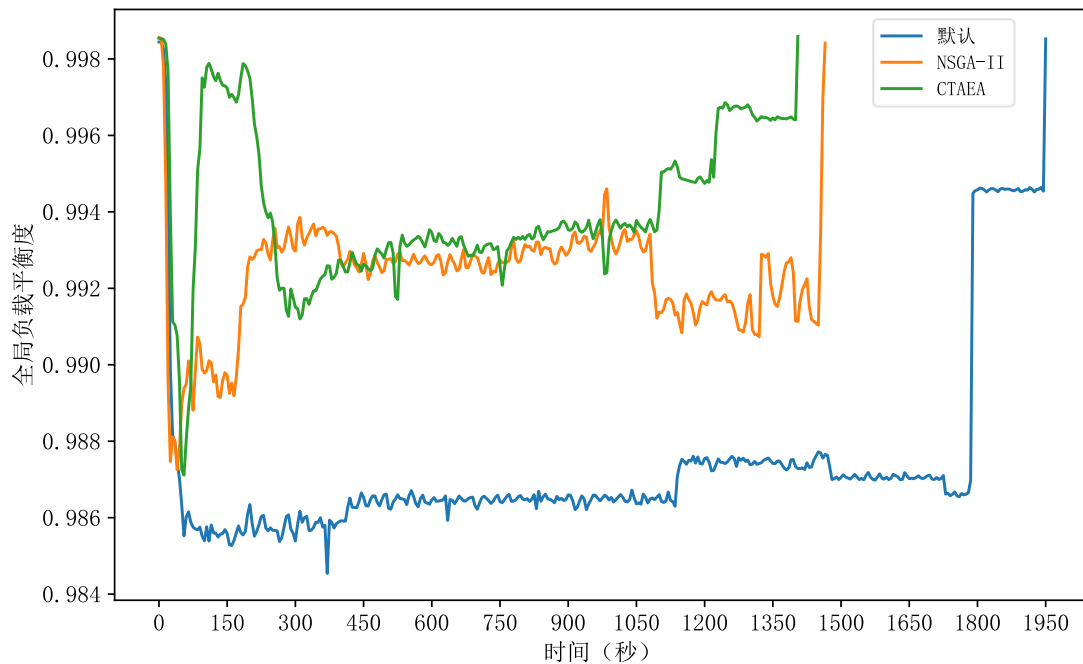


图 5-30 不同调度策略下全局负载平衡度变化情况

如表 5-6 所示，未提交任务前，全局负载平衡度的初始值保持在 0.9986 左右；分别在不同调度策略下提交任务，结合图 5-30 中全局负载平衡度变化情况可知，在任务执行期间，三种调度策略下全局负载平衡度分别在第 365 秒、第 35 秒、第 50 秒左右出现最小值为 0.9845、0.9873 和 0.9871。对于任务执行期间的平均全局负载平衡度，两种多目标优化算法驱动下的调度策略均高于默认调

度策略，在 NSGA-II 和 C-TAEA 下，分别提高了 0.27%和 0.38%，C-TAEA 相比与 NSGA-II，提高了 0.11%。由此表明，本文设计的调度策略能够在减少任务完成时间的条件下，保证较高的全局资源负载平衡，并且在相同调度模型下，采用 C-TAEA 算法驱动的调度策略的效果优于 NSGA-II。

表 5-6 不同调度策略下全局负载平衡度对比

调度策略	默认	NSGA-II	C-TAEA
提交任务前全局负载平衡度	0.9986	0.9986	0.9986
任务执行期间最小全局负载平衡度	0.9845	0.9873	0.9871
任务执行期间平均全局负载平衡度	0.9904	0.9931	0.9942

总体而言，本文设计的算力调度策略在进行调度算法求解时能够找到有效的解集，验证了本文设计的调度模型的正确性和调度算法的有效性，通过分析任务的执行时间与任务执行期间全局的负载平衡度，验证了本文设计的调度策略能够根据任务特性与节点算力相匹配，有效减少了批量提交的任务的执行时间，并能够在一定程度上保证全局资源较高的负载平衡。

5.4 本章小结

本章对系统测试拓扑结构、系统运行时环境进行了详细的说明，介绍了系统核心功能的实现细节，并对系统关键功能和设计的相关策略进行了实验测试，最终的测试结果表明本文设计的框架、策略符合预期结果，相比于传统的解决方案有了进一步的提升。

第六章 总结与展望

6.1 总结

随着智能驾驶、元宇宙、AIGC 等新兴产业的蓬勃发展，复杂科学计算、模拟等超算任务对算力资源的需求量急剧攀升。与此同时，产业界为了支撑自身超算任务计算和业务发展需要，基于 Kubernetes 和 Karmada 技术搭建了算力调度系统。由 Kubernetes 和 Karmada 搭建的算力调度系统提供了基本的调度能力，但由于默认策略存在较大的局限性，任务与节点性能无法有效匹配，导致任务执行效率低，且无法有效保证资源的负载平衡。在此背景下，本文基于 Kubernetes 和 Karmada 实现了多集群异构算力调度系统，并对其进行了优化，以解决以上关键问题。

本文从节点硬件性能参数出发，分析影响节点性能的重要参数，构建多维度的算力度量指标体系，并采用多属性决策方法 TOPSIS 对各维度下的指标进行综合评价，以此作为节点在该维度下的算力值，以识别节点间的性能差异，为后续算力调度提供决策支撑。

在 Kubernetes 环境下，默认仅支持 CPU 和内存资源的访问和分配，而在实际应用中，GPU、磁盘 IO、网络带宽等资源也是保证任务稳定运行的因素。对于 GPU 资源，本文引入 NVIDIA 官方提供的设备插件 NVIDIA Device Plugin，并配置 GPU 共享，实现了多个 Pod 能够同时访问同一 GPU 资源；对于磁盘 IO，通过 postStart 钩子将限制命令注入容器所在的 cgroup，实现对容器磁盘 IO 的限制；对于网络带宽，设计了网络带宽限制组件，借助 wondershaper 网络带宽限制工具，在容器创建时限制其网络接口的带宽，实现对容器网络带宽的限制。

在资源监控中，本文使用 Prometheus 监控套件实现了单集群的监控，在此基础上，设计了高效的多集群监控框架 MCM，通过压缩和减少上报数据量的方式，极大降低了多集群监控产生的跨集群网络流量。

在算力调度层面，本文结合任务特性、算力值、任务资源需求信息以及全局节点的资源使用状态，面向多类型任务构建了调度模型。调度模型中包含了最小化批次任务执行时间和最大化全局多维资源负载平衡两个优化目标，使用 C-TAEA 多目标优化算法求解调度问题，调度方案有效减少了批次任务的执行

时间和资源的负载平衡。

通过实验表明,本文提出的算力度量方案在异构环境下能有效对节点间各维度下的性能进行区分。同时,在多集群环境下,本文设计的监控策略相对于 Prometheus 联邦多集群监控策略能够极大减少多集群监控产生的跨集群流量,保证了系统其他功能的正常执行。除此以外,针对多类型任务,本文设计的算力调度策略相比于 Karmada 和 Kubernetes 默认调度策略能够有效减少任务执行时间,并能够保证全局节点之间一定的资源负载平衡,提高了系统的稳定性。

6.2 展望

本文设计的算力调度策略在一定程度上减少了任务执行时间,提高了节点间的资源负载平衡,但在某些方面还存在一些不足,未来主要改进和完善的工作包括以下几个方面:

(1) 本文从节点硬件的细粒度参数出发,度量了节点的实际性能。但在实际过程中,节点上任务之间的资源争用会导致任务的执行效率降低,因此,在未来的工作中,可以考虑引入资源争用的影响来进一步量化节点的实际性能。

(2) 在算力调度层面,本文目前只针对没有依赖关系的任务进行调度,在分布式机器学习、大数据计算任务等分布式场景下,任务与任务之间是一个 DAG 结构,互相存在依赖关系,这会带来跨节点甚至跨集群之间的通信开销。因此,后续可以考虑在本文工作的基础上针对分布式任务进行调度。

(3) 由于实验条件有限,本文实现的算力调度系统在智能算力资源方面仅支持 GPU 资源,未将 FPGA、AISC、RDMA 等其他新型智能资源纳入其中,未来在实验条件满足的情况下,可以加入其他新型智能硬件资源,组成异构智能算力资源池,进一步支持多种智能算力的调度。

(4) 超算任务从提交到执行分为编译、分解、调度、运行四个过程。本文的工作是奠定在任务分解后实现的,超算任务的编译和分解是一个复杂的过程,未来可以在本文的基础上实现编译和分解的工作,进一步支持超算任务。

(5) 本文目前只针对提高任务处理效率和系统稳定性做了相应的调度优化工作,暂未考虑任务特定需求(如任务优先级),后续工作可以将任务的特定需求作为调度依据纳入其中。

致谢

在硕士研究生生涯的最后阶段，怀着感激之情，回顾这段充满挑战与成长的岁月，我深深地意识到自己能够顺利完成学业，离不开许多人的支持和帮助。

首先，我要衷心感谢我的导师。作为我的指导教师，您在学术研究和项目实践上给予了我无尽的启迪和帮助。无论是在课题的选择、研究思路的确定，还是在论文的写作过程中，您都倾注了大量的心血。您的严谨治学态度和精益求精的精神不仅在学术上指引我前行，更在做人做事上为我树立了榜样。感谢您在我迷茫时的悉心指导和在我遇到困难时的耐心鼓励，让我在学术道路上不断进步。

其次，我要感谢实验室的师兄、师弟和同级的兄弟姐妹们。特别感谢课题组的师兄师姐们，在研一研二期间，无私地分享了宝贵的经验和知识，为我快速适应实验室的工作环境提供了巨大的帮助。感谢课题组的兄弟姐妹们，我们一起经历了无数个日夜的学习和实验，相互交流、分享心得，使我的研究工作更加顺利和高效。

同时，我要特别感谢我的家人。感谢我的父母，您们给予了我无尽的爱和支持。无论在任何时候，您们都是我最坚强的后盾。在我求学的这段时间里，您们默默地承受着思念和牵挂，却从未表现出一丝的抱怨，总是鼓励我勇往直前。感谢我的姐姐和弟弟，你们在我需要时，总是给予无条件的支持和帮助，让我能够安心地投入到学业中。

最后，我要感谢我的女朋友。感谢你在我身边陪伴我，在我疲惫时给予我鼓励和安慰。你无私的支持和理解使我在学术研究的道路上更加坚定。你的爱和包容让我在压力面前更加从容。

总之，感谢所有在我求学道路上给予过我帮助和支持的人们。正是有了你们的支持和帮助，我才能够顺利完成学业。未来的日子里，我将铭记这段难忘的岁月，不断努力，为实现自己的梦想而奋斗。

参考文献

- [1] Feng S, Yan X, Sun H, Feng Y, Liu H X. Intelligent driving intelligence test for autonomous vehicles with naturalistic and adversarial environment[J]. Nature communications, 2021, 12(1): 748.
- [2] Wang Y, Su X, Zhang N, Xing R, Liu D, Luan T H, Shen X. A survey on metaverse: Fundamentals, security, and privacy[J]. IEEE Communications Surveys & Tutorials, 2022, 25(1): 319-352.
- [3] Wu T, He S, Liu J, Sun S, Liu K, Han Q L, Tang Y. A brief overview of ChatGPT: The history, status quo and potential future development[J]. IEEE/CAA Journal of Automatica Sinica, 2023, 10(5): 1122-1136.
- [4] OpenAI. Scaling Kubernetes to 7,500 nodes [EB/OL]. [2024-04-14]. <https://openai.com/research/scaling-kubernetes-to-7500-nodes>.
- [5] Li L, Lu Y. Status, Opportunities, and Barriers in Implementing Industry 4.0 in the US[M]//Industry 4.0 in SMEs Across the Globe. CRC Press, 2021: 209-227.
- [6] 张振.布局全国算力网络国家枢纽节点 构建国家算力网络体系——国家发展改革委高技术司主要负责同志就《全国一体化大数据中心协同创新体系算力枢纽实施方案》答记者问[J].中国经贸导刊,2021(12):24-26.
- [7] European Commission. 2030 digital compass: the European way for the digital decade[J]. COM (2021) 118 final, 2021.
- [8] 翟万江.加快构建全国一体化算力网 赋能数字经济高质量发展——五部门联合印发《关于深入实施“东数西算”工程加快构建全国一体化算力网的实施意见》[J].中国科技产业,2024(01):32-33.DOI:10.16277/j.cnki.cn11-2502/n.2024.01.009.
- [9] 中国移动研究院, 浪潮电子信息产业股份有限公司, 新华三技术有限公司. 分布式异构智能算力管理和调度技术深度研究报告[EB/OL]. (2023-12).
- [10] 华为技术有限公司. (2023-08-30). 华为云 UCS 产品介绍[EB/OL]. (算力统一调度). [2024-04-10]. <https://support.huaweicloud.com/productdesc-ucs/UCS%20产品介绍.pdf>
- [11] 中泰证券. (2024). 中泰证券 2023 环境、社会及公司治理(ESG)报告[EB/OL]. (《支持异构算力纳管的大数据云原生平台的研究与实践》). [2024-04-10]. <https://php.cnstock.com/texts/2024/20240329/F0C806406DAF71CD17B97F2970BEB78F.pdf>.
- [12] 智汇云. (2024-01-24). K8S 集群闲置算力分享[EB/OL]. (闲置算力分享). [2024-04-10]. <https://zhuanlan.zhihu.com/p/679585453>.
- [13] 覃剑,赵蓓蕾,巫细波.中国数字经济一线城市算力建设研究[J].城市观察,2022,(04):125-136+163-164.
- [14] 苏娟,董彦君,赵晶,董敏,杜松怀.“东数西算”背景下多数据中心联合消纳可再生能源途径研究综述[J].高电压技术,2024,50(01):55-64.DOI:10.13336/j.1003-6520.hve.20231390.
- [15] 孙毅,王会梅,鲜明,向航.Kubeflow 异构算力调度策略研究[J].计算机工程,2024,50(02):25-32.DOI:10.19678/j.issn.1000-3428.0067396.
- [16] 李俊俊,董建刚,李坤.基于 Kubernetes 的集群节能策略研究[J/OL].计算机工程,1-13[2024-04-09].<http://kns.cnki.net/kcms/detail/31.1289.TP.20240119.1457.011.html>.
- [17] Cheng W, Xu Y, Xu Q, Zhang H, Li H, Shao X. CSFRL: A Reinforcement Learning Tech-

- nology Enabled Computing Power Scheduling Framework Based on Kubernetes[C]//2023 IEEE 34th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC). IEEE, 2023: 1-6.
- [18] 柴若楠,郜帅,兰江雨,刘宁春.算力网络中高效算力资源度量方法[J].计算机研究与发展,2023,60(04):763-771.
- [19] Brodtkorb A R, Hagen T R, Sætra M L. Graphics processing unit (GPU) programming strategies and trends in GPU computing[J]. Journal of Parallel and Distributed Computing, 2013, 73(1): 4-13.
- [20] Jouppi N P, Young C, Patil N, Patterson D, Agrawal G. In-datacenter performance analysis of a tensor processing unit[C]//Proceedings of the 44th annual international symposium on computer architecture. 2017: 1-12.
- [21] Gandola M, Grassi M, Mele F, Dedolli I, Malcovati P, Bertuccio G. The sparse readout RIGEL Application Specific Integrated Circuit for Pixel Silicon Drift Detectors in soft X-ray imaging space applications[J]. Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment, 2022, 1040: 167249.
- [22] Ruiz-Rosero J, Ramirez-Gonzalez G, Khanna R. Field programmable gate array applications—A scientometric review[J]. Computation, 2019, 7(4): 63.
- [23] Emma P G. Understanding some simple processor-performance limits[J]. IBM journal of Research and Development, 1997, 41(3): 215-232.
- [24] 祝淑琼,徐青青,李小涛,陈维.算力度量与任务调度: 物联网端侧设备策略研究[J/OL].电信科学:1-24[2024-04-09].<http://kns.cnki.net/kcms/detail/11.2103.tn.20240319.1715.010.html>.
- [25] 乔楚.算力度量与算网资源调度思路分析[J].通信技术,2022,55(09):1165-1170.
- [26] 夏天豪,夏长清,潘昊,许驰,金曦.基于强化学习的算力资源度量方法[J].燕山大学学报,2023,47(03):246-254.
- [27] Li J, Lv H, Lei B, Xie Y. A computing power resource modeling approach for computing power network[C]//2022 International Conference on Computer Communications and Networks (ICCCN). IEEE, 2022: 1-2.
- [28] Liu P. Convergence of high performance computing, big data, and machine learning applications on containerized infrastructures[J]. 2023.
- [29] Senjab K, Abbas S, Ahmed N, Khan A R. A survey of Kubernetes scheduling algorithms[J]. Journal of Cloud Computing, 2023, 12(1): 87.
- [30] Menouer T. KCSS: Kubernetes container scheduling strategy[J]. The Journal of Supercomputing, 2021, 77(5): 4267-4293.
- [31] Han R, Wen S, Liu C H, Yuan Y, Wang G, Chen L Y. EdgeTuner: Fast scheduling algorithm tuning for dynamic edge-cloud workloads and resources[C]//IEEE INFOCOM 2022-IEEE Conference on Computer Communications. IEEE, 2022: 880-889.
- [32] Zhong Z, Buyya R. A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources[J]. ACM Transactions on Internet Technology (TOIT), 2020, 20(2): 1-24.
- [33] 刘志彬,黄秋兰,胡庆宝,程耀东,胡誉.Kubernetes 异构资源细粒度调度策略的设计与实现[J].计算机工程,2023,49(02):31-36+45.DOI:10.19678/j.issn.1000-3428.0063709.
- [34] 李华东,张学亮,王晓磊,刘惠,王鹏程,杜军朝.Kubernetes 集群中多节点合作博弈负载均衡

- 策略[J].西安电子科技大学学报,2021,48(06):16-22+122.DOI:10.19665/j.issn1001-2400.2021.06.003.
- [35] Fu Y, Zhang S, Terrero J, Mao Y, Liu G, Li S, Tao D. Progress-based container scheduling for short-lived applications in a kubernetes cluster[C]//2019 IEEE International Conference on Big Data (Big Data). IEEE, 2019: 278-287.
- [36] Liu Z, Chen C, Li J, Cheng Y, Kou Y, Zhang D. KubFBS: A fine - grained and balance - aware scheduling system for deep learning tasks based on kubernetes[J]. Concurrency and Computation: Practice and Experience, 2022, 34(11): e6836.
- [37] 王壮,王平辉,王彬丞,武文博,王斌,丛鹏宇.深度学习容器云平台下的 GPU 共享调度系统[J].计算机科学,2023,50(06):86-91.
- [38] Osmani L, Kauppinen T, Komu M, Tarkoma S. Multi-cloud connectivity for kubernetes in 5g networks[J]. IEEE Communications Magazine, 2021, 59(10): 42-47.
- [39] Pivotto J, Brazil B. Prometheus: Up & Running[M]. " O'Reilly Media, Inc.", 2023.
- [40] Yang Z, Ye Z, Fu T, Luo J, Wei X, Luo Y, Wang X, Wang Z, Zhang T. Tear Up the Bubble Boom: Lessons Learned From a Deep Learning Research and Development Cluster[C]//2022 IEEE 40th International Conference on Computer Design (ICCD). IEEE, 2022: 672-680.
- [41] NVIDIA. k8s-device-plugin[EB/OL]. [2024-04-10]. <https://github.com/NVIDIA/k8s-device-plugin>.
- [42] Intel. Intel® Xeon® E-2468 Processor (24M Cache, 2.60 GHz) Specifications [EB/OL]. [2024-04-10]. <https://www.intel.cn/content/www/cn/zh/products/sku/236184/intel-xeon-e2468-processor-24m-cache-2-60-ghz/specifications.html>.
- [43] Hubert B, Geul J, Séhler S. WonderShaper: Command-line utility for limiting an adapter's bandwidth[J]. URL: <https://github.com/magnific0/wondershaper>, 2021.
- [44] Li K, Chen R, Fu G, Yao X. Two-archive evolutionary algorithm for constrained multiobjective optimization[J]. IEEE Transactions on Evolutionary Computation, 2018, 23(2): 303-315.
- [45] tensorflow. TensorFlow benchmarks[EB/OL]. [2024-04-10]. <https://github.com/tensorflow/benchmarks>.
- [46] esimov. Gobrot[EB/OL]. [2024-04-10]. <https://github.com/esimov/gobrot>.
- [47] Villar O. Learning Blender[M]. Addison-Wesley Professional, 2021.
- [48] Guo W, Lin K, Ye W. Deep embedded k-means clustering[C]//2021 International Conference on Data Mining Workshops (ICDMW). IEEE, 2021: 686-694.
- [49] Tan C, Li S, Gao X, Guan W, Wang X, Liu Z, Wu L, Li S Z. Openstl: A comprehensive benchmark of spatio-temporal predictive learning[J]. Advances in Neural Information Processing Systems, 2024, 36.
- [50] Li H, Liu H, Ji X, Li G, Shi L. Cifar10-dvs: an event-stream dataset for object classification[J]. Frontiers in neuroscience, 2017, 11: 244131.
- [51] axboe. Fio[EB/OL]. [2024-04-14]. <https://github.com/axboe/fio>.
- [52] The Free Software Foundation (FSF). GUN Wget[EB/OL]. [2024-04-14]. <https://www.gnu.org/software/wget>.

附录

A 作者在硕士研究生期间参加的科研项目及成果

(一)本人在研究生期间参加的科研项目如下：

1. 参与一项国家自然科学基金

(二)本人在研究生期间竞赛获奖如下：

1. 2021 年 11 月，中国高校大数据挑战赛三等奖
2. 2022 年 6 月，全国大学生计算机技能应用大赛一等奖
3. 2023 年 4 月，蓝桥杯全国软件和信息技术专业人才大赛区域赛三等奖
4. 2023 年 12 月，全国高校计算机能力挑战赛二等奖

B 图版

图 1-1 OpenAI 单集群 Kubernetes 历年节点数	1
图 2-1 Kubernetes 架构	8
图 2-2 Kubernetes 调度框架扩展点	10
图 2-3 Karmada 架构	11
图 2-4 Prometheus 架构	13
图 2-5 Docker 架构及工作原理	16
图 2-6 Nvidia device plugin 工作原理	17
图 3-1 Karmada 与 Kubernetes 默认调度策略	20
图 3-2 Pod 实际资源使用情况	20
图 3-3 多集群异构算力调度系统应用场景	21
图 4-1 系统总体架构图	25
图 4-2 系统功能模块图	26
图 4-3 算力度量流程	28
图 4-4 算力度量指标体系	31
图 4-5 单集群资源监控架构	35
图 4-6 多集群资源监控架构	36
图 4-7 多集群资源监控时序图	37
图 4-8 GPU 资源管理架构图	40
图 4-9 Pod 带宽限制原理	41
图 4-10 磁盘 IO 限制命令	42
图 4-11 算力调度模型	43
图 4-12 个体编码方式	47
图 4-13 调度算法求解流程图	49
图 5-1 系统部署拓扑图	51
图 5-2 Docker 和 Kubernetes 安装流程	53
图 5-3 NVIDIA Device Plugin 和 Prometheus 部署流程	54
图 5-4 Karmada 安装流程	54

图 5-5 多集群管理配置流程	54
图 5-6 Prometheus 监控指标界面	55
图 5-7 GPU Exporter 可视化面板	55
图 5-8 Node Exporter 可视化面板.....	56
图 5-9 MCM Member 关键代码实现	56
图 5-10 MCM Controller 关键代码实现	57
图 5-11 Pushgateway 中缓存的跨集群监控数据	57
图 5-12 NVIDIA 设备插件配置	57
图 5-13 网络带宽控制器关键代码实现	58
图 5-14 网络带宽节点代理关键代码实现	58
图 5-15 全局调度器关键代码实现	59
图 5-16 未配置 GPU 共享时部署 GPU 任务测试结果.....	60
图 5-17 配置 GPU 共享时部署 GPU 任务测试结果.....	60
图 5-18 资源限制测试结果	61
图 5-19 任务运行过程	61
图 5-20 通用算力有效性实验结果	62
图 5-21 智能算力有效性实验结果	62
图 5-22 存储能力有效性实验结果	62
图 5-23 网络能力有效性实验结果	63
图 5-24 跨集群监控流量测试脚本	63
图 5-25 跨集群网络流量速率	64
图 5-26 每次抓取的跨集群网络流量	65
图 5-27 多集群监控资源消耗情况	66
图 5-28 不同调度策略下任务平均、最小、最大完成时间对比	67
图 5-29 不同调度策略下任务完成时间对比	69
图 5-30 不同调度策略下全局负载平衡度变化情况	70

C 表版

表 2-1 影响 Karmada 调度决策的因素	13
表 4-1 随机一致性指数	33
表 4-2 多集群监控指标信息	38
表 4-3 限制磁盘 IO 所需的环境变量	42
表 5-1 系统硬件环境	52
表 5-2 系统软件环境	52
表 5-3 多集群监控实验节点配置	64
表 5-4 算法参数信息	67
表 5-5 任务完成时间对比分组	68
表 5-6 不同调度策略下全局负载平衡度对比	71

附：贵州大学学位论文原创性声明和使用授权声明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究在做出重要贡献的个人和集体，均已在文中以明确方式标明。本人在导师指导下所完成的学位论文及相关的职务作品，知识产权归属贵州大学。本人完全意识到本声明的法律责任由本人承担。

论文作者签名：杨宇瞻 日期：2024 年 6 月 13 日

关于学位论文使用授权的声明

本人完全了解贵州大学有关保留、使用学位论文的规定，同意学校保留或向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权贵州大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

本学位论文属于：

保 密 ()，在_____年解密后适用授权。

不保密 (✓)

(请在以上相应方框内打“√”)

论文作者签名：杨宇瞻 导师签名：蒋新忠

日期：2024 年 6 月 13 日